國立清華大學

資訊工程學系

專題研究成果報告

在 GPU 功耗上限下以尾數歸零提升科學計算效能

GH200 與 MI350X

Software Mantissa Zeroing on Power-Capped GPUs:
NVIDIA GH200 and AMD MI350X

學　　生： 陳克盈

學　　號： 112062205

指導教授： 周志遠 教授

中華民國 115 年 10 月

摘要

GPU 越來越常在功耗上限下運作：NVIDIA GH200 允許使用者設定整顆模組的上限，AMD MI350X 的矩陣乘法在出廠的 1000 W 上限下就有 82–100% 的時間撞上限。此時 GPU 會把時脈降到功耗不超過上限，程式的速度便取決於每次運算消耗的能量；而 GPU 的耗電隨資料而變，位元翻轉越多越耗電。另一方面，許多科學應用需要的精度遠少於 FP64 的 52 個尾數位元。

本專題研究「尾數歸零」：以軟體將運算元用不到的低位尾數設為 0，不改變資料格式，仍由原本的 FP64、BF16 或 FP8 運算單元計算，再於功耗上限下把省下的功耗換成更高的時脈。我們在 GH200 叢集與兩個各有 8 張 MI350X 的節點上，以成對交錯的量測評估矩陣乘法、HPL、HPL-MxP、Quantum ESPRESSO、DF-MP2、高斯過程地統計等程式。以全 1 取代 0 填入被捨去的位元（不再翻轉、但不為 0），保留了 65–97% 的效益；在數值與 1 的個數都固定時，只改變運算元的排列順序就讓每次運算的能量相差 8–9 個百分點，所以省電來自相鄰運算元之間變化的位元。據此建立的兩參數模型，只用原生的效能曲線、閒置功耗與一兩次量測，就能預測截斷版本在每個上限下的加速，矩陣乘法的平均誤差為 1.5 與 2.1 個百分點，兩個沒有參與建模的應用為 1.6。

MI350X 的預設上限就受限，所以截斷在出廠設定下就加速：FP64 HPL（保留 36 位有效位數）快 4.6–5.1%，HPL-MxP（對角優勢的測試矩陣，BF16 保留 3 位）快 4.9–5.1%，高斯過程快 7.4%，DF-MP2 快 10%；在 500 W 快 7–19%，所有正確性檢查都通過。GH200 的預設上限下主要表現為省電，受限時 HPL 快 3.3–4.7%。尾數歸零並非處處最佳：INT8 模擬 FP64（GEMMul8）在內積長、上限寬鬆時更快（DF-MP2 快 67%；HPL 在它本身的殘差門檻下 1000 W 快 34%），但內積短時比原生慢，需要的工作空間限制了問題大小，列內指數跨度寬時會損失精度（高斯基底 DFT 錯 52.5 mE_h）；把 HPL 的共同殘差門檻收緊到低於 HPL 本身的門檻，500 W 下測得的排序會反轉（截斷 +7%，模擬 –4%）。只用矩陣乘法量測擬合的逐次呼叫選擇規則，在沒有參與擬合的 DF-MP2 上 99% 的呼叫選對。兩種方法中，只有截斷保留原格式，也適用於 BF16 與 FP8。

註：本報告中所有截斷後的 HPL/HPL-MxP 與 MLPerf 數字皆為研究用途，並非 TOP500 或 MLPerf 的正式成績。

關鍵詞：功耗上限、能源效率、浮點精度、尾數截斷、GPU、INT8 模擬、HPL

Abstract

When a GPU runs at its power limit, the energy of each operation sets the clock. We lower that energy without changing the format: software zeroes the low-order mantissa bits an application does not need, and the original FP64, BF16 or FP8 units execute. On an NVIDIA GH200 cluster and two 8-GPU AMD MI350X nodes, filling the discarded bits with ones instead of zeros, which stops them toggling without making them zero, retains 65–97% of the saving: switching activity is the main contributor. A two-parameter model built on this predicts a truncated kernel's speedup under every cap from the native curve, the idle power and one or two measurements, within 1.5–2.1 percentage points on average for GEMMs and 1.6 for two applications not used to build it. MI350X is power-bound at its stock 1000 W limit, so truncation pays at default settings: 4.6–5.1% for FP64 HPL (36 significand bits), 4.9–5.1% for HPL-MxP on a diagonally dominant matrix (3 BF16 bits), 7.4% for a Gaussian-process likelihood and 10% for DF-MP2, and 7–19% at 500 W, with every accuracy check passed. INT8 emulation of FP64 (GEMMul8) is faster where inner dimensions are long and the cap generous (DF-MP2 +67%; HPL +34% under its own residual check at 1000 W), but slower than native at inner dimension 1024, limited by workspace memory and prone to precision loss on wide per-row exponent spans; tightening a common HPL residual threshold below the benchmark's own reverses the measured order at 500 W (+7 versus −4%). Of the two methods, only truncation keeps the format and applies to BF16 and FP8. Truncated HPL, HPL-MxP and MLPerf numbers in this report are research results, not official TOP500 or MLPerf scores.

Keywords: power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, INT8 emulation, HPL

重點摘要

結論：在功耗上限壓住時脈的 GPU 上，把運算元用不到的低位尾數歸零，可以讓以矩陣乘法為主、具正確性檢查的科學程式快數個百分點到近兩成，物理結果在各程式自己的判準內不變。GH200 的預設上限下主要是省電，MI350X 在出廠設定下就加速；收益可以事先預測，與 INT8 模擬互補。

四個主要發現：

1. **機制**（第 5.1、6.2 節）：填值保留 41–97% 的效益；數值固定時只改排列順序就差 8–9 個百分點：來自相鄰運算元之間變化的位元。
2. **預測**（第 4.6、6.4 節）：以閒置功耗為 P_0 的兩參數模型，矩陣乘法的平均誤差 1.5 ／ 2.1 個百分點，兩個外部應用 1.6。
3. **完整程式**（第五、六章）：收益與證據強度見下表。
4. **與改變格式的分界**（第 5.7、5.8、6.7 節）：勝負取決於內積長度、上限與容許誤差；逐次呼叫的選擇規則在保留的矩陣乘法案例中 MI350X 選對 98%、GH200 83–98%（依容許誤差），高斯過程 24 次中選對 22 次。

主要結果與證據強度

工作負載	操作點與正確性判準	GH200（8 節點或單節點）	MI350X（8 張 GPU）：1000 ／ 500 W	證據
HPL	36 位有效位數，最近捨入；殘差距門檻 35–100 倍	500–800 W：3.3–4.7%；1000 W 省電 7.5–8.6%	4.8 ± 0.3 ／ 7.0 ± 0.6%	GH200 6 對；MI350X 兩節點 6 次
HPL-MxP	BF16 保留 3 位；殘差通過	500 W：5.8%；600–800 W：1.7–2.3%	5.0 ± 0.4 ／ 19.3 ± 1.3%	對角優勢測試矩陣
Quantum ESPRESSO	依收斂門檻保留 16–42 位；總能量每一位不變	矽 2.1%；金表面 3.6%	未測	暖機後 6 對
DF-MP2	20 位；能量誤差 $\le 2.2 \times 10^{-8}\,E_\mathrm{h}$	500–700 W：7.4–7.7%	10.3 ± 0.6 ／ 18.6 ± 0.3%	GH200 6 輪；MI350X 4 輪
高斯過程	28 位；估出的參數不變	500–700 W：7.5–7.9%	7.4 ± 0.6 ／ 11.0 ± 1.0%	GH200 6 輪；MI350X 4 輪

區間為 95% t 區間。MI350X 以分析單位（整節點的一次執行或 8 張 GPU 的一輪）計算，兩節點合併。

其他結果：非矩陣乘法的第三方程式 MFEM 與 SW4 只快 0.3–2.0%，LAMMPS 落在雜訊內（第 5.1.2 節）；已使用 FP8 的 AI 上，LoRA 再截到 2 位反而需要多 22–39% 的樣本才達標，推論上 BF16 保留 3 位與 FP8 打平（附錄 C）。

主要限制（第 7.3 節）：兩個平台各一個世代，MI350X 只用單一節點；安全位數因平台而異；風扇轉速無法固定；功耗為裝置自己的讀數。

目錄

摘要 **i**

Abstract **ii**

重點摘要 **iii**

圖目錄 **vii**

表目錄 **viii**

符號與縮寫說明 **x**

第一章 緒論 **1**

1.1 研究背景：功耗成為算力的上限 1

1.2 功耗隨資料而變 1

1.3 應用用不到全部的尾數 1

1.4 研究問題 2

1.5 主要貢獻 2

1.6 報告架構 3

第二章 背景知識 **4**

2.1 浮點格式與尾數 4

2.2 GPU 功耗與位元翻轉 4

2.3 功耗上限與時脈 5

2.4 其他以降低精度換取速度的做法 5

2.5 本報告使用的基準與應用 6

第三章 相關研究 **7**

3.1 資料相依的 GPU 功耗 7

3.2 功耗受限的高效能計算 7

3.3 硬體層的尾數遮罩 7
3.4 精度調校與 FP64 模擬 8
3.5 地統計的混合精度 8
3.6 依收斂程度調整精度 9
3.7 本研究的定位 9

第四章 研究方法 **10**
4.1 尾數歸零與三種捨入方式 10
4.2 四種實作層次 10
4.3 實驗平台 12
4.4 量測與統計 12
4.5 等效功耗 ΔW 13
4.6 功耗上限下的效能模型 13
4.7 誤差模型 14

第五章 GH200 實驗結果 **15**
5.1 省電機制：位元不翻轉或數值為零 15
5.2 功耗上限下的矩陣乘法微基準 17
5.3 HPL 與 HPL-MxP：具正確性檢查的完整基準 18
5.4 科學軟體：Quantum ESPRESSO 與 PySCF 24
5.5 高功耗的真實應用：DF-MP2 26
5.6 跨度寬且功耗高的應用：高斯過程地統計 27
5.7 與 FP64 模擬及單精度的分界 30
5.8 指數跨度：只能以截斷處理的資料 34
5.9 已使用 FP8 的 AI 工作負載 36

第六章 第二平台：AMD MI350X **37**
6.1 平台、量測與統計單位 37
6.2 預設上限就受限：省電機制與時脈 38
6.3 功耗上限下的矩陣乘法 40
6.4 功耗模型的驗證 41
6.5 HPL 與 HPL-MxP 44
6.6 DF-MP2 與高斯過程 45
6.7 與 INT8 模擬（GEMMul8）的正面比較 47

6.8 逐次呼叫的選擇 51
6.9 量測陷阱：風扇與溫度 54
6.10 小結：兩個平台的異同 55

第七章 討論 **57**
7.1 適用條件 57
7.2 與改變格式做法的關係 57
7.3 限制與效度威脅 59
7.4 規則合規性 61

第八章 結論與未來工作 **62**
8.1 結論 62
8.2 未來工作 63

參考文獻 **65**

附錄 A 實驗環境、程式位置與重現方式 **70**
A.1 實驗環境 70
A.2 程式位置 70
A.3 重現方式 73

附錄 B 結果與 Slurm 工作對照 **74**

附錄 C AI 工作負載的細節 **77**
C.1 訓練：Llama 2 70B LoRA 微調 77
C.2 推論：BF16 截斷與 FP8 的比較 78

圖目錄

5.1 省電機制：被捨去的位元填 0、全 1 或固定樣式時的省電 16
5.2 非矩陣乘法核心在 1000 W 下的省電 16
5.3 矩陣乘法微基準在各功耗上限下的加速 18
5.4 調參後 8 節點 HPL 的效能與功耗上限 21
5.5 8 節點 HPL-MxP 在各功耗上限下的加速 22
5.6 Quantum ESPRESSO 金表面的總能量誤差與保留位數 24
5.7 方陣 FP64 矩陣乘法：截斷、INT8 模擬與單精度的比較 31
5.8 纈胺黴素的兩種真實矩陣形狀上截斷與模擬的速度 32
5.9 截斷與 INT8 模擬的分界點掃描 32
5.10 指數跨度對各方法逐元素誤差的影響 35
5.11 各應用的整體指數跨度與列內小元素比例 36

6.1 MI350X 矩陣乘法在各功耗上限下的加速 40
6.2 功耗模型預測與實測的矩陣乘法加速 41
6.3 MI350X 上四個應用在各上限下的加速 46
6.4 MI350X 上截斷與 GEMMul8 的分界 48
6.5 MI350X 上截斷與 GEMMul8 在相同精度下的比較 48

C.1 Llama 2 70B LoRA 的驗證損失：原生與截斷 2 位 77
C.2 六個語言模型在各格式下的困惑度增加 78

表目錄

2.1 本報告使用的浮點格式 4

3.1 本研究與相關研究的比較 9

4.1 各實驗的位數定義、去除低位的方式與樣本數 11

5.1 第三方非矩陣乘法程式的係數截斷結果 17
5.2 調參後 8 節點 HPL 在各功耗上限下的效能 20
5.3 HPL 縮放殘差隨保留位數與矩陣大小 N 的變化（門檻 16） 21
5.4 8 節點 HPL-MxP 在各功耗上限下的效能 23
5.5 科學軟體所需的尾數位數（攔截所有 FP64 矩陣乘法） 24
5.6 Quantum ESPRESSO 程式內截斷與自動調整位元數的時間與耗電 . . . 25
5.7 DF-MP2（紫杉醇，def2-TZVP）各變體的 MP2 時間與相對原生的加速 27
5.8 高斯過程對數概似評估的加速與偏差 28
5.9 截斷與 tile 混合精度疊加的效果 29
5.10 ExaGeoStatCPP 的 tile Cholesky 分解時間比較（ABBA 兩輪平均） . . 30
5.11 單精度做法的矩陣乘法相對誤差隨內積長度 k 的變化 34
5.12 各應用的列內指數跨度，以及截斷與 INT8 模擬的誤差 35

6.1 兩平台的填值對照 38
6.2 MI350X 上的位元變化對照 39
6.3 功耗模型的誤差 41
6.4 MI350X 上 HPL 的殘差 44
6.5 MI350X 的主要結果（兩節點、分析單位） 46
6.6 MI350X 上截斷與 GEMMul8 的正面比較 47
6.7 MI350X 上 HPL 對共同殘差門檻的敏感度 49
6.8 逐次呼叫選擇與固定選擇的比較 51
6.9 GH200 與 MI350X 的比較 56

7.1 各類工作負載在功耗受限時的收益 58

A.1 實驗環境與軟體版本 70
A.2 程式與資料位置 70

B.1 各結果對應的 Slurm 工作 74
B.2 MI350X 各結果對應的紀錄 75

C.1 BF16 最近捨入保留 3 位與 FP8 在 FP8 容易出問題的場景下的比較 . . 78

符號與縮寫說明

縮寫／符號	說明
3xTF32	把每個 FP32 數拆成兩個 TF32、以三次 tensor core 乘法近似 FP32 精度的做法
ABBA	成對交錯的執行順序（A、B、B、A），用來消除溫度與先後順序的影響
BF16、FP16、FP8	16 位元的 bfloat16、16 位元的 IEEE 半精度、8 位元浮點（本報告指 E4M3）
cuBLAS	NVIDIA 的 GPU 線性代數函式庫；本研究使用的第 13 版內建 FP64 矩陣乘法的 INT8 模擬
DFT	密度泛函理論（density functional theory）
DF-MP2	密度擬合的二階 Møller–Plesset 微擾理論
E_h、Ry	原子單位制的能量單位 Hartree 與 Rydberg（1 E_h = 2 Ry）
FP64、FP32、TF32	IEEE 雙精度、IEEE 單精度，以及 tensor core 使用的 19 位元 TF32 格式
GEMM、DGEMM	一般矩陣乘法；FP64 的一般矩陣乘法
GEMMul8	以中國餘數定理實作 Ozaki 第二版方法的 INT8 模擬函式庫
GH200	NVIDIA Grace Hopper superchip（Grace CPU 與 Hopper GPU 封裝在同一模組）
HBM	高頻寬記憶體（high-bandwidth memory）
HPL、HPL-MxP	TOP500 排名所用的 FP64 Linpack 基準；以低精度分解加迭代修正的混合精度 Linpack 基準
INT8 模擬	把 FP64 數拆成多片 8 位元整數、以整數 tensor core 計算 FP64 矩陣乘法的做法（Ozaki 方法）
LoRA	低秩適配（low-rank adaptation）微調
MLE	最大概似估計（maximum likelihood estimation）
SCF	自洽場（self-consistent field）迭代
SM	串流多處理器（streaming multiprocessor）；本報告的「時脈」皆指 SM 時脈
k	保留的尾數位數；HPL 的位數依 rocHPL 慣例為含隱藏位的有效位數
ΔW	等效功耗：未修改的程式要追上尾數歸零版本，每個節點需要多給的功耗上限（第 4.5 節）
α	矩陣乘法誤差模型的係數（第 4.7 節）
a	電路的切換活動率（式 (1.1)）

第一章　緒論

1.1　研究背景：功耗成為算力的上限

大型計算系統的電力與散熱都有預算。機房依機櫃的供電能力設定功耗上限，學生超算競賽（Student Cluster Competition, SCC）限制整個隊伍的總功率，NVIDIA GH200 superchip 也允許使用者直接設定整顆模組（GPU、CPU 與記憶體）的功耗上限（本叢集的驅動程式接受 200–1000 W，見第 4.3 節）。AMD Instinct MI350X 的出廠封裝上限是 1000 W，而它自己的 FP64、BF16 與 FP8 矩陣乘法在這個上限下有 82–100% 的時間撞上限（第 6.2 節）。當程式的耗電碰到上限時，GPU 的功耗管理會降低時脈與電壓，直到功耗不超過上限。因此在功耗受限時，效能不再由硬體的最高時脈決定，而是由每次運算所消耗的能量決定：每次運算的耗電越低，時脈便能拉得越高。

1.2　功耗隨資料而變

CMOS 電路的動態功耗約為

$$P_{\mathrm{dyn}} \approx a\, C\, V^2 f, \tag{1.1}$$

其中 C 為負載電容、V 為電壓、f 為時脈，a 為切換活動率（本報告的 α 另指第 4.7 節的誤差係數）：資料位元在相鄰時脈之間翻轉（0 變 1 或 1 變 0）得越多，電路充放電的次數越多，耗電就越高。因此同一個程式、同樣的運算量，輸入資料不同，耗電也不同。前人已在 A100 與 GH200 上量到，同樣的矩陣乘法，隨機輸入比固定值或全零輸入多耗 60–67% 的功率（見第三章）。

1.3　應用用不到全部的尾數

FP64 有 52 個尾數位元，約相當於 16 位十進位有效數字，但許多應用並不需要這麼高的精度：HPL 只要求殘差小於門檻；平面波密度泛函計算的自洽迭代在前期

只需要很寬鬆的精度；AI 推論甚至以 FP8（只有 3 個尾數位元）也能維持準確度。這些用不到的低位元在運算中仍持續隨機翻轉，所消耗的電力並未帶來有用的精度。

1.4　研究問題

綜合以上三點，本專題的核心想法是：**以軟體將用不到的低位尾數設為 0，使其不再翻轉，並在功耗上限下把省下的功耗轉換為效能**。本專題要回答以下四個問題：

1. **機制：**為什麼少幾個尾數位元就會省電？省電來自位元不再翻轉，還是來自數值為 0？
2. **效益：**在功耗上限下，真實的科學應用能換得多少效能與省電？需要付出多少精度？
3. **預測：**能否不必在每個上限下量測每個截斷版本，就預測它的收益？
4. **適用範圍：**與直接改用較小格式的做法（INT8 模擬 FP64、FP32/3xTF32、FP8）相比，尾數歸零在哪些情況下有價值？結論能否移植到另一家廠商的 GPU？

1.5　主要貢獻

1. **收益從哪裡來、能預測多少**。以全 1 或固定樣式取代 0 填入被捨去的位元，在兩家廠商的矩陣引擎、FP64 向量單元與記憶體搬移上都保留了 41–97% 的效益；在數值與 1 的個數都固定時，只改變運算元的排列順序，每次運算的能量就差 8–9 個百分點。把相鄰運算元之間的位元翻轉分成「跨列」與「沿 k」兩個方向計數，可以預測沒有參與擬合的排列的能量（留一法 $R^2 = 0.93$；只算翻轉總數 0.79、1 的個數 0.34）。以實測閒置功耗為 P_0 的一階模型，若每個截斷版本鎖頻量一次來校正，對兩個平台各 42 個「上限 × 格式」的點平均誤差為 1.5 與 2.1 個百分點（最大 4.6 與 10.4），對兩個沒有參與建模的應用平均 1.6（最大 3.6）；若只用合成運算元的位元翻轉來校正，矩陣乘法的加速平均誤差 1.2 個百分點，但這只檢驗了三個預測的 R，套到 DF-MP2 則高估 4.4。若改用應用自己的運算元在鎖頻下量一次矩陣乘法、再乘上它佔動態能量的比例，對兩個應用的預測平均誤差 1.2（整個程式鎖頻校正為 1.0），完全不必執行截斷版本。

2. **在真實程式中、以各自的正確性判準成立的收益**。截斷實作於程式內，與未修改的程式成對比較，並在第二組節點或第二個節點上重複：HPL、HPL-MxP、Quantum ESPRESSO、DF-MP2 與高斯過程地統計都在各自的判準（殘差、總能量、估計參數）內加速；MI350X 在出廠的預設上限下就快 4.6–10%；HPL-MxP 除了對角優勢的測試矩陣，也在非對角優勢的矩陣上驗證（3 位仍快 5.0%）。安全的保留位數必須在每個平台上以多組問題大小驗證。

3. **何時該改用 INT8 模擬，逐次呼叫決定**。在相同精度下，模擬在內積長、上限寬鬆時較快，截斷在內積短、上限嚴格、容許誤差嚴格時較快；記憶體吃緊與列內指數跨度寬也有利於截斷，而兩種方法中只有截斷適用於 BF16 與 FP8。只用矩陣乘法量測擬合的逐次呼叫選擇規則，在保留下來的 MI350X 矩陣乘法案例中 97.7% 選對（永遠模擬或永遠截斷：52% 與 48%）；在 GH200 上重新擬合，容許誤差 10^{-9} 與 10^{-11} 時分別選對 98.3% 與 83.3%（永遠截斷：60% 與 80%）。DF-MP2 有 99.3% 的呼叫都是模擬最快，規則與之相符；在區塊大小讓交叉點隨上限移動的高斯過程上，24 次執行選對 22 次，任一固定選擇只對 12 次。

另有兩項量測上的觀察支撐這些結果，而不是額外的貢獻：硬體能量累加器避免了取樣偏差；依前一次執行調整的機箱風扇，會透過溫度讓受限的 GPU 速度改變多達 4.5%，交錯順序無法消除。

1.6　報告架構

第二章介紹浮點格式、GPU 功耗與功耗上限等背景知識；第三章整理相關研究；第四章說明研究方法、實驗平台與功耗上限下的效能模型；第五章呈現 GH200 上的實驗結果，分為省電機制與效益上界、具正確性檢查的完整程式（HPL、HPL-MxP、Quantum ESPRESSO、DF-MP2、高斯過程地統計），以及與改變格式做法的分界三部分；第六章在第二個平台 AMD MI350X 上重做主要實驗、驗證效能模型，並與 INT8 模擬正面比較；第七章討論適用範圍與限制；第八章為結論與未來工作。附錄 A 列出實驗環境、程式位置與重現方式，附錄 B 列出各結果對應的工作，附錄 C 是 AI 工作負載的細節。

第二章　背景知識

2.1　浮點格式與尾數

一個浮點數由符號 s、指數 e 與尾數（fraction）f 三部分組成，正規數的數值為

$$x = (-1)^s \times 1.f \times 2^{e-\text{bias}}. \tag{2.1}$$

指數決定值域（能表示多大與多小的數），尾數決定精度（有效數字的位數）。本報告用到的格式列於表 2.1。

表 2.1　本報告使用的浮點格式

格式	符號／指數／尾數位元	約略的十進位有效位數	常見用途
FP64	1/11/52	16	科學計算、HPL
FP32	1/8/23	7	一般單精度計算
TF32	1/8/10	3	tensor core 的 FP32 加速模式
BF16	1/8/7	2–3	AI 訓練與推論
FP8 E4M3	1/4/3	1	AI 推論與訓練

本報告所稱「保留 k 位」，是指保留最高的 k 個尾數位元、將其餘位元設為 0。例如 FP64 保留 36 位，即把 52 個尾數位元中最低的 16 個清為 0（HPL 依 rocHPL 的慣例改用含隱藏位的有效位數，見第 4.1 節）。符號與指數不變，所以值域不變，只是每個值的精度變粗：直接截斷時，每個值的相對誤差不超過 2^{-k}；最近捨入時不超過 $2^{-(k+1)}$。資料格式與程式介面完全不需更動，這是它與「改用較小格式」最大的不同。

2.2　GPU 功耗與位元翻轉

依式 (1.1)，GPU 的動態功耗與切換活動率成正比。資料在暫存器、運算元匯流排、乘法器與記憶體之間流動時，只要某個位元與前一筆資料不同，對應的線路就要充電或放電。隨機資料的低位尾數，相鄰兩筆之間約有一半的位元會翻轉；若把它們全部設為 0，相鄰兩筆資料在這些位置便永遠相同，這些線路就不再切換。在

乘法器中，一個運算元的低位為 0 時，對應的部分積恆為 0，這部分電路也就不再切換。記憶體方面，HBM 與 DRAM 的存取能耗同樣隨資料的翻轉率變化。

2.3　功耗上限與時脈

GH200 的模組功耗上限涵蓋 GPU、Grace CPU 與記憶體。GPU 會持續調整時脈，使功耗不超過上限（實測撞上限時的 1 秒平均功耗常低於上限，限電器依據的應是更短時間尺度的功耗）。因此會出現兩種情況：

- **功耗受限**（例如上限 500 W）：每次運算的耗電降低後，GPU 能在相同功耗下以更高的時脈運作，效果表現為**加速**。
- **不受限**（上限高於程式的用電，時脈停在最高的 1980 MHz）：速度不變，效果表現為**省電**。

1000 W 是 GH200 的預設值，也是最高上限；但功耗很高的程式（例如 FP64 矩陣乘法與 HPL）在 1000 W 下仍有部分時間撞上限，因此兩種效果會同時出現。

AMD MI350X 的上限則是每張 GPU 的封裝功耗（PPT），不含 CPU，預設與最高值同樣是 1000 W。不同的是，MI350X 的矩陣乘法在預設上限下大部分時間都撞上限，所以截斷在出廠設定下就表現為加速（第 6.2 節）。MI350X 也允許鎖定最高時脈，使我們能在完全不受功耗限制的情況下比較每次運算的能量。

2.4　其他以降低精度換取速度的做法

INT8 模擬 FP64（Ozaki 方法）：把 FP64 數拆成多片低精度整數，以 INT8 tensor core 分別相乘後再合併，片數越多越精確。cuBLAS 13 內建此類模擬；GEMMul8 實作第二版 Ozaki 方法，以中國餘數定理把數值拆到多個模數上計算。

FP32 與 3xTF32：以單精度取代 FP64 計算。3xTF32 把每個 FP32 數拆成兩個 TF32，以三次 tensor core 乘法達到接近 FP32 的精度。

FP8：Hopper 上 AI 推論的主流格式，資料寬度為 BF16 的一半，矩陣乘法的峰值是 BF16 的兩倍，以逐張量或逐通道的縮放因子彌補較窄的值域。

混合精度迭代修正：以低精度完成 LU 分解，再以 FP64 迭代修正到高精度，混合精度的 HPL-MxP 基準就屬於這一類。

這些做法都改變資料格式，依靠吞吐量更高的運算單元加速；尾數歸零則沿用原本的運算單元，只降低輸入的翻轉。兩者各有適用場合，第 5.7 節將比較兩者的分界。

2.5　本報告使用的基準與應用

HPL：求解稠密線性系統的 FP64 基準，為 TOP500 排名的依據。最後會檢查縮放後的殘差是否小於 16，超過即判定失敗。

HPL-MxP：以 BF16 完成 LU 分解、再以 FP64 迭代修正的混合精度基準。

Quantum ESPRESSO：平面波密度泛函理論軟體，以自洽場（SCF）迭代求基態，每一圈以 Davidson 方法對角化，收斂門檻逐圈收緊。

PySCF（GPU4PySCF）：高斯基底的量子化學軟體，本報告用於 DFT、HF+MP2 與密度擬合 MP2（DF-MP2）。

高斯過程地統計：自寫的 ExaGeoStat 式最大概似估計核心，以及 ExaGeoStatCPP（StarPU 與 Chameleon 的 tile Cholesky）。

MI350X 上的版本：GPU4PySCF 不能在 MI350X 上執行，DF-MP2 改以 PySCF 在 CPU 上建立三指標張量、再以自寫的 HIP 程式收縮；高斯過程的似然函數移植為 HIP 程式（rocBLAS ／ rocSOLVER）；HPL 為 rocHPL，HPL-MxP 與 GH200 使用同一份原始碼。

MFEM、SW4、LAMMPS：不以矩陣乘法為主的 FP64 程式，分別為有限元素、地震波傳播與分子動力學。

MLPerf Llama 2 70B LoRA 與六個語言模型：AI 訓練與推論。

第三章　相關研究

3.1　資料相依的 GPU 功耗

Bhalachandra 等人在 A100 上發現，同樣大小的 DGEMM，隨機運算元比固定值多耗 67% 的功率 [1]。Gregersen 等人在 A100、H100 與 V100 上改變 GEMM 輸入的分佈與位元樣式，功耗最多相差近 40%，其中把最低位元歸零可以降低功耗；但他們刻意選擇不會降頻的矩陣大小，沒有量測功耗上限下的效能 [2]。He 在 A100 上示範全零輸入的矩陣乘法比隨機輸入快約 15%，並把功耗上限從 330 W 逐步降到 100 W：可預測輸入的加速隨上限降低而變大（圖中約從 1.2 倍增加到 150 W 時的 2.3 倍），到 100 W 才轉而變小 [3]。Antepara 等人在 GH200 上量到零輸入的 DGEMM 只用 563 W，隨機輸入則用 898 W [4]。Lucas 與 Juurlink 建立的 GPU ALU 功耗模型中，主項是位元翻轉數 [5]。這些工作都以全零、常數等特殊輸入示範此現象，沒有截斷真實資料的部分尾數，也沒有評估數值精度或真實應用。

3.2　功耗受限的高效能計算

Rountree 等人最早在硬體強制的功耗上限下量測效能，發現同型處理器在同一上限下的執行時間會因製程差異而不同（約 6%） [6]；Inadomi 等人在大型系統上量到，同一功耗上限下各節點的 DGEMM 效能可相差 64%，並依此變異分配功耗 [7]。Patki 等人在固定的總功耗下配置更多節點，再以功耗上限分配電力（超量配置）[8]；Zeus 在 GPU 上依功耗上限與批次大小調整 DNN 訓練的能耗 [9]。這些工作調整的是功耗如何分配，本專題則降低每次運算所需的能量，兩者可以疊加。我們量到同一上限下節點間 13–17% 的時脈差異（第 4.4 節），也與上述觀察一致。

3.3　硬體層的尾數遮罩

以「把尾數設為 0 來省電」在硬體設計上早有前例：Tong 等人的可變位寬浮點乘法器，每次運算的能量最多可減少 66% [10]；Pool 等人為行動 GPU 的著色器選擇

精度，以能耗模型估計頂點轉換可省 23–36% 的能量 [11]，像素著色器的算術最多可省 71% [12]；Guenther 等人在基頻處理器運算單元的輸入端遮罩尾數，最多省能 23% [13]；AMD 的專利在 GPU 著色器中加入硬體歸零模組 [14]。這些做法都需要新硬體，或以模擬器估算能耗。資料搬移方面，DRAM 與 HBM 的能耗隨資料的翻轉率變化 [15–17]。

3.4　精度調校與 FP64 模擬

Precimonious 等工具自動把變數換成較低精度的型別，目標是提升速度 [18]；rpe 與 chop 以同樣的位元操作模擬低精度，但不量測功耗 [19, 20]。Transprecision computing 主張依需求在時間與空間上細粒度地調整精度 [21]；Klöwer 等人從資料本身估計真正承載資訊的尾數位數，再以最近捨入去除其餘位元，用於壓縮氣候資料 [22]。Ozaki 方法 [23] 可以在整數矩陣單元上模擬 FP64 [24, 25]，Uchino 等人評估了它的效能與功耗 [26]。cuBLAS 內建的定點模擬讓 A 的每一列共用一個縮放因子，列內的動態範圍必須由尾數位元承擔；它的自動模式依指數跨度決定切片數，所需位元超過上限時改回原生 FP64 [27,28]。形式上最接近本專題的是 Grützmacher 等人：他們把 FP64 值依尾數切段存放、保留完整指數，只讀取前面的段落，使受頻寬限制的 PageRank 快 10–30% [29]；他們截斷是為了節省頻寬，本專題則是為了降低運算的切換功耗。

3.5　地統計的混合精度

ExaGeoStat 以 FP64 的 tile Cholesky 分解計算高斯過程的概似 [30]。之後的混合精度版本先把位置依空間填充曲線排序，使強相關集中在對角線附近；對角 tile 保留 FP64，再依離對角線的距離或 tile 的範數，把整個 tile 轉成 FP32、FP16 或 FP8：FP64/FP32 平均快 1.6 倍 [31]，加上 FP16 最多快 2.64 倍 [32]，結合低秩近似最多快 12 倍 [33]，在 GH200 上以四種精度比 FP64 快 3 倍 [34]，也用於 GH200 系統上的氣候模擬器 [35]。Cao 等人在單張 V100、A100 與 H100 上量測過這類分解的功耗與能耗 [36]，但沒有設定功耗上限，也沒有分析時脈。單一 FP32 不足以計算這類概似 [32]，與第 5.6 節的結果一致。這些做法都是把整個 tile 換成其他格式；本專題保留 FP64、只截斷運算元的尾數，兩者可以疊加。GPU 上的 RI-MP2 也曾以單精度與混合精度 GEMM 加速 [37]。

3.6　依收斂程度調整精度

Woo 等人在 LOBPCG 特徵值求解器中，依收斂狀態在 FP32、FP64 與混合精度之間切換 [38]；Huang 等人在 GPU4PySCF 中依 SCF 能量的變化選擇 INT8 模擬的切片數 [39]；Dawson 等人發現量子化學的密度矩陣純化約需 37 位有效尾數即已足夠 [40]。這些工作都是改用吞吐量較高的格式來加速，並未研究功耗。

3.7　本研究的定位

表 3.1 比較本研究與相關研究。據我們所知，本研究是第一個在兩家廠商的現役 GPU（NVIDIA GH200 與 AMD MI350X）上，以純軟體截斷 FP64/BF16 運算元的尾數來降低切換功耗、在功耗上限下把省下的功耗轉換為吞吐量，並在真實 HPC 應用上以物理誤差驗證的系統性研究。在 FP64 內「保留指數、捨去尾數」本身並不新 [29]，地統計的混合精度也已有人量測過功耗 [36]，全零輸入在不同功耗上限下的加速也已有人量測過 [3]；本專題的不同在於目的（降低切換功耗）、做法（不改變格式）、量測條件（功耗上限下的時脈與吞吐量），以及一個能從原生曲線預測截斷收益的模型。

表 3.1　本研究與相關研究的比較

工作	新硬體	截斷真實資料	在功耗上限下量測效能	驗證應用的數值結果
Bhalachandra、Gregersen、Antepara	否	否（全零等樣式）	否	否
He	否	否	是（330 → 100 W 掃描，全零輸入）	否
Tong、Pool、Guenther、AMD	是	是	否	部分
Woo、Huang	否	改變格式	否	是
Grützmacher（尾數切段儲存）	否	是（保留指數、捨去尾數，目的為節省頻寬）	否	是（PageRank）
ExaGeoStat 系列（Abdulah、Cao、Ren）	否	改變格式（整個 tile）	否（Cao 等人量過功耗，未設上限）	是
本研究	**否**	**是**	**是（兩個平台，含多 GPU 的 HPL）**	**是**

第四章　研究方法

4.1　尾數歸零與三種捨入方式

對一個有 p 個尾數位元的運算元（FP64 為 52、BF16 為 7、FP8 E4M3 為 3），保留最高的 k 個位元，其餘 $p - k$ 個設為 0。本研究比較三種去除低位的方式：

截斷：直接以位元遮罩清除低位，即往 0 的方向捨去。

最近捨入：先加上保留的最低位權重的一半（即被捨去部分的最高位），再以遮罩清除。攔截工具、GEMM 微基準、Quantum ESPRESSO、DF-MP2 與高斯過程的實作在平手時往離零較遠的方向捨入；HPL、LoRA 的權重載入與語言模型推論的實作則是平手取偶。

隨機捨入：先加上一個小於被捨去位元權重的隨機整數，再以遮罩清除。

每個元素只需一次整數 AND 運算（最近捨入再多一次加法，隨機捨入另需產生亂數），可以融入產生該運算元的核心；本報告的 HPL 另外執行一個捨入核心、攔截工具另外做一次複製，這些額外成本都計入量到的時間。本報告所稱「截斷到 k 位」泛指保留 k 位；除非特別註明，應用程式（Quantum ESPRESSO、PySCF、DF-MP2、高斯過程、ExaGeoStat 與第三方程式）與 HPL 使用最近捨入，GEMM 微基準、分界點掃描與 HPL-MxP 使用往零截斷。HPL 的位數依 rocHPL 的慣例為含隱藏位的有效位數：36 位相當於保留 35 個尾數位元。各實驗的位數定義、去除低位的方式與樣本數整理於表 4.1。

4.2　四種實作層次

1. **在基準程式中歸零**。矩陣乘法微基準在產生輸入時歸零；HPL 在每次 FP64 GEMM 之前以一個捨入核心處理兩個輸入（執行時以環境變數指定位數）；HPL-MxP 在編譯時指定位數，每次寫回 BF16 時歸零。
2. **攔截函式庫**。對不修改原始碼的科學軟體，以 `LD_PRELOAD` 攔下 cuBLAS 的所有 FP64 與複數 FP64 矩陣乘法入口，把 A、B 複製並捨入後再呼叫原函式（呼

表 4.1　各實驗的位數定義、去除低位的方式與樣本數

實驗	k 的意義	去除低位的方式	樣本
GEMM 微基準、機制對照、分界點掃描	尾數位元	往零截斷	微基準 3 節點 × 2 對（95% 區間）；機制 1000 W 每個組合 2–4 次；分界點掃描每格兩輪
HPL（8 節點）	有效位數，含隱藏位（36 = 35 個尾數位元）	最近捨入（平手取偶）	兩組節點，每個上限 2 + 4 對（第二組先暖機），95% 區間
HPL-MxP（8 節點）	BF16 尾數位元	往零截斷	兩組節點，每個上限 2 + 4 對（第二組先暖機），95% 區間
Quantum ESPRESSO、PySCF、DF-MP2、高斯過程、ExaGeoStat、MFEM、SW4、LAMMPS	尾數位元	最近捨入（平手離零）	QE 先暖機一次再交錯 6 對；DF-MP2 與高斯過程每格 6 輪；以上附 95% 區間；其餘多為每格兩輪（位數掃描與攔截版的效能對照各一次）
LoRA 訓練	FP8 尾數位元	往零截斷（另測最近捨入與隨機捨入）	兩個 seed、四組配對
語言模型推論	BF16/FP8 尾數位元	最近捨入（平手取偶）	六個模型，各一次

叫端的資料不變）。同一個函式庫也能統計運算元的指數分佈、改用 cuBLAS 的 FP64 模擬，或改以 FP32/3xTF32 計算，用於公平比較。

3. **程式內改寫**。在 Quantum ESPRESSO 的 Davidson 對角化中，對波函數區塊原地歸零，不需要複製；第三方非矩陣乘法程式（MFEM、SW4、LAMMPS）在設定階段對儲存的係數陣列歸零一次；GPU4PySCF 的 DF-MP2 在三指標張量建好後原地歸零一次；高斯過程核心則在每個面板進行尾端更新之前原地歸零。

4. **依收斂門檻自動調整**。Davidson 每一圈會收到一個收斂門檻 ethr，在整個 SCF 過程中從約 10^{-2} 收緊到約 10^{-13}。我們保留

$$k = \min(52,\ \max(16,\ \lceil -\log_2 \mathrm{ethr} \rceil + m)) \tag{4.1}$$

個位元，其中 m 為餘裕，使前期使用較少位元、最後幾圈才用足。這是一條經驗規則：ethr 是特徵值的絕對容許誤差（單位 Ry），與尾數的相對誤差不同，m 用來吸收兩者的差距；此規則不需要對應用建模，只使用程式本來就會計算的容許誤差。

LoRA 訓練則在載入後對凍結的 FP8 基底權重歸零一次；推論實驗在線性層中模擬各種格式，其中 FP8 使用真實的 FP8 tensor core 矩陣乘法。

4.3　實驗平台

實驗平台為 14 個節點的叢集，每個節點有一顆 NVIDIA GH200 480GB superchip（72 核 Grace CPU、Hopper GPU 搭配 96 GB HBM3，最高 SM 時脈 1980 MHz），以 200 Gb/s InfiniBand 連接，由 Slurm 管理。模組功耗上限可設為 200–1000 W（預設 1000 W）。規格表列出的可設範圍是 450–1000 W [41]；本報告低於 450 W 的設定超出規格表的範圍，但驅動程式接受，實測功耗也貼著上限。軟體為 NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）。多節點實驗（HPL、HPL-MxP、LoRA）使用 8 個節點。完整的軟體版本列於附錄 A.1。

第二個平台是兩個各有 8 張 AMD Instinct MI350X 的節點（ROCm 7.2.3），功耗上限為每張 GPU 的封裝功耗上限（1–1000 W，預設 1000 W），不含 CPU。它的平台細節、量測方式與統計單位與 GH200 不同，在第 6.1 節說明。

4.4　量測與統計

功耗：每 250–500 ms 以 nvidia-smi 取樣模組功耗、SM 時脈、溫度與降頻原因。耗電量以驅動程式的 1 秒平均模組功耗對時間積分（對忽高忽低的負載，單一瞬時讀數會有 −37% 到 +41% 的偏差）；表中的平均功耗是量測視窗內瞬時取樣的平均，穩態下與 1 秒平均相差約 1% 以內。

公平比較：每次量測都等 GPU 降到 45 °C 以下才開始；在同一功耗上限內，以交錯順序（A、B、B、A）在同一組節點上執行各變體，以消除溫度與先後順序的影響。

信賴區間：同一節點（多節點基準為同一組節點）、同一上限、同一輪的「歸零 ÷ 原生 − 1」為一個成對樣本，以 t 分布取 95% 區間，藉此消除節點間 13–17% 的時脈差異。矩陣乘法微基準為 3 個節點各 2 對；Quantum ESPRESSO 為先暖機後的 6 對；DF-MP2 與高斯過程為兩個工作共 6 輪；HPL 與 HPL-MxP 為兩組節點共 6 對（第一組 2 對、第二組先暖機後 4 對）。區間把同一工作或同一組節點內的各對當成獨立樣本，而它們並非完全獨立，所以區間可能偏窄。每個工作的第一次執行常偏慢（暖機），而交錯順序的第一個多半是原生，會使第一對的收益偏大，所以 Quantum ESPRESSO 的量測，以及 DF-MP2、高斯過程、HPL 與 HPL-MxP 的第二個工作，都

先執行一次不計入的暖機（第 5.4 節）。

MI350X：以硬體的能量累加器計算能量，信賴區間以分析單位（量測視窗、整節點的一次執行或 8 張 GPU 的一輪）計算，並以溫度迴歸與受控的熱歷史處理風扇造成的偏誤（第 6.1、6.9 節）。

資料檢查：每個量測視窗都記錄實際生效的功耗上限，與預定值不符者一律剔除（叢集為共用資源）；硬體功耗煞車與溫度降頻以降頻計數器判讀（在 HPL 的兩次掃描 3315、3582 與 HPL-MxP 的診斷工作 1675 中開啟），或以取樣時記錄的降頻原因判讀。

4.5　等效功耗 ΔW

在功耗上限 P 下，將尾數歸零版本的效能記為 $p'(P)$，原生版本的效能記為 $p(P)$。沿著原生版本實測的「效能對上限」曲線線性內插，找出原生版本達到 $p'(P)$ 所需的上限 P_{native}，並定義

$$\Delta W(P) = P_{\text{native}} - P, \qquad p(P_{\text{native}}) = p'(P). \tag{4.2}$$

其意義為：未修改的程式要追上尾數歸零版本，每個節點需要多給的功耗上限。這比百分比更直接地對應到機房的電力預算。上限之間以直線內插；由於效能對上限的曲線向下彎，兩個上限相隔較遠時（例如 800–1000 W），ΔW 會高估。

4.6　功耗上限下的效能模型

等效功耗需要在每個上限下量測每個截斷版本。以下的模型只用原生程式的效能曲線、平台的閒置功耗與一兩次量測，就能預測截斷版本在每個上限下的效能。設 GPU 在上限 P 下停在目標時脈 f，電壓 $V(f)$ 由目標時脈決定，但切換發生在有效時脈 sf，其中 $s \le 1$ 是隨資料而變的時脈降低（MI350X 上觀察到，第 6.2 節；沒有時 $s = 1$）。以 c 表示程式的等效切換電容、P_0 表示不隨切換變化的功耗，則

$$P = P_0 + c\,s\,V(f)^2 f = P_0 + c\,s\,g(f), \tag{4.3}$$

效能則與 sf 成正比。截斷改變 c 與 s，不改變 g。對原生（n）與截斷（r）版本定義

$$R = \frac{c_r s_r}{c_n s_n}, \qquad \sigma = \frac{s_r}{s_n}; \tag{4.4}$$

在相同的鎖定目標時脈下，R 就是兩者動態功耗之比，σ 是兩者有效時脈之比，都可以直接量測。在上限 P 下，截斷版本停在 $c_r s_r g(f_r) = P - P_0$ 的目標時脈 f_r；原生版本要達到同樣的 f_r，需要上限 P'，滿足 $c_n s_n g(f_r) = P' - P_0$。兩式相除得

$$P' = P_0 + \frac{P - P_0}{R}, \qquad p_r(P) = \sigma\, p_n(P'), \qquad \Delta W_f = P' - P = (P - P_0)\Big(\frac{1}{R} - 1\Big), \tag{4.5}$$

其中 $p_r(P) = \sigma\, p_n(P')$ 是因為兩者都在目標時脈 f_r 運作，而效能與 s 成正比。ΔW_f 是原生版本要達到截斷版本的**目標時脈**需要多給的上限；式 (4.2) 的等效功耗則是原生版本要達到截斷版本的**效能**需要多給的上限，在模型中為

$$\Delta W = p_n^{-1}(\sigma\, p_n(P')) - P, \tag{4.6}$$

只有在 $\sigma = 1$ 時（GH200，依假設）才等於 ΔW_f，$\sigma > 1$ 時（MI350X，$\sigma = 1.01$–1.05）比它大。本報告列出的 ΔW 都是沿實測的原生曲線內插得到的實測值，模型只用來預測加速。模型假設 s 在所用的時脈範圍內不隨 f 改變、程式維持計算受限；預測值不超過程式受時脈限制的峰值。

模型需要原生的效能曲線 $p_n(P)$ 與 P_0。我們把 P_0 設為實測的閒置功耗，與驗證資料無關：GH200 模組為 174 W（三個節點 169–179 W），MI350X 每張 GPU 為 262 W。截斷版本以 1300 MHz 的鎖頻量測取得 R 與 σ（MI350X，不使用上限掃描的任何一點），或以最低上限的速度推得 R（GH200 無法鎖頻，$\sigma = 1$，該點不列入評估）。驗證結果見第 6.4 節。式 (4.5) 中同時脈的部分 $\Delta W_f \propto P - P_0$ 也說明了為什麼實測的等效功耗隨上限增加，而相對加速並不一定如此。

4.7　誤差模型

兩個輸入都保留 k 位時，矩陣乘法結果的相對誤差（Frobenius 範數）約為

$$\frac{\|\widehat{C} - C\|_F}{\|C\|_F} \approx \alpha \cdot 2^{-k}, \tag{4.7}$$

與內積長度無關（累加仍為 FP64）。在均勻隨機資料上（$k \geq 4$），最近捨入的 $\alpha \approx 0.27$、隨機捨入 0.38、截斷 0.70，截斷的誤差約為最近捨入的 2.6 倍。截斷的誤差幾乎全是系統性的縮小，對會累積偏差的流程有害，這是科學軟體應採用最近捨入的原因（第 5.4 節）。

第五章　GH200 實驗結果

本章分三部分。第一部分（第 5.1、5.2 節）說明省電的機制與效益的上界；第二部分（第 5.3–5.6 節）是具正確性檢查的完整程式：HPL、HPL-MxP、Quantum ESPRESSO、DF-MP2 與高斯過程地統計；第三部分（第 5.7–5.9 節）是尾數歸零與改變格式做法（INT8 模擬、單精度、FP8）的分界。除非特別說明，百分比都是與同一功耗上限、同一節點上的原生版本比較。本章的結果都在 GH200 上量得；第二個平台 MI350X 的結果與兩平台的比較在第六章。

5.1　省電機制：位元不翻轉或數值為零

5.1.1　填值對照

尾數歸零同時改變兩件事：低位元不再隨資料翻轉，而且被捨去部分的數值變成 0。為了分開這兩種效應，我們把被捨去的位元改填全 1，或填入一組所有元素共用的固定非零樣式；這兩種填法同樣不會翻轉，但數值不是 0。我們在 1000 W 下量測三個層次：FP64 tensor core 矩陣乘法（每個組合 4 次）、只使用一般 FP64 運算單元的乘加迴圈，以及純記憶體搬移（後兩者各 2 次；圖 5.1）。保留 0 位的填值版本都以最高時脈 1980 MHz 執行，因此比較的是功耗；保留 20 位的填值版本也接近最高時脈（1963–1971 MHz），但有部分時間撞上限，功耗差會略為低估；原生 FP64 矩陣乘法仍有 95% 的時間撞上限、時脈只有 1822 MHz，其功耗是被上限截住的值，所以「不翻轉」所佔的比例會算得偏保守。

在這組量測中（1000 W 下每個組合 2–4 次），三個層次的方向一致：只要讓低位元不再翻轉，就取得完整省電的 62–97%；填 0 比填 1 或固定樣式再多省一些。記憶體搬移幾乎完全是「不翻轉」的效應（86–97%）；乘法器則還有額外的「數值為 0」效應：保留 20 位時，填 1 的低位與另一個運算元相乘的部分積仍會隨後者翻轉，填 0 則整片不動。保留 0 位時兩個運算元的尾數都是常數，部分積本來就不翻轉，但填 0 仍比填 1 省電（707 W 對 774 W），可能來自乘積的對齊與累加較單純（全 0 時乘積是 2 的冪），此點尚未驗證。在 500 W（功耗受限、比較效能；單一節點、各兩次）重做同樣的對照，矩陣乘法的「不翻轉」佔效益的 77–92%、乘加迴圈佔 57–85%，方

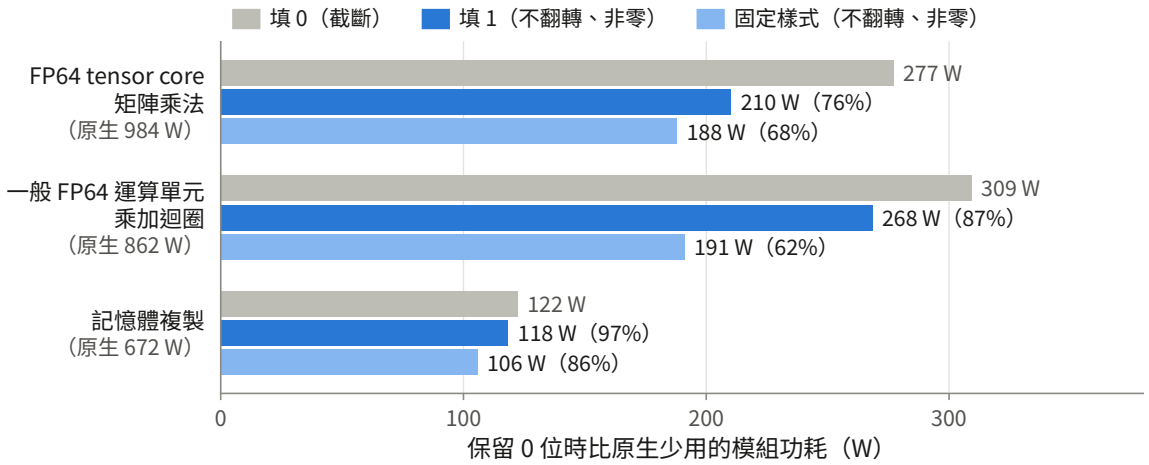


圖 5.1　省電機制：保留 0 位、被捨去的位元填 0、全 1 或固定樣式時，三個層次在 1000 W 下比原生少用的模組功耗。填 1 與固定樣式同樣不翻轉，但數值不為 0；括號內為相對填 0 的比例。
資料來源：Slurm 工作 1752–1753、1762、1815、1858；`gh200/results/m1_toggle_summary.csv` 等

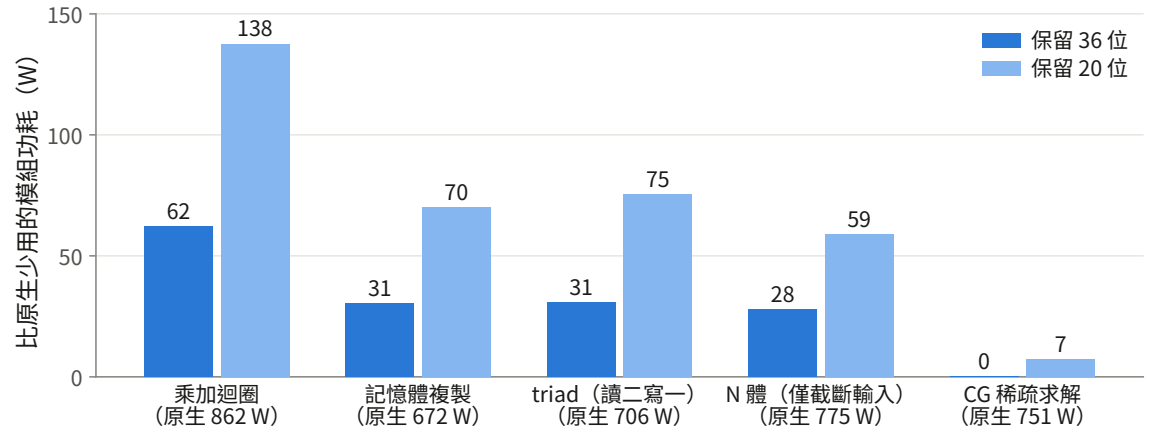


圖 5.2　非矩陣乘法核心在 1000 W 下保留 36 位與 20 位時，比原生減少的模組功耗。
資料來源：Slurm 工作 1815、1858、1860；`gh200/results/m4_*_summary.txt`

向相同。不過樣本很少（1000 W 每個組合 2–4 次、500 W 一個節點兩次，填值只做了保留 0 位與 20 位），這些比例應視為這組量測的估計，而非精確的分解。此結果在 GH200 的模組功耗上為前人提出的翻轉假說 [2] 提供了量化的支持；資料搬移的部分也與 DRAM 能耗隨翻轉率變化的觀察一致 [15, 17]。

5.1.2　不只是 tensor core：非矩陣乘法的 FP64 程式

為了確認省電機制不限於 tensor core，我們也測試不使用矩陣乘法的 FP64 程式；這也是 Ozaki 類模擬無法取代的部分，因為模擬只適用於稠密矩陣乘法。我們先在核心層級測試：只用一般 FP64 運算單元的乘加迴圈、記憶體複製與 triad、N 體重力計算，以及一個 CG 稀疏求解器（圖 5.2）。

在低功耗上限下，這些省電會轉成加速：乘加迴圈在 500 W 快 4.8–9.0%（36–20

表 5.1　第三方非矩陣乘法程式的係數截斷結果

軟體	被截斷的係數	係數佔記憶體流量	結果
MFEM（免組裝有限元素，3827 萬未知數）	每個求積點的幾何與係數資料	約 43%	450 W 快 0.9%（36 位）／2.0%（20 位），550 W 快 0.7%/1.4%；解的相對變化 $\le 2.3 \times 10^{-11}$
SW4（地震波，1.23 億格點）	材料參數 ρ、μ、λ	約 9%	350 W 快 0.6%（36 位）／1.3%（20 位），450 W 快 0.3%/0.5%；波形最大相對差 $\le 3.4 \times 10^{-6}$
LAMMPS（EAM 分子動力學）	位能表	約 0%（常駐快取）	各上限 0.0–0.4%，落在雜訊內（負面對照成立）

位），記憶體複製在 450 W 快 7.7–14.0%。但 N 體計算只快 1–4%：它第一步就是座標相減 $dx = x_j - x_i$，差值的尾數又是滿的，後面約 20 個運算照常翻轉。這說明效益隨「計算深度」遞減，只有直接讀取截斷資料的運算受惠。CG 幾乎沒有效益，因為被截斷的矩陣值只佔每圈記憶體流量的一部分，索引與向量不能截斷。

第三方真實軟體　為了確認核心層級的結果能否延伸到真實程式，我們在三個不以矩陣乘法為主的 FP64 GPU 程式中，於設定階段對「儲存後每一步都會被乘進去的係數陣列」歸零一次（不設定時與原版逐位元相同），結果列於表 5.1。

MFEM 的係數佔了近一半的記憶體流量、直接進入乘法器，收益仍只有 1–2%。三者的收益順序（MFEM ≥ SW4 > LAMMPS）與係數佔記憶體流量的比例一致。

小結　尾數歸零的省電不限於 tensor core，一般 FP64 運算單元與記憶體搬移也能省電；但在真實程式中，被截斷的資料只是耗電的一部分，核心層級在 450–500 W 的 4.8–14.0% 加速，在真實程式中被其餘滿尾數的運算稀釋到 0.3–2.0%。

5.2　功耗上限下的矩陣乘法微基準

我們先在最單純的工作負載上量出效益的上界。測試使用 cuBLAS 矩陣乘法（FP64 為 $32768 \times 32768 \times 2048$，BF16 為 $32768 \times 32768 \times 6144$），輸入是 $[-1, 1)$ 的均勻隨機值；在 3 個節點、300–1000 W 共八個上限下，每個上限以交錯順序各執行 2 次（圖 5.3）。

大多數信賴區間只有 ± 0.1–1.2 個百分點，效益遠大於量測雜訊。換算成等效功耗，FP64 保留 27 位在 500–800 W 相當於多給 79–162 W，BF16 保留 0 位相當於多給 121 W 到 200 W 以上。最佳能效也隨之提高：FP64 從原生的 82 GFLOP/J（500 W）

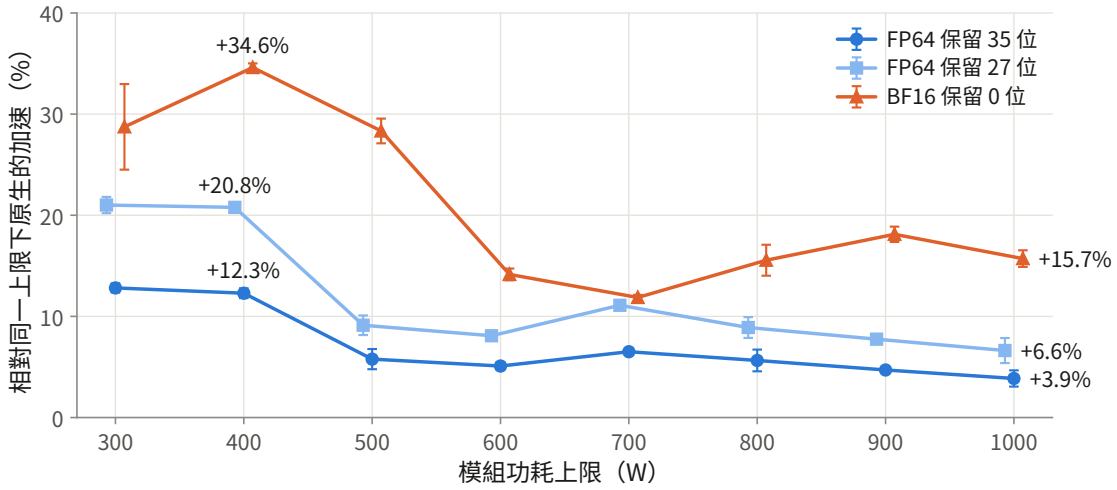


圖 5.3　FP64 與 BF16 矩陣乘法在各功耗上限下保留不同位數的加速（95% 信賴區間，3 個節點各 2 對；FP64 保留 27 位與 BF16 的點分別向左、右錯開約 7 W 以免重疊）。

資料來源：Slurm 工作 1667–1669（work1–3）；`gh200/results/e1_ci.csv`

升到保留 27 位的 90 GFLOP/J（400 W），BF16 從 950 GFLOP/J（600 W）升到保留 0 位的 1159 GFLOP/J（500 W）。效益與功耗上限並非單調關係，在 300–400 W 最大。一個可能的解釋（尚未驗證）是：上限極低時電壓已在下限，功耗大致與時脈成正比，省下的功耗能換到較多時脈；而在 1000 W 時，FP64 與 BF16 矩陣乘法仍受功耗上限限制，所以仍有 3.9%（FP64 保留 35 位）到 15.7%（BF16 保留 0 位）的加速。

需要強調的是，這些是均勻隨機資料、保留位數很低時的數字，屬於翻轉最劇烈的情況，代表效益的上界，而不是應用的操作點。下一節檢視具正確性檢查的完整程式。

5.3　HPL 與 HPL-MxP：具正確性檢查的完整基準

5.3.1　實驗設定與原生 HPL 的調校

HPL 的實驗使用其內建的量化選項：在每次 FP64 矩陣乘法之前，以一個捨入核心把兩個輸入捨入到 k 位（平手取偶的最近捨入），乘法本身仍是 FP64 cuBLAS。實驗使用 8 個節點、$N = 286720$（調參後的設定），在 400–1000 W 下比較原生與保留 36 位，並在兩組節點上各做一次功耗上限掃描。甲組（work4–11，工作 3315）每個上限依「原生、36 位、28 位、28 位、36 位、原生」的交錯順序執行，每個變體與原生配成兩對；乙組（work3–8、work12、work13，工作 3582）先執行一次不計入的暖機，再以「原生、36 位、36 位、原生」重複兩輪，每個上限 4 對。兩組合計每個上限 6 對。

為了確保比較的基準是最佳的原生效能，我們先調校原生 HPL。最早的量測沿用 H200 時期的設定（$N = 262144$、$NB = 1024$、2×4 網格、$f = 0.05$、NCCL 預設值），1000 W 下原生只有 374.8–376.0 TFLOP/s。我們在 1000 W 掃描 N、NB、$P \times Q$、f 與 NCCL 參數（共 24 組設定、26 次執行）：NCCL 固定 8 個通道與 2 MiB 緩衝貢獻約 4.5%，N 加大到 286720 再多約 2%，4×2 網格反而慢約 13%。同一個最佳設定重跑三次為 390.2–398.8 TFLOP/s（相差約 2%；其中 390.2 與 393.4 在同一個工作內，398.8 在另一個工作），f、NCCL 通道與緩衝、NB、N 的其他鄰近設定都沒有超出這個範圍（$f = 0.15$ 反而慢約 2%）。以下的功耗上限掃描都使用這個設定（各結果對應的 Slurm 工作列於附錄 B）。

乙組量測當天，多數節點的第二個 InfiniBand 埠只以 2.5 Gb/s 連線（第一個埠為 200 Gb/s），NCCL 與 MPI 會把部分流量放在這個埠上，使 8 節點 HPL 只剩 157 TFLOP/s。我們把兩者固定在第一個埠後重做；即使如此，乙組的原生在 1000 W 只有 365.5 TFLOP/s，比甲組低 7%，SM 時脈卻反而較高（1961 對 1936 MHz），表示有更多時間花在等待通訊。兩組之間的差異因此同時包含節點與網路狀態的不同；每一對都在同一個工作內配對，所以這不影響各組內的比較。

5.3.2　功耗上限下的效能

表 5.2 與圖 5.4 為兩組節點的結果。在規格表可設範圍（450–1000 W）內的 500–800 W，保留 36 位有效位數（含隱藏位，即 35 個尾數位元）在 6 對合併後快 3.3–4.7%（95% 區間為 $\pm$0.1–0.5 個百分點），每次求解的模組耗電少 4.1–4.6%；甲組快 4.0–5.0%，乙組快 3.0–4.6%。換算成等效功耗，500–700 W 相當於每個節點多給 42–74 W（甲組 42–68 W，乙組 49–74 W）。800 W 落在 800 與 1000 W 兩個量測點之間，只能直線內插；效能對上限的曲線向下彎，內插值（甲組 127 W、乙組 107 W）必然偏高，所以不列入範圍。

在 500–800 W，加速（3.0–5.0%）都略低於 SM 時脈的增加（4.5–7.0%），符合有一部分時間花在不隨時脈加快的通訊上；乙組的通訊比例較高，加速也較低。低於規格表下限的 400 W 則相反：加速 9.9 ± 1.0%（甲組 11.0%、乙組 9.3%），每次求解少耗電 8.5%，但時脈只多 6.3–6.6%。400 W 的額外收益來自時脈以外的因素，原因尚未查明，因此我們把它與 500–800 W 分開報告。

等效功耗與百分比的關係取決於原生曲線的斜率：上限低時曲線陡，多給 1 W 換到的效能多，所以 400 W 的 9.3–11.0% 只相當於 39–42 W；上限高時曲線變平，同樣幾個百分點的效能要多給更多瓦數才補得回來。因此不同上限的 ΔW 不能只看百

表 5.2　調參後 8 節點 HPL 在各功耗上限下的效能（$N = 286720$，兩組節點）

	原生（TFLOP/s）		保留 36 位		加速			ΔW		耗電
上限	甲	乙	甲	乙	甲 2 對	乙 4 對	6 對合併	甲	乙	6 對合併
400 W*	225.0	228.0	249.9	249.2	+11.0%	+9.3%	+9.9 ± 1.0%	39 W	42 W	−8.5 ± 0.7%
500 W	288.0	278.2	300.7	291.0	+4.4%	+4.6%	+4.5 ± 0.1%	42 W	49 W	−4.1 ± 0.2%
600 W	318.0	304.5	334.0	318.3	+5.0%	+4.5%	+4.7 ± 0.3%	57 W	58 W	−4.6 ± 0.5%
700 W	346.2	328.3	362.9	341.3	+4.8%	+3.9%	+4.2 ± 0.5%	68 W	74 W	−4.4 ± 0.4%
800 W	370.8	345.8	385.6	356.3	+4.0%	+3.0%	+3.3 ± 0.5%	(127 W)†	(107 W)†	−4.1 ± 0.5%
1000 W	393.9	365.5	409.2	367.5	+3.9%	+0.6%	+1.7 ± 2.0%	—	—	−8.3 ± 1.1%

甲組：work4–11（工作 3315），每個上限 2 對；乙組：work3–8、work12、work13（工作 3582，網路較慢，見第 5.3.1 節），先暖機一次，每個上限 4 對。效能為各組的平均；加速為各對「36 位 ÷ 原生 − 1」的平均，合併欄附 6 對的 95% t 區間（6 對分屬兩組節點，區間可能偏窄）；ΔW 為等效多出的上限，耗電為每次求解的模組耗電變化。「36 位」為含隱藏位的有效位數，即 35 個尾數位元。* 低於規格表的可設範圍（450–1000 W）。† 在 800 與 1000 W 之間直線內插，偏高，不列入範圍。耗電為驅動程式讀數的模組耗電。資料來源：`gh200/results/` 中的 `e3_tuned_summary.csv`、`e3_more_summary.csv` 與 `e3_pairs_ci.csv`。截斷後的數字為研究用途，並非正式成績。

分比比較。

量測雜訊方面，同一個工作內的配對非常一致：在 400–800 W，乙組每個上限的 4 對相差最多 0.4 個百分點，甲組的 2 對最多 0.6 個百分點。兩組節點之間的差異則較大：在 600–800 W，甲組比乙組多 0.5–0.9 個百分點，在 500 W 則少 0.2 個百分點。合併 6 對的區間把它們視為獨立樣本，而它們其實分屬兩組節點，所以這個區間仍可能偏窄；反過來說，只用一組節點上的兩對，會明顯低估不確定性。調參時同一設定的 390.2–398.8 TFLOP/s（約 2%）也主要是不同工作之間的差異。

在預設的 1000 W 上限下，兩組節點的速度結論不同，省電則一致。甲組的原生有 75% 的取樣撞上限（SM 平均 1936 MHz），而且有 15% 的取樣同時出現溫度降頻（保留 36 位為 5%；其他上限與乙組都沒有）；保留 36 位後撞上限的取樣只剩 7%（1977 MHz），快 3.9%（兩對為 2.7% 與 5.1%，差距較大的一對正是原生溫度降頻最多的一次）、每次求解少耗電 7.5%。所以甲組的 3.9% 有一部分來自較低的溫度，而不只是功耗上限。乙組的原生有 66% 的取樣撞上限，但 SM 平均已達 1961 MHz，離最高的 1980 MHz 只差 1%，所以保留 36 位只快 0.6%（4 對為 0.4–0.6%），每次求解則少耗電 8.6%。省下的功耗能不能變成速度，取決於原生的時脈被上限（以及溫度）壓低多少；每次求解少耗電則在兩組節點上都成立（6 對合併為 8.3 ± 1.1%）。未調參的設定（$N = 262144$）在 400–800 W 快 3.4–8.3%、1000 W 快 0.7% 並省電 7.7%，方向相同。乙組全程開啟降頻計數器：8 個節點都沒有觸發硬體功耗煞車，溫度降頻只有一個節點累計 2.5 秒。

依規格，每顆 GH200 的 FP64 tensor core 峰值為 67 TFLOP/s [41]，8 顆合計約 536 TFLOP/s。甲組調參後的原生 HPL 在 1000 W 達到約 73%（兩次平均

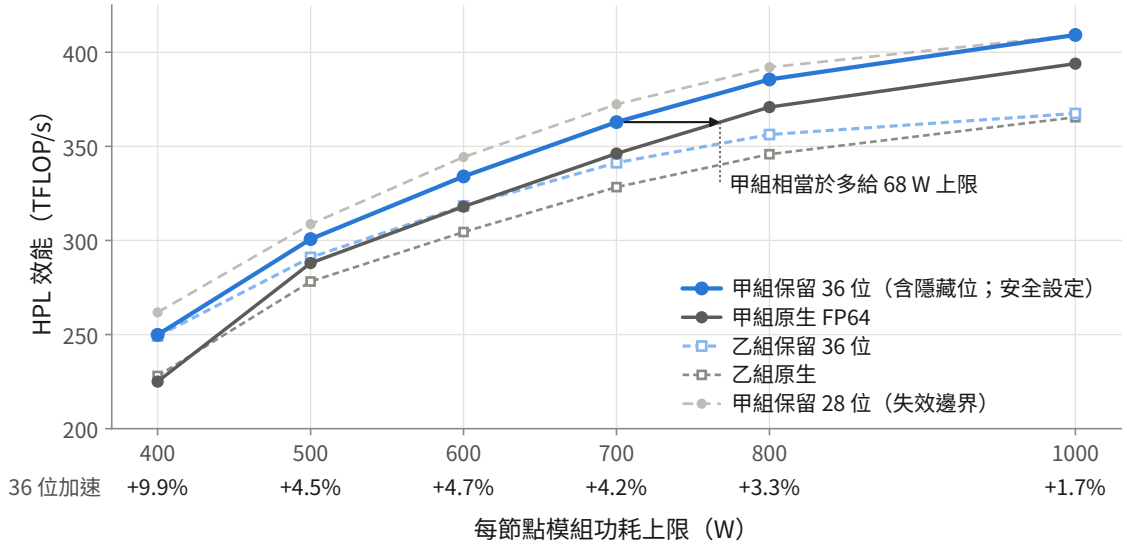


圖 5.4　調參後 8 節點 HPL 的效能與功耗上限：甲組（實線）與乙組（虛線）的原生與保留 36 位，以及甲組的保留 28 位。下方為兩組節點共 6 對的平均加速。保留 28 位的殘差已接近門檻，只作為失效邊界的參考。

資料來源：Slurm 工作 3315（甲組）、3582（乙組）；`gh200/results/` 中的 `e3_tuned_summary.csv`、`e3_more_summary.csv`、`e3_pairs_ci.csv`

表 5.3　HPL 縮放殘差隨保留位數與矩陣大小 N 的變化（門檻 16）

保留位數	$N = 131072$	$N = 196608$	$N = 262144$	$N = 286720$	判定
原生（53 位）	1.5×10^{-4}	1.3×10^{-4}	2.6×10^{-5}	7.6×10^{-5}	通過
36	0.16	0.072	0.042	0.038	通過，距門檻約百倍（最小 98 倍）
28	13.97	**18.09**	14.70	15.40	$N = 196608$ 失敗，其餘都接近門檻
24	456	462	91	—	全部失敗

393.9 TFLOP/s，調參時單次最高 398.8 TFLOP/s），保留 36 位達到 409.2 TFLOP/s（研究用途，並非正式成績）；乙組受網路影響，原生與保留 36 位分別只有 365.5 與 367.5 TFLOP/s。保留 28 位可以再多 0–5 個百分點（例如甲組 700 W 時 372.3 TFLOP/s），但它在這個 N 的殘差已達 15.4（門檻 16），在 $N = 196608$ 時更是失敗，所以不作為操作點。

5.3.3　殘差與失效邊界

HPL 最後會檢查縮放殘差是否小於 16。殘差會隨矩陣大小 N 變化，所以我們在 1000 W 以三組 N 檢查（每組 2 次，殘差完全重現），再加上調參後的 $N = 286720$（兩組功耗上限掃描中保留 36 位的 36 次與保留 28 位的 12 次，殘差也完全重現），結果列於表 5.3。

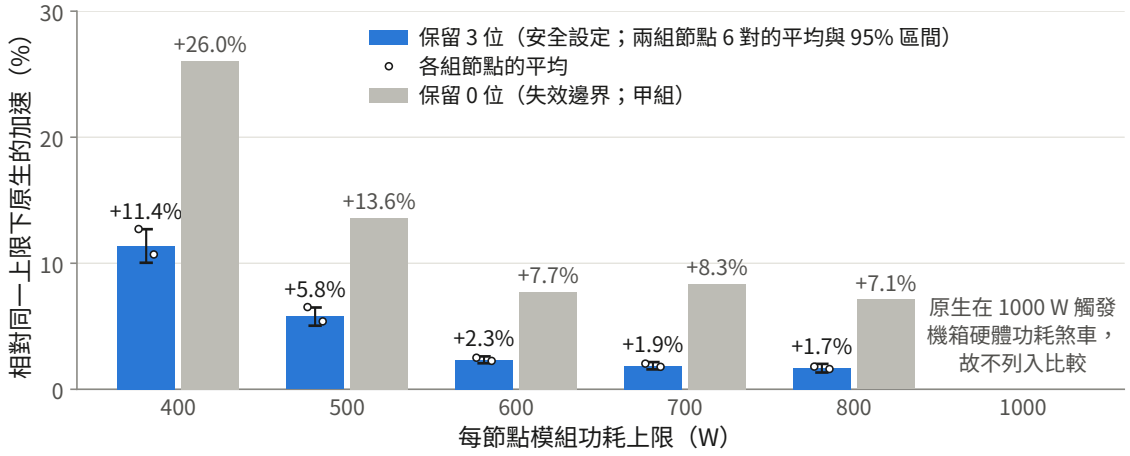


圖 5.5　8 節點 HPL-MxP 在各功耗上限下的加速：保留 3 位為兩組節點共 6 對的平均與 95% 區間（圓點為各組的平均），保留 0 位只有甲組（對角優勢的測試矩陣，規則只允許用於測試）。資料來源：Slurm 工作 1671（甲組）、3583（乙組）；`gh200/results/` 中的 `e2_summary.csv`、`e2_pairs_ci.csv`

從 36 位減到 28 位少了 8 位，殘差放大 86–410 倍，量級與 $2^8 = 256$ 相符。這表示 28 位距門檻不到 1 位，換一個 N 就可能失敗；36 位則有約 7 位的餘裕，因此我們以 36 位作為 HPL 的操作點。

5.3.4　HPL-MxP

HPL-MxP 以 BF16 完成 LU 分解，再以 FP64 迭代修正。我們在每次寫回 BF16 時歸零，使用 8 個節點、$N = 491520$。此處使用的測試矩陣是對角優勢矩陣（對角元等於該列非對角元絕對值的和），對角 tile 使用近似分解（`--approx-diag`），迭代修正使用 Richardson 法搭配 FP32 預條件子。依 HPL-MxP 的規則，這類對角優勢的矩陣只能用於測試與調校 [42]；在強對角優勢的矩陣上，很粗的 L、U 仍是好的預條件子，所以以下「保留 3 位仍穩定」的結論不能直接外推到正式允許的矩陣。

HPL-MxP 也在兩組節點上量測：甲組（work4–7、work10–13，工作 1671）每個上限 2 對；乙組與 HPL 的乙組相同（工作 3583，NCCL 與 MPI 固定在第一個 IB 埠），先暖機一次，每個上限 4 對。HPL-MxP 的 LU 以 BF16 計算、速度快，通訊所佔的比例比 HPL 高，所以乙組的原生比甲組慢得更多（400–800 W 低 7–13%）。如圖 5.5 與表 5.4 所示，保留 3 位在 $N = 245760$–491520 的四組大小都穩定：殘差 1.3–3.8，固定 3 次修正（以下殘差皆為逐次的範圍）。在 $N = 491520$，6 對合併後它在 500 W 快 $5.8 \pm 0.7\%$（甲／乙組 6.5/5.4%），600–800 W 快 1.7–2.3%（95% 區間為 ± 0.3 個百分點，兩組只差 0.2–0.3 個百分點），每次求解少耗電 3.6–5.9%；600 W 以上的加速雖小，6 對的區間都不含 0。在低於規格表下限的 400 W 快 $11.4 \pm 1.3\%$。500–700 W 的

表 5.4　8 節點 HPL-MxP 在各功耗上限下的效能（$N = 491520$，報告值含迭代修正，兩組節點）

	原生 BF16（TFLOP/s）		保留 3 位		加速			ΔW		耗電
上限	甲	乙	甲	乙	甲 2 對	乙 4 對	6 對合併	甲	乙	6 對合併
400 W*	2076	1922	2340	2128	+12.7%	+10.7%	+11.4 ± 1.3%	35 W	32 W	−10.1 ± 0.5%
500 W	2823	2555	3007	2692	+6.5%	+5.4%	+5.8 ± 0.7%	45 W	45 W	−5.9 ± 0.6%
600 W	3229	2858	3310	2922	+2.5%	+2.3%	+2.3 ± 0.3%	37 W	35 W	−3.6 ± 0.3%
700 W	3449	3039	3519	3094	+2.0%	+1.8%	+1.9 ± 0.3%	34 W	39 W	−4.3 ± 0.5%
800 W	3655	3178	3720	3229	+1.8%	+1.6%	+1.7 ± 0.3%	—	—	−4.8 ± 1.4%

甲組：work4–7、work10–13（工作 1671），每個上限 2 對；乙組：work3–8、work12、work13（工作 3583，網路較慢，見第 5.3.1 節），先暖機一次，每個上限 4 對。效能為各組的平均；加速為各對的平均，合併欄附 6 對的 95% t 區間（6 對分屬兩組節點，區間可能偏窄）；ΔW 為等效多出的上限，耗電為每次求解的模組耗電變化。* 低於規格表的可設範圍（450–1000 W）。800 W 的等效功耗需要 800 W 以上的原生點：甲組只能借用另一個工作（1675）的 900 W 原生點（換算為 37 W），乙組沒有，所以不列；甲組的原生在 1000 W 會觸發機箱的硬體功耗煞車，所以 1000 W 也不比較。資料來源：`gh200/results/` 中的 `e2_summary.csv`、`e2_more_summary.csv` 與 `e2_pairs_ci.csv`。

等效功耗為 34–45 W（兩組皆是）。每個上限的修正次數都從原生的 1 次變成 3 次；LU 本身的加速從 400 W 的 13.5–16.0% 降到 800 W 的 5.2–6.0%，扣除多出的修正時間後，600 W 以上只剩約 2%。保留 0 位（甲組）雖然快 7.1–26.0%，但 $N = 491520$ 的殘差已達 10.0–11.1（門檻 16），$N = 393216$ 時為 11.7–13.5，在 $N = 245760$ 時修正次數暴增到 52 次、效能下降 70%，所以只是失效邊界。

保留 0 位在這個 N 上更快（例如甲組 400 W 時 2616 TFLOP/s），但在較小的 N 上修正次數暴增，所以不作為操作點。另有一個附帶發現：風扇全開時，原生 HPL-MxP 在 1000 W 會觸發機箱電源的硬體功耗煞車（每次 4–10 秒），反而比 800 W 慢；尾數歸零版本的尖峰功耗較低：在診斷工作 1674 的取樣中，原生的每次執行都出現硬體功耗煞車的降頻旗標，保留 0 位的版本則沒有；開啟降頻計數器的工作 1675 只執行原生，也記錄到煞車。這個現象只在這兩個診斷工作中觀察，因此只作為附帶觀察。

5.3.5　小結

在具殘差檢查把關的完整基準上，功耗受限時尾數歸零都帶來正面收益；但保留位數必須以多組問題大小驗證，並選擇有餘裕的操作點；HPL-MxP 的結果目前限於對角優勢的測試矩陣。

表 5.5　科學軟體所需的尾數位數（攔截所有 FP64 矩陣乘法）

系統	結果
Quantum ESPRESSO 金表面（112 原子）	最近捨入 24 位誤差 8.1×10^{-6} Ry、20 位 5.5×10^{-5} Ry、16 位發散
Quantum ESPRESSO 矽（Si 1000）	28 位誤差 3.5×10^{-7} Ry；20 位是失效邊緣（一次收斂、一次中止）
PySCF 纈胺黴素 DFT（門檻 10^{-7}）	32–40 位都與原生同樣 12 圈收斂；32 位誤差 $1.2 \times 10^{-5}\ E_h$（約 0.007 kcal/mol）
PySCF 紫杉醇 HF+MP2	往零截斷 44 位誤差 $5.6 \times 10^{-8}\ E_h$；36 位 $2.7 \times 10^{-5}\ E_h$

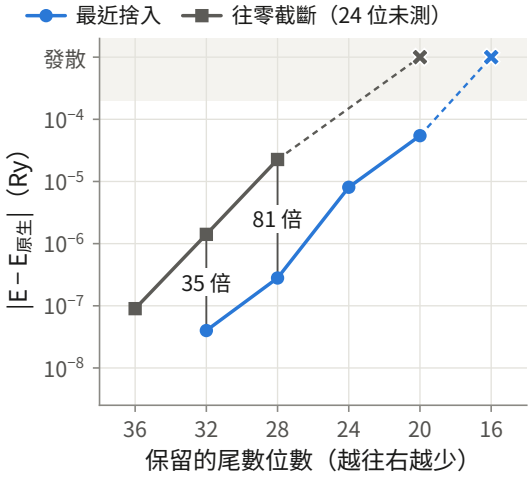


圖 5.6　Quantum ESPRESSO 金表面的總能量誤差與保留位數：往零截斷與最近捨入。
資料來源：Slurm 工作 1729、1738（金表面攔截掃描）

5.4　科學軟體：Quantum ESPRESSO 與 PySCF

5.4.1　所需的尾數位數

我們以攔截函式庫截斷所有 FP64 矩陣乘法，觀察物理結果（總能量）在多少位時開始偏離，結果列於表 5.5。

平面波密度泛函只需要 20–28 位，高斯基底的量子化學需要 32–36 位；後者與 Dawson 等人以模擬得到的「約 37 位有效尾數即已足夠」一致 [40]。

5.4.2　捨入方式的影響

如圖 5.6 所示，同樣保留 32 位與 28 位時，往零截斷的總能量誤差分別是最近捨入的 35 倍與 81 倍，遠大於單次矩陣乘法的差距（式 (4.7) 的係數 α 只差約 2.6 倍）：截斷永遠把數值往 0 拉，誤差有固定方向，會在自洽迭代中一路累積；最近捨入沒有偏差，誤差會互相抵銷。

表 5.6　Quantum ESPRESSO 程式內截斷與自動調整位元數的時間與耗電

做法	系統	上限	時間	耗電量	總能量誤差
依收斂門檻自動調整	金表面	450 W	$-3.6 \pm 0.2\%$	$-3.4 \pm 0.1\%$	0‡
依收斂門檻自動調整	矽	550 W	$-2.1 \pm 0.1\%$	$-2.2 \pm 0.1\%$	0‡
程式內固定 24 位	金表面	450 W	$-2.0 \pm 0.2\%$	$-2.1 \pm 0.1\%$	3×10^{-8} Ry
程式內固定 28 位	矽	550 W	$-2.0 \pm 0.1\%$	$-2.1 \pm 0.2\%$	$< 10^{-8}$ Ry
攔截所有 GEMM，最近捨入 28 位（函式庫內建時的上限）	矽	550 W	−6.2%	−6.4%	3.5×10^{-7} Ry

‡ 總能量印出的每一位（10^{-8} Ry）都與原生相同。前四列為程式內的做法：先執行一次不計入的暖機，再與未修改的原生程式交錯執行 6 對，數字為 6 對的平均 $\pm$ 95% t 區間。先前在另一組節點、沒有先暖機的兩個工作（排除含暖機的第一對後各 4 對）依序為 -4.0 ± 0.4/-3.7 ± 0.5、-2.4 ± 0.2/-2.4 ± 0.2、-2.4 ± 0.5/-2.4 ± 0.4、-2.2 ± 0.3/-2.2 ± 0.5%（時間／耗電）。最後一列與同樣付出複製成本的基準比較，各執行一次。耗電為驅動程式讀數的模組耗電。資料來源：`gh200/results/qe_warm_au_summary.csv`、`gh200/results/qe_warm_si_summary.csv`（先前的工作：`gh200/results/qe_adapt_au_6pairs.csv`、`gh200/results/qe_adapt_si_6pairs.csv`）。

5.4.3　效能與耗電

這些程式在 1000 W 下不太受功耗上限限制（金表面的平均功耗遠低於上限，矽仍有部分時間撞上限），所以我們在受限較明顯的 450–550 W 量測，結果列於表 5.6。每個工作先執行一次不計入的原生暖機，再以交錯順序執行三個區塊、共 6 對。先前沒有暖機的工作中，有兩個工作（2009、2010）的第一次執行（原生）明顯偏慢，會使含它的那一對收益偏大；先暖機的設計就沒有這個問題。

自動調整在金表面快 3.6%、在矽快 2.1%，耗電少 3.4% 與 2.2%。先前在另一組節點、沒有先暖機的兩個工作得到 4.0% 與 2.4%：兩組節點相差 0.3–0.4 個百分點，大於各自的區間，表示節點之間的差異也會影響結果，因此表中以先暖機的新量測為準。金表面上，自動調整在 6 對中每一對都比固定 24 位快 1.2–1.8 個百分點；矽上兩者幾乎相同（差 0.05 ± 0.11 個百分點）。因此自動調整至少追平事後才找得到的最佳固定位數，而且不需要事先試驗。最後一列攔截所有矩陣乘法，代表「如果函式庫內建截斷」時的上限；攔截本身約有 5% 的複製成本。

自動調整的規則（式 (4.1)）讓位元數隨收斂從 16 升到 39 位（金表面）或 42 位（矽），使截斷誤差大致不超過 Davidson 當圈本來就容許的誤差，並由後續迭代修正；因此總能量在印出的所有位數（10^{-8} Ry）都與原生相同，SCF 圈數也相同。不過金表面的收益有一部分並非功耗效應：自動調整讓 Davidson 少做了約 7–9% 的工作（`h_psi` 呼叫 311 次對 336 次）。在一次功耗上限被外部重設為 1000 W、幾乎不受功耗限制的執行中（原生 1957–1960 MHz，截斷的兩種版本約 1973 MHz），它仍比原生與時脈相同的固定 24 位快約 2%（73.5–73.7 秒對 74.8–75.1 秒）。所以金表

面的 3.6% 中約有一半來自迭代減少，原因尚未查明；矽上沒有這個現象（206 次對 205 次），2.1% 可視為功耗效應。

5.4.4　單精度的可行性

Quantum ESPRESSO 的運算元值域放得進 FP32，所以我們也嘗試把所有 GEMM 改為 FP32、TF32 或 3xTF32（3xTF32 是攔截工具內自行撰寫的實作：拆成兩個 TF32、以三次 FP32 累加的 tensor core 乘法計算）。結果金表面的 FP32 在第 26 圈、TF32 與 3xTF32 在第 2 圈中止；矽的 FP32 在第 14 圈、3xTF32 在第 9 圈中止，積分電荷也明顯偏離。原因在第 5.7.4 節說明。這是把所有 GEMM 都替換掉的極端做法；只在部分步驟使用 FP32 的混合精度（例如 Woo 等人 [38]）不在此限。

5.4.5　小結

科學軟體需要 20–36 位，在功耗受限時能取得 2–6% 的時間與耗電改善。最有價值的是「隨收斂門檻調整位元數」：總能量印出的每一位都不變，而且不需要事先知道安全的位數。

5.5　高功耗的真實應用：DF-MP2

第 5.4 節的科學軟體在 1000 W 下平均用不到上限（金表面約 535–600 W、矽 1000 約 800 W），必須壓低功耗上限才看得到明顯的加速。因此我們另外選擇一個高功耗的應用：GPU4PySCF 的密度擬合二階微擾理論（DF-MP2）。它的 MP2 階段 GPU 功耗約 800 W（1000 W 時模組約 870 W，只有部分時間撞上限），MP2 階段中矩陣乘法佔 GPU 時間的 78%，而且幾乎都是同一種收縮：把密度擬合張量的兩個切片相乘（FP64 DGEMM）。截斷實作於程式內：張量建好後原地截斷一次尾數，之後所有矩陣乘法都受惠，不需要複製。同一個程序先做一次 HF，再依序執行各種 MP2 版本（紫杉醇，def2-TZVP 基底）。每個上限在兩個工作、兩個節點上共交錯執行 6 輪（第二個工作在每個上限先執行一次不計入的暖機），結果列於表 5.7。

截斷在功耗受限時快 4.7–7.7%；1000 W 時速度幾乎不變（+0.5–0.8%），但少用 61–127 W、每次計算少耗電 8.4–14.3%，與 HPL 同一量級。GPU4PySCF 內建的 FP32 更快，誤差也在化學精度內，因為這個張量的指數跨度很窄（整體 27 個、列內中位數 18 個 2 的次方），FP32 放得下。cuBLAS 模擬在這個形狀一律較慢。

表 5.7　DF-MP2（紫杉醇，def2-TZVP）各變體的 MP2 時間與相對原生的加速

	原生	截斷 32 位	截斷 20 位	內建 FP32	cuBLAS 模擬 32 位
能量誤差	—	$10^{-11}\,E_\mathrm{h}$	$1.2\times10^{-8}\,E_\mathrm{h}$	(2.8–5.7) $\times10^{-8}\,E_\mathrm{h}$	$2\times10^{-8}\,E_\mathrm{h}$
1000 W	39.5 s	+0.5 ± 0.2%	+0.8 ± 0.1%	+9.8%*	慢 35%
700 W	43.9 s	+4.9 ± 0.4%	+7.4 ± 0.4%	+18%	慢 28%
500 W	51.5 s	+4.7 ± 0.2%	+7.7 ± 0.5%	+15%	慢 18%
1000 W 時少用的功耗（耗電）	—	61–75 W（8.4%）	106–127 W（14.3%）	—	—

原生欄為 MP2 階段的時間，其餘為相對原生的加速，數字為兩個工作共 6 輪的平均 ± 95% t 區間；最後一列為 1000 W 時比原生少用的模組功耗與每次計算少耗的電（驅動程式讀數）。*6 輪的平均；其中 1 輪只快 2.6%，其餘 5 輪為 10.9–11.5%。資料來源：`gh200/results/` 中的 `mp2_summary.csv` 與 `mp2_more_summary.csv`。

小結　在高功耗的真實應用上，截斷取得與 HPL 同一量級的收益；但資料跨度窄時，改變格式（FP32）可能更快。改變格式在什麼情況下會出問題，見第 5.8 節。

5.6　跨度寬且功耗高的應用：高斯過程地統計

前面的應用不是功耗高而資料跨度窄（DF-MP2），就是資料跨度寬而功耗不高：高斯基底 DFT 的列內跨度很寬（第 5.8 節），但幾種 DFT 變體在 1000 W 下只用到 650–690 W。不過只要功耗上限低於它們的用電，寬跨度的 DFT 也會受限：纈胺黴素（格點等級 6）在 550/450 W 時，截斷 36 位快 3.3–3.5%、省電 2–3%，誤差 $10^{-6}\,E_\mathrm{h}$；模擬要用到 40 位才不會算錯，時間是截斷 36 位的 1.5–1.6 倍。

5.6.1　對數概似評估

為了找到兩者兼具的例子，我們轉向地統計中的高斯過程最大概似估計，也就是 ExaGeoStat 這類軟體所做的計算。每次評估對數概似都要對 $n\times n$ 的 Matérn 共變異矩陣做 Cholesky 分解，計算量與 HPL 一樣幾乎全是矩陣乘法；共變異隨距離指數衰減，所以同一列中數值大小相差很多。我們撰寫了一個 ExaGeoStat 式的核心（n = 80000 個隨機位置、分塊 Cholesky、尾端更新為 FP64 矩陣乘法），在 6 個範圍參數上評估對數概似；截斷實作於程式內（每個面板在尾端更新前，以依記憶體順序走訪的單次融合核心截斷一次；早先兩次分開、存取不合併的版本只快 5.2–5.7%）。原生在 1000 W 用到 953–970 W，面板有 88% 的元素比所在列最大值小 2^{-32} 以上，

表 5.8　高斯過程對數概似評估相對原生 FP64 的加速與偏差（$n = 80000$，6 個範圍參數，6 輪）

上限	截斷 36 位 （偏差 2×10^{-5}）	截斷 28 位 （偏差 7×10^{-4}）	cuBLAS 模擬 36 位 （偏差 2×10^{-3}）	FP32 （偏差 87）
1000 W	+3.0 ± 0.3%，省電 3.9 ± 0.7%	+3.8 ± 0.4%，省電 8.2 ± 0.6%	慢 41%	慢 8.2%
700 W	+5.0 ± 0.1%	+7.9 ± 0.2%	慢 31%	+3.2%
500 W	+4.8 ± 0.1%	+7.5 ± 0.1%	慢 21%	+3.9%

數字為同一節點上兩個工作共 6 輪的平均 ± 95% t 區間（第二個工作在每個上限先執行一次不計入的暖機）。偏差為對數概似的最大絕對偏差（對數概似的數值約為 10^5），每輪完全相同。資料來源：`gh200/results/gp_summary_v2.csv`、`gh200/results/gp_more_summary.csv`。

跨度寬與功耗高兩者兼具。結果列於表 5.8。

截斷 28 位在功耗受限時快 7.5–7.9%；1000 W 時這組也快 3.8% 並省電 8.2%，但 1000 W 的加速隨設定而變（第 5.6.3 節依 Morton 排序的 $n = 60000$ 只快 0.0–0.1%，因為原生只有部分時間撞上限）；偏差對參數估計沒有影響。FP32 的偏差大到無法用於估計（$n = 8000$ 時 FP32 仍估出相同的值，n 變大後才偏離，可能與條件數隨 n 變差有關）；cuBLAS 模擬比截斷 28 位不準，又比原生慢 21–41%（尾端更新的 k 等於區塊大小 1024，依第 5.7 節的分界，這個形狀本來就不利於模擬）。另外，這裡的列內跨度比 DFT 更寬，模擬卻沒有算錯，只是精度變差：列內跨度寬是模擬出錯的必要條件，而非充分條件。

5.6.2　最大概似估計

表 5.8 是在固定的 6 個範圍參數上評估；為了確認估出的參數本身不變，我們也執行一維的估計：變異數與平滑度固定，以黃金分割搜尋範圍參數（取對數），每個變體評估 17 次對數概似（$n = 60000$，1000 與 700 W，各交錯兩輪；`gh200/results/gp_mle_summary.csv`）。截斷 36 位、28 位與 cuBLAS 模擬估出的範圍參數都與原生完全相同（$\hat{\theta} = 0.05007952$，印出的 8 位全同），最大對數概似只差 4.8×10^{-6} 與 2.3×10^{-4}；FP32 的估計偏了 0.49%，對數概似低 0.72（$n = 8000$ 時連 FP32 都估出同一個值）。700 W 時整個估計以截斷 28 位快 7.1–7.3%、36 位快 4.8–4.9%，耗電少 6.6–6.8% 與 4.5–4.8%；1000 W 時也快 1.8–2.3% 與 1.6–2.2%，耗電少 9.1–9.9% 與 3.5–5.6%；cuBLAS 模擬慢 32–41%。

表 5.9　截斷與 tile 混合精度疊加的效果（n = 60000，Morton 排序，倍數皆相對原生 FP64）

上限	截斷 28 位	混合精度	混合精度 + 截斷 28 位	疊加後再快
1000 W	同速，省電 14.7%（這組原生只有 19–54% 的時間撞上限）	1.61 倍	1.65 倍	1.9%
700 W	+7.4%	1.60 倍	1.71 倍	7.0%
500 W	+7.2%	1.63 倍	1.73 倍	6.1%
對數概似最大偏差	1.3×10^{-4}	0.043	0.043	—

5.6.3　與 tile 混合精度疊加

前人的地統計混合精度（第 3.5 節）是把位置依空間填充曲線排序後，把整個 tile 換成低精度格式。我們在自寫的核心中加入類似的變體：位置依 Morton 曲線排序；在尾端更新中，每個 tile 欄在 FP64 範圍以外的 tile（依 tile 範數規則決定，要求精度 10^{-9}，同文獻 [36]）改用 FP16 輸入、FP32 累加的 tensor core。範圍參數 0.02 時有 52% 的 tile 使用 FP16，0.08 時只有 3%。六個範圍參數合計的結果列於表 5.9（n = 60000，ABBA 兩輪；`gh200/results/gp_mp_summary.csv`）。

截斷可以疊加在改變格式的做法之上：把混合精度剩下的 FP64 tile 再截斷到 28 位，功耗受限時再快 6–7%，偏差不變。把 FP16 運算元也截斷到 5 個尾數位元則沒有額外收益。ExaGeoStatCPP 的混合精度選項（`--computation=mixed`）只有列舉值、沒有實作，所以無法與原版直接比較。這組實驗的位置依 Morton 曲線排序，截斷 28 位在 700/500 W 快 7.2–7.4%，與表 5.8（未排序）一致。

5.6.4　實際的 ExaGeoStat

我們建置了 ExaGeoStatCPP（StarPU 與 Chameleon 的 tile Cholesky，cuBLAS 13.6，tile 大小 1920），以攔截工具截斷它的 FP64 tile GEMM，執行最大概似估計（N = 60000，每次 6 個 BOBYQA 評估；資料以原生版本產生一次，之後每次都讀取同一份；`gh200/results/exageostat_tdp.csv`）。它的 Matérn 共變異數在 CPU 上產生，每次評估約 9 秒，GPU 分解只需 2.3 秒（1000 W）。分解時間的比較（ABBA 兩輪平均）列於表 5.10。

即使在 1000 W，分解也部分受功耗限制（GPU 平均約 620–670 W，常有功耗上限造成的降頻事件）。對數概似的相對變化最多為 1.2×10^{-10}（36 位）與 9.8×10^{-8}（28 位）。450 W 的兩輪差異大（截斷 28 位對複製基準為 +6.9% 與 +11.9%，SM 707 MHz 對 926 MHz），因為 CPU 產生共變異數時與 GPU 共用模組上限。攔截工具本身的複

表 5.10　ExaGeoStatCPP 的 tile Cholesky 分解時間比較（ABBA 兩輪平均）

上限	截斷 36 位 對複製基準	截斷 28 位 對複製基準	截斷 28 位 對未修改的原生	SM 時脈 （複製基準 → 28 位）
1000 W	快 3.0%	快 4.0%	慢 3.1%	1913 → 1975 MHz
600 W	快 5.0%	快 8.0%	慢 1.3%	1166 → 1305 MHz
450 W	快 5.6%	快 9.4%	慢 0.5%	—

製要多花 7–9%，與未修改的 ExaGeoStat 相比仍慢 0.5–3.1%；要取得這個收益，截斷必須實作於函式庫內（例如 Chameleon 的 GEMM codelet）。

5.6.5　小結

在跨度寬且功耗高的矩陣乘法工作上，整個矩陣使用同一種格式的做法（FP32、INT8 模擬）不是結果錯誤就是較慢；在測試的三種整個矩陣的做法（FP32、cuBLAS 模擬 36 位、截斷）中，只有截斷又快又準，估出的範圍參數也不變。前人的 tile 混合精度快 1.6 倍，截斷還能疊加其上，在功耗受限時再快 6–7%。在實際的 ExaGeoStat 上，截斷使分解比複製基準快 3–9%，但攔截工具的複製成本將其抵銷，因此需要在函式庫內實作。本節的限制是：主要結果來自自寫的核心與合成資料，混合精度變體是簡化的重新實作。

5.7　與 FP64 模擬及單精度的分界

以較少位元計算 FP64 矩陣乘法還有另一條路：以 INT8 tensor core 模擬 FP64。我們在同一節點、同一功耗上限下，把誤差對齊到同一量級後比較速度。以小矩陣與 CPU 精確值對照確認：在相同位元數下，截斷與模擬的誤差屬於同一量級。本節的截斷都是往零截斷；應用程式用的最近捨入在相同位數下歸零的位元數相同、只多一次加法，誤差則小約 2.6 倍（約 1.4 位），所以同樣精度下最近捨入可以少保留約 1 位，分界點會稍微往有利於截斷的方向移動；我們沒有以最近捨入重做這組掃描。GEMMul8 使用 3.5.1 版（2026 年 9 月 29 日發布；截至本報告，之後的 3.5.2 只修正 HIP 編譯）。

5.7.1　功耗上限改變勝負

圖 5.7 比較「方陣」（$m = n = 32768$、$k = 2048$ 的合成資料，以下沿用此稱呼）上的各種做法。GEMMul8 的功耗很高：1000 W 時就用到約 970 W，450 W 時時脈

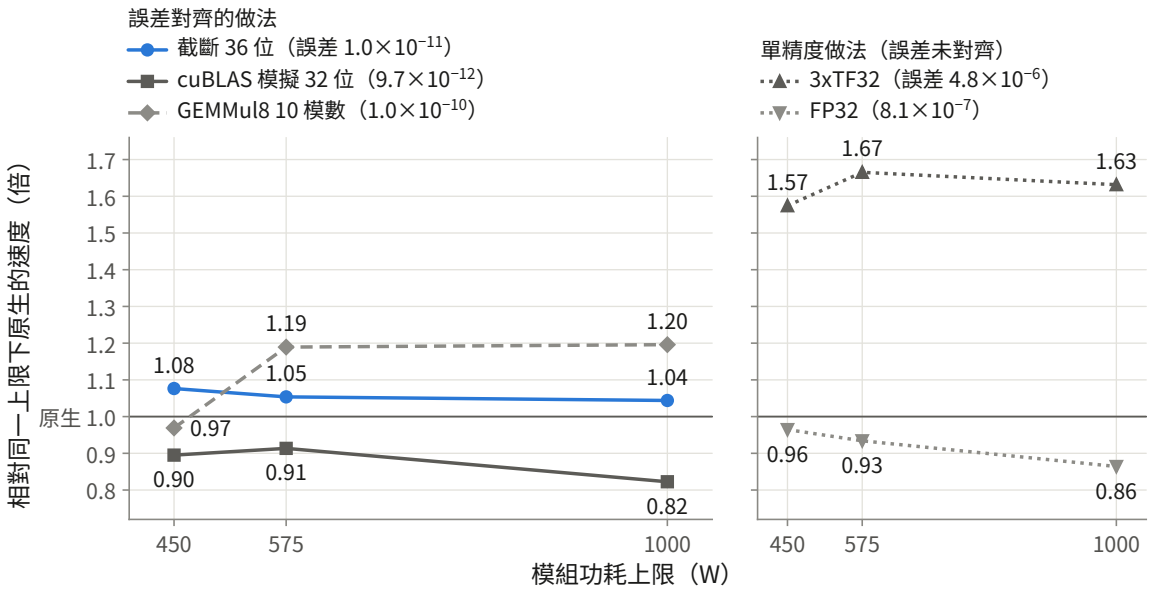


圖 5.7　方陣 FP64 矩陣乘法在 450、575 與 1000 W 下相對原生的速度。左：誤差對齊的截斷 36 位與 INT8 模擬（cuBLAS 32 位、GEMMul8 10 模數）；右：單精度做法（3xTF32、FP32，誤差未對齊）。

資料來源：Slurm 工作 1894；`gh200/results/m3_ozaki2_tf32x3_summary.txt`

掉到 650–720 MHz，反而輸給截斷。cuBLAS 模擬（28 位）在 1000 W 只用約 706 W；相反地，cuBLAS 模擬在方陣上隨上限降低而更有優勢（28 位時從 1000 W 的 1.07 倍升到 450 W 的 1.20 倍）。所以上限收緊對誰有利，取決於模擬本身的功耗；截斷在每個上限都比原生快，但幅度不隨上限單調變化（第 5.2 節）。在較粗的精度等級（10^{-9}）則是模擬較快；cuBLAS 模擬以約 8 位為一片，精度呈階梯式變化，截斷則能以 1 位為單位調整。

5.7.2　矩陣形狀決定勝負

我們攔截 PySCF 纈胺黴素計算的全部 45,574 次 GEMM，發現兩種形狀各佔 35% 的運算量（圖 5.8）。模擬需要先把輸入拆片、最後再合併，這部分的成本與矩陣元素數（約 $mk + kn + mn$）成正比，而乘法本身是 mnk。在 $n = 300$ 的瘦長矩陣上，拆片成本與乘法相當：與截斷 36 位同精度的 cuBLAS 模擬（32 位）在 1000/575 W 只有原生 0.38/0.45 倍的速度，較粗的 28 位也只有 0.47/0.57 倍；在 $k = 76800$ 的長內積矩陣上，拆片成本被均攤，INT8 的高運算量便顯現出來。截斷沒有拆片成本，在同一個上限下對任何形狀的加速都差不多（1000 W 時兩種形狀分別快 4.6% 與 6.2%）。因此最佳做法是依形狀選擇：瘦長矩陣用截斷，長內積矩陣用模擬。

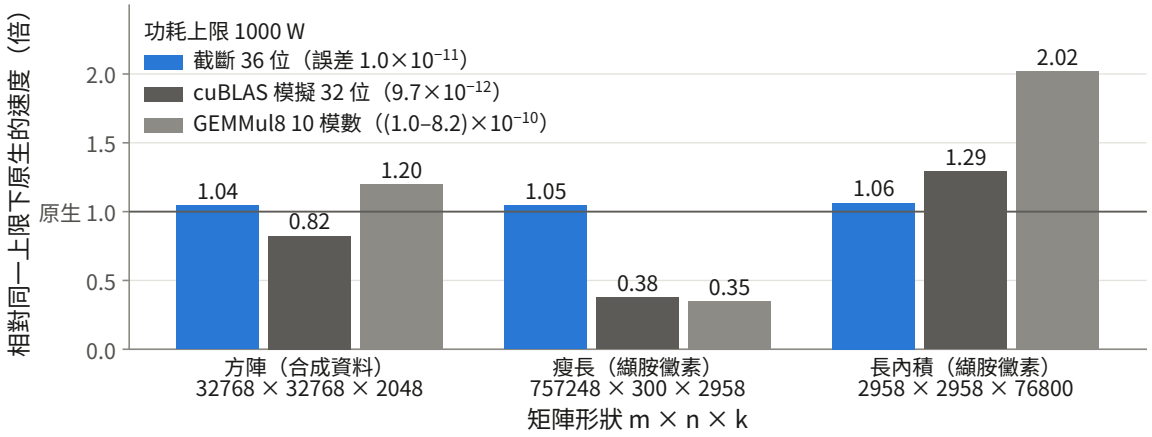


圖 5.8　1000 W 下，繙胺黴素 DFT 的兩種真實矩陣形狀（瘦長 757248 × 300 × 2958 與長內積 2958 × 2958 × 76800）及作為參考的方陣上，截斷與模擬相對原生的速度。

資料來源：Slurm 工作 1894、1896、1897；`gh200/results/m3_realshape_summary.txt`

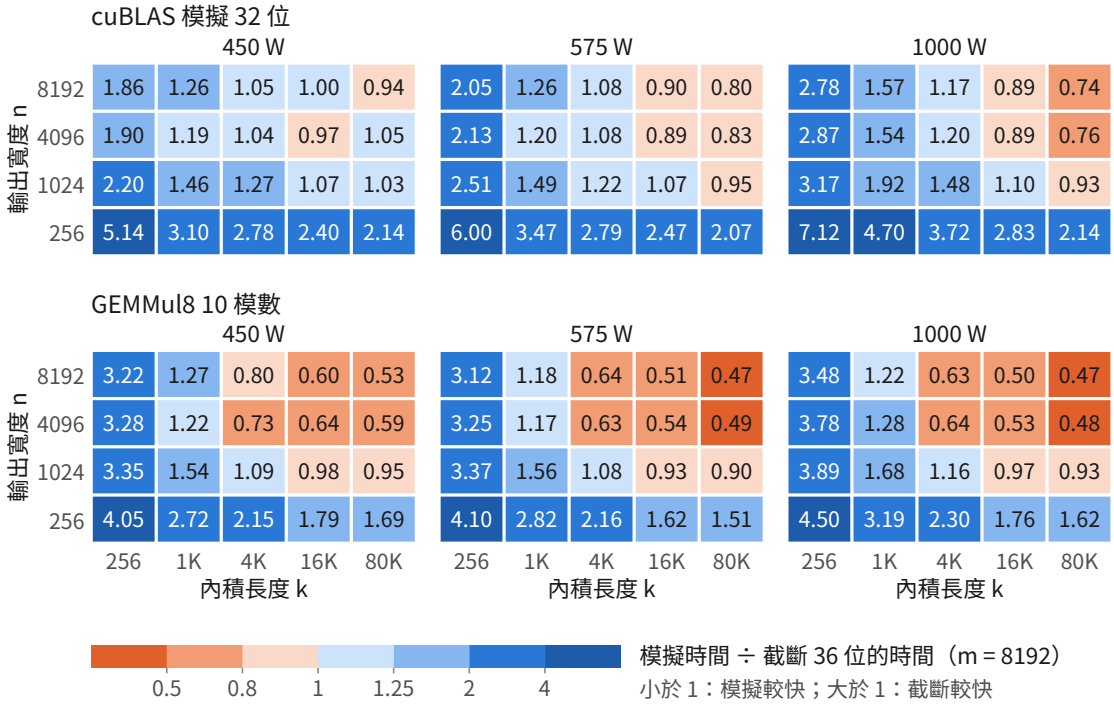


圖 5.9　截斷與 INT8 模擬的分界點掃描（m = 8192）：模擬時間相對截斷保留 36 位的倍數。

資料來源：Slurm 工作 2005–2008（480 筆）；`gh200/results/crossover_*.csv`

5.7.3　分界點掃描與預測公式

為了把「依形狀選擇」變成可用的規則，我們固定 m = 8192，n 取 256、1024、4096、8192，k 取 256 到 81920，在 1000、575 與 450 W 各量測兩輪，共 480 筆（圖 5.9）。

截斷保留 36 位在全部 60 組（形狀 × 上限）都比原生快（450 W 為 3.8–11.9%、575 W 為 3.6–5.9%、1000 W 為 0.5–4.8%）。模擬大致只在 n 與 k 都大時較快：cuBLAS

模擬主要在 $n \geq 4096$ 且 $k \geq 16384$ 時勝出（另有 $n = 1024$、$k = 81920$ 在 1000 與 575 W 勝出），勝出的形狀在 1000、575、450 W 分別有 5、5、2 種；但在模擬本身用不滿功耗的形狀上（例如 $n = 8192$、$k = 4096$，模擬對截斷的時間比從 1.17 降到 1.05），上限越低反而對模擬越有利，方向取決於模擬本身是否受功耗限制。GEMMul8 在 $n \geq 4096$ 且 $k \geq 4096$ 時只要截斷的 0.47–0.80 倍時間（$n = 1024$、$k \geq 16384$ 時也略快），但精度低一個量級（10^{-10} 對 10^{-11}）。以

$$\frac{t_{\mathrm{emu}}}{t_{\mathrm{trunc}}} = A + B\left(\frac{1}{m} + \frac{1}{n}\right) + \frac{C}{k} \tag{5.1}$$

擬合，GEMMul8 的決定係數 R^2 達 0.98–0.99；若只用單一的有效大小（m、n、k 倒數和的倒數）擬合，則只有 0.86–0.89，表示 GEMMul8 的輸入拆片與輸出合併是兩種不同的成本。cuBLAS 模擬的 R^2 只有 0.88–0.89，與單一有效大小的模型相近，仍有模型未捕捉的因素（例如不同形狀選到不同核心）。四個 n 值分別在不同節點上量測：每格的比值是同節點配對，但形狀效應與節點差異因此無法分開，這是實驗設計上的缺陷，而不只是限制。此外每個上限只有約 20 個點擬合 3 個參數，R^2 只能當作描述性的指標。

式 (5.1) 只以 $m = 8192$ 的資料擬合。我們以先前量測、未參與擬合的兩種纈胺黴素真實形狀檢驗（1000 W 與 575 W、兩種模擬方法，共 8 個案例）：瘦長的 $757248 \times 300 \times 2958$ 預測截斷勝出、長內積的 $2958 \times 2958 \times 76800$ 預測模擬勝出，8 個案例的勝負全部判斷正確。倍數本身的誤差在 cuBLAS 模擬為 6–15%、GEMMul8 為 24–37%，後者較大是因為瘦長矩陣的 $m = 757248$ 遠超出擬合範圍。不過這 8 個案例都離分界很遠（模擬的時間不是截斷的一半到八成，就是兩倍以上），勝負本來就不難判斷，只是寬鬆的檢驗；公式對 m 的依賴也尚未驗證。因此式 (5.1) 目前只是描述這組量測的經驗式；要用來替函式庫選擇方法，還需要在分界附近、其他 m 以及同一節點上重新量測與驗證。

5.7.4　FP32 與 3xTF32 不適用的原因

3xTF32（自行撰寫的實作，不是 CUTLASS 的版本）在方陣上最快，但它的誤差隨內積長度 k 成長；截斷後的 FP64 仍以 FP64 累加，誤差與 k 無關（表 5.11）。

在 4096×4096 上，k 每增加 8 倍，3xTF32 的誤差就增加 7.5–7.9 倍，與 tensor core 以 FP32 累加時朝零捨入的既有觀察一致 [43]。Quantum ESPRESSO 的 GEMM 內積長度就是平面波數（數十萬），所以 3xTF32 退化到與 TF32 同一量級，實測無法收斂（第 5.4.4 節）。

表 5.11　單精度做法的矩陣乘法相對誤差隨內積長度 k 的變化

內積長度 k	3xTF32	TF32	FP32
2048	4.8×10^{-6}	2.6×10^{-4}	8.1×10^{-7}
16384	3.8×10^{-5}	—	2.3×10^{-6}
95386（金表面的平面波數）	5.6×10^{-5}	2.7×10^{-4}	2.8×10^{-6}
375286（矽的平面波數）	2.2×10^{-4}	3.4×10^{-4}	1.9×10^{-6}

前兩列為 4096 × 4096、後兩列為 1024 × 1024 的均勻隨機矩陣。

5.7.5　小結

截斷與模擬沒有一方全面勝出，分界點由所需精度、矩陣形狀與功耗上限決定，兩者應依形狀互補使用。把所有 GEMM 換成單精度（FP32、3xTF32），在長內積上無法取代 FP64 截斷。

5.8　指數跨度：只能以截斷處理的資料

截斷保留每個元素自己的 11 位指數，只改變尾數；改變格式的做法都必須先處理指數：FP32 與 FP8 以整個張量的範圍為準，INT8 模擬（Ozaki 類）以每一列的最大值為基準切片，比最大值小 2^{-k} 以上的元素在保留 k 位時會被捨去。因此「數值大小相差多少」會限制整個矩陣共用縮放的換格式做法，而截斷不受影響。

5.8.1　合成實驗

我們讓矩陣乘法（8192 × 8192 × 16384；此形狀在跨度小時是模擬較快）的每個元素，其指數在 S 個 2 的次方內均勻分佈（圖 5.10）。

截斷 36 位的誤差完全不受跨度影響，加速在各跨度下也大致相同（575 W 快 5.6–6.3%、1000 W 快 4.4–5.2%）。固定位數的 cuBLAS 模擬精度變差 4–9 倍（相當於少了 2–3 位），而模擬以約 8 位為一片，必須多付一整片才能追上；保證 FP64 精度的動態模式所需位數從 58 升到 301 位，超過上限就改用原生 FP64，連同判斷跨度的額外工作，時間是截斷的 1.6–2.6 倍；FP32 在跨度 256 時溢位。GEMMul8 在這種均勻分佈的資料上幾乎不受影響。另外，若只是每列各自有一個比例（整體跨度寬、列內窄），模擬完全不受影響，但 FP32 仍然溢位。

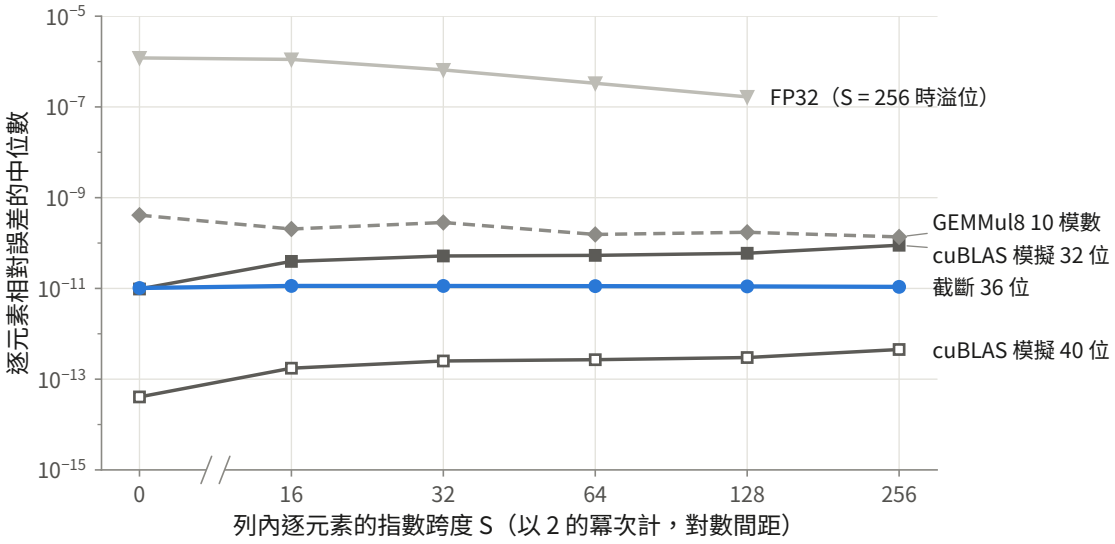


圖 5.10　指數跨度 S 變大時，各方法逐元素相對誤差的中位數。

資料來源：Slurm 工作 2034–2036（1000/575 W，兩輪）；`gh200/results/range_sweep.csv`

表 5.12　各應用的列內指數跨度，以及截斷與 INT8 模擬的誤差

應用	列內比最大值小 2^{-32} 以上的元素	截斷 36 位	cuBLAS 模擬 固定 36 位（5 片）	GEMMul8 （10 模數）
PySCF 纈胺黴素 DFT	24–44%（格點上的原子軌域矩陣達 87%）	$9.6 \times 10^{-7}\,E_\mathrm{h}$	**$0.0525\,E_\mathrm{h}$（約 33 kcal/mol）**	發散
QE 矽	0.24–0.54%	0 Ry	6×10^{-8} Ry	記憶體不足
QE 金表面	0.008–0.02%	0 Ry	5×10^{-8} Ry	3.7×10^{-7} Ry

元素比例依運算量加權。QE 金表面的跨度統計取自只跑 2 圈的基準輸入，能量誤差取自完整收斂的輸入。「記憶體不足」指 GPU 記憶體不足。

5.8.2　真實應用

我們在攔截工具中加入逐列跨度統計（每個矩陣乘法抽樣 256 列／行，並以合成資料驗證與理論值相符），量出各應用中「比所在列最大值小 2^{-32} 以上」的元素比例，並在各應用上直接比較截斷與模擬（表 5.12、圖 5.11）。

同樣是模擬 36 位，在列內跨度窄的 QE 上完全正常；在列內跨度寬的 DFT 上，SCF 照常收斂、沒有任何警告，能量卻錯了 52.5 mE_h，而截斷在兩者上都正常。模擬加到 40 位（6 片）就能算對（誤差 $8 \times 10^{-8}\,E_\mathrm{h}$），但時間是截斷 36 位的 1.5–1.6 倍（第 5.6 節）；出錯的是使用者指定的固定位數模式。cuBLAS 的自動模式（依指數跨度決定位數，上限設為 79 位）則沒有算錯（誤差 $4 \times 10^{-9}\,E_\mathrm{h}$）：89% 的運算量需要 80 位以上而改用原生 FP64，其餘以 72–79 位模擬，加上判斷跨度的額外工作，時間是原生的 2.1 倍（1000 W）。所以寬跨度資料上的模擬並非不能用，而是不划算。反過來，跨度很窄的 HPL（依運算量加權，只有 2.7×10^{-6} 的元素比所在列最大值小 2^{-32}

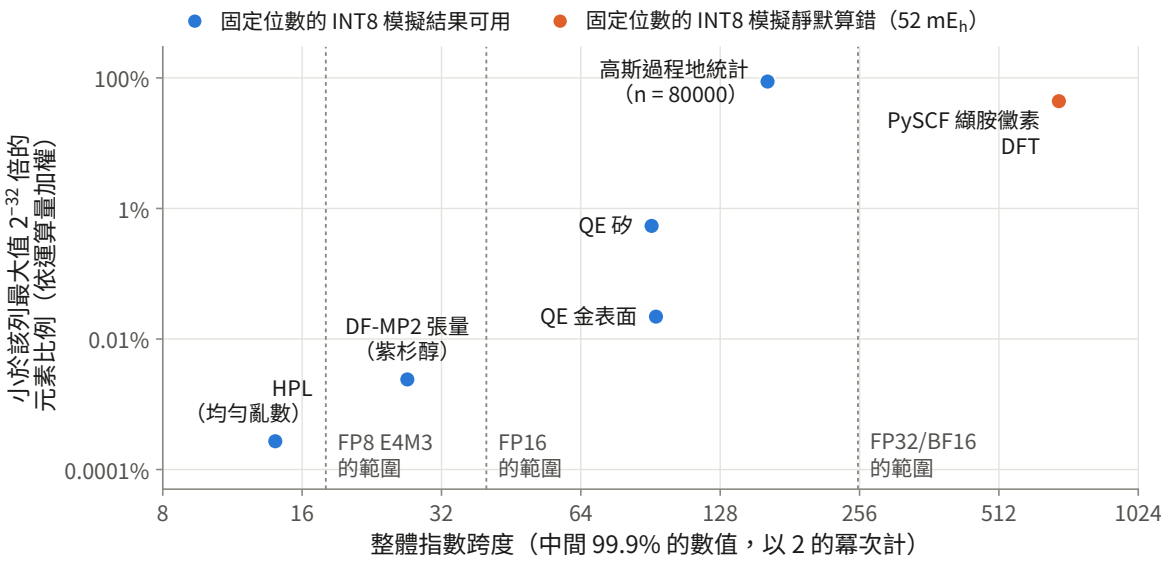


圖 5.11　各應用的整體指數跨度與列內小元素比例，以及固定位數的 INT8 模擬是否算錯。資料來源：`gh200/results/range_map_points.csv`（每個點的來源列於檔內；HPL 與高斯過程的點來自工作 3291、3304、3305）

以上）以固定 36 位模擬也通過殘差檢查（0.197；截斷 36 位為 0.057），但在單 GPU、$N = 81920$ 時只有原生 65% 的速度。合成的均勻分佈低估了這個問題，因為真實的原子軌域值隨距離指數衰減，同一列中很多元素比最大值小得多。

5.8.3　小結

指數跨度會限制改變格式的做法：逐張量縮放的格式取決於整體跨度，逐列切片的 INT8 模擬取決於列內跨度。跨度寬時，整個矩陣共用縮放的格式或固定位數的模擬可能出錯，截斷則不受影響；但列內跨度寬只是模擬出錯的必要條件（第 5.6 節的高斯過程就沒有出錯），以 tile 為單位改變格式也能處理這類資料。列內跨度可以用攔截工具事先量測，作為選擇方法時的風險指標。

5.9　已使用 FP8 的 AI 工作負載

FP8（3 個尾數位元）已是 Hopper 上 AI 的主流格式（細節見附錄 C）。Llama 2 70B LoRA 微調中，把凍結的 FP8 基底權重再截到 2 位，每一步只快約 1%，驗證損失卻從第一次評估就高 0.008–0.010，達標反而慢 19–40%。推論中，以最近捨入保留 3 位的 BF16 與 FP8 精度相當，在 FP8 容易出問題的場景也打平，但 FP8 的矩陣乘法快約 1.8 倍；在 FP8 之上再截斷只是以精度換速度。因此在 Hopper 上，FP8 已取得減少尾數位元幾乎全部的好處，尾數歸零的價值在 FP64 科學計算。

第六章　第二平台：AMD MI350X

第五章的結果都來自同一個 GH200 叢集。為了檢驗結論能否移植到另一家廠商、另一種功耗管理，我們在兩個各有 8 張 AMD Instinct MI350X 的節點上重做主要實驗：機制對照、矩陣乘法、FP64 HPL、HPL-MxP、DF-MP2 與高斯過程，並與 INT8 模擬（GEMMul8）正面比較。本章也用這些量測驗證第 4.6 節的功耗模型。每個實驗都在第二個節點上重複一次（同一節點、同一個工作內的各輪仍可能相關，所以區間描述的是收益在這兩個節點上有多穩定，而不是在大量 GPU 之間的分布）；除非特別說明，百分比都相對同一節點、同一功耗上限下未修改的原生程式。

6.1　平台、量測與統計單位

兩個節點（n01、n02）各有 8 張 MI350X OAM GPU（gfx950，288 GB HBM3E，最高時脈 2200 MHz），軟體為 ROCm 7.2.3（rocBLAS、hipBLASLt、rocSOLVER、RCCL）與 Open MPI 5.0.7（UCX）。功耗上限是每張 GPU 的封裝功耗上限（PPT），以 `amd-smi` 設定，可設 1–1000 W；預設 1000 W 也是最高值，不含 CPU。每張 GPU 的閒置功耗為 244–275 W。

與 GH200 不同，MI350X 的 8 張 GPU 同時執行同一個設定，共用機箱的風扇與電源。我們每 100 ms 讀取 8 張 GPU 的功耗、時脈、溫度，以及硬體的能量累加器（每單位 15.26 μJ，能量取視窗兩端的差，不受取樣頻率影響）和功耗、溫度限制的累計時間；每個量測視窗都記錄實際生效的上限。HIP 的裝置編號與 PCI 位址的順序不同，所以分析一律以程式印出的 PCI 位址對應功耗記錄。

同一個機箱、同一個工作中的量測並不獨立，因此本章的信賴區間改以較大的分析單位計算（同一工作內的單位仍可能相關）：矩陣乘法以「一個量測視窗」（8 張 GPU 先平均）為單位，HPL 與 HPL-MxP 以「一次整節點執行」為單位，DF-MP2 與高斯過程以「一輪」（8 張 GPU 先平均）為單位，兩個節點的單位合併後取 95% t 區間。各變體的順序為成對交錯（ABBA，或後述的成對 ABBA）。溫度的影響與控制見第 6.9 節。

表 6.1　以全 1 ／固定樣式填入被捨去的位元時，保留的歸零效益比例（%）。GH200：1000 W 的模組功耗或 600 W 的吞吐量；MI350X：鎖頻下每次運算的能量（矩陣乘法為三個時脈的範圍）、600 W 的吞吐量或 1000 W 的功耗。最後一欄是 MI350X 上歸零本身的效益。

核心與保留位數	GH200	MI350X	MI350X 歸零的效益
FP64 矩陣乘法，0 位	76/68	73–75/69	每次運算少耗 20–29%
FP64 矩陣乘法，20 位（GH200）、27 位（MI350X）	87/82	81–83/73–74	每次運算少耗 7–10%
BF16 矩陣乘法，0 位	92/92	77–78/68–72	每次運算少耗 13–15%
FP8 矩陣乘法，0 位	—	65–66/61	每次運算少耗 9–10%
FP64 乘加迴圈，0 位	87/62	74/63	快 33%
記憶體複製，0 位	97/86	83/41	功耗少 9%

資料來源：鎖頻填值 `mi350/logs/e1b_clk900_*`、`e1b_clk1300*`、`e1b_clk1700_*`；非矩陣乘法核心 `mi350/logs/n02/nongemm_*`

6.2　預設上限就受限：省電機制與時脈

MI350X 在出廠的 1000 W 上限下就受功耗限制。原生 FP64 矩陣乘法有 82% 的時間停在上限，時脈 1890 MHz；保留 0 位時只耗 785 W、時脈 2183 MHz，快 17%。在我們的量測中，FP64、BF16 與 FP8 矩陣乘法在 1000 W 有 82–100% 的時間撞上限。GH200 的預設上限下，截斷多半只表現為省電；MI350X 則在出廠設定下就直接轉成速度。

位元不翻轉還是數值為零？與第 5.1 節相同，我們把被捨去的位元填成全 1 或固定樣式：位元同樣不再翻轉，但數值不是 0。MI350X 可以鎖定最高時脈，所以我們把時脈鎖在 900、1300 或 1700 MHz（低於任何設定會達到的時脈，功耗限制的時間只有 0–0.6%），直接比較每次運算的能量（8 張 GPU × ABBA）。表 6.1 與 GH200 並列：在兩家廠商的矩陣引擎、FP64 向量單元與記憶體搬移上，填值都保留了大部分的效益，所以主要原因是位元不再翻轉，而不是數值為 0。這些比例不是精確的因果分解：填值也會改變乘法器內的部分積、對齊與累加。比例幾乎不隨時脈改變（FP64 保留 27 位：900、1300、1700 MHz 分別為 83、81、82%），總省電量則隨時脈增加（FP64 保留 0 位：每次運算少耗 20、24、29%），與 CV^2f 相對靜態功耗變大的預期一致。效益也不限於矩陣引擎：FP64 乘加迴圈保留 0 位在 600 W 快 33%，在 600 W 受限的記憶體複製快 28%。

是位元的變化，不是數值。填值也會改變數值本身。為了把「翻轉」和「數值」完全分開，我們在 MI350X 上再做三種對照，讓數值或 1 的個數固定，只改變相鄰運算元之間有多少位元變化（表 6.2，FP64 矩陣乘法、鎖頻 1300 與 1700 MHz）：

- **排序對打亂**：每一列（行）放同一組值，只是沿 k 方向遞增排列或偽隨機排列，

表 6.2　MI350X、FP64 矩陣乘法、鎖頻：沿 k 與跨列（A 的列、B 的行）相鄰運算元之間的平均位元翻轉數、每個元素 1 的個數，以及每次運算的能量（相對原生，1300 ／ 1700 MHz；8 張 GPU × ABBA，95% 區間在 ±0.5 個百分點以內）。

保留位數	排列	沿 k 翻轉	跨列翻轉	1 的個數	每次運算的能量（%）
52	打亂	27.2	27.2	34.5	+0.6 ／ +0.7
52	排序	23.2	21.2	34.5	−7.3 ／ −8.6
27	填 0	15.2	15.2	22.8	−8.4 ／ −9.6
27	填 1	15.2	15.2	47.8	−7.1 ／ −8.2
27	krep	15.2	27.7	35.3	−0.6 ／ −0.5
27	mrep	27.6	15.2	35.3	−3.7 ／ −4.2
0	填 0	1.7	1.7	9.3	−24.5 ／ −28.6
0	排序	0.0	0.0	9.3	−32.0 ／ −38.1
0	krep	1.7	27.7	35.3	−1.5 ／ −1.7
0	mrep	27.7	1.7	35.3	−11.4 ／ −13.4

資料來源：`mi350/results/mech_toggle.csv`（`mi350/analysis/mech_toggle.py`；job 109、110）

數值與 1 的個數完全相同。

- **krep**：被捨去的位元在每一列隨機、沿 k 方向重複（不是 0，也不是全矩陣共用的固定樣式）。
- **mrep**：krep 的轉置，沿 k 方向隨機、但所有列相同。

同一組值只改排列順序，每次運算的能量就差 8–9 個百分點（保留 52 位與 0 位皆然），而數值與 1 的個數完全相同；在第二個節點 n02 上重複，結果相差 0.3 個百分點以內（job 173）。krep 沿 k 方向不翻轉，卻幾乎不省電；mrep 沿 k 方向翻轉、但跨列不翻轉，省下約一半：矩陣引擎連續看到的運算元同時來自相鄰的列與相鄰的 k，而且兩者的權重不同。我們用實測運算元的逐位元翻轉數（`--stats-only`，資料產生方式固定，所以就是量測時的運算元）擬合 15 種排列的能量變化，並以「留一排列」的預測 R^2 評分（三次量測：1300、1700 MHz 與 n02）：翻轉總數 0.79、1 的個數 0.34；跨列與沿 k 分開再加交互項 0.93（最大誤差 5.8 個百分點）——跨列每次翻轉約 0.43%，沿 k 約 0.06%，交互項為負，也就是某個位元在兩個方向都不變時省得最多，所以 krep 與 mrep 加起來不等於填 0。依位元重要性加權反而更差（$R^2 < 0$），把符號、指數與尾數分開也沒有幫助（`mi350/analysis/mech_regress.py`）。這種不對稱暗示運算元路徑看到相鄰列的機會比相鄰 k 多（例如某個運算元在 lane 之間廣播），但我們看不到實際的資料流。這組對照只做了 MI350X 的 FP64。

鎖頻下仍隨資料而變的時脈。時脈鎖定、完全不受功耗限制時，有效時脈仍隨資料改變：鎖在 1300 MHz 時，原生 FP64 矩陣乘法穩定在 1171–1179 MHz（8 張 GPU

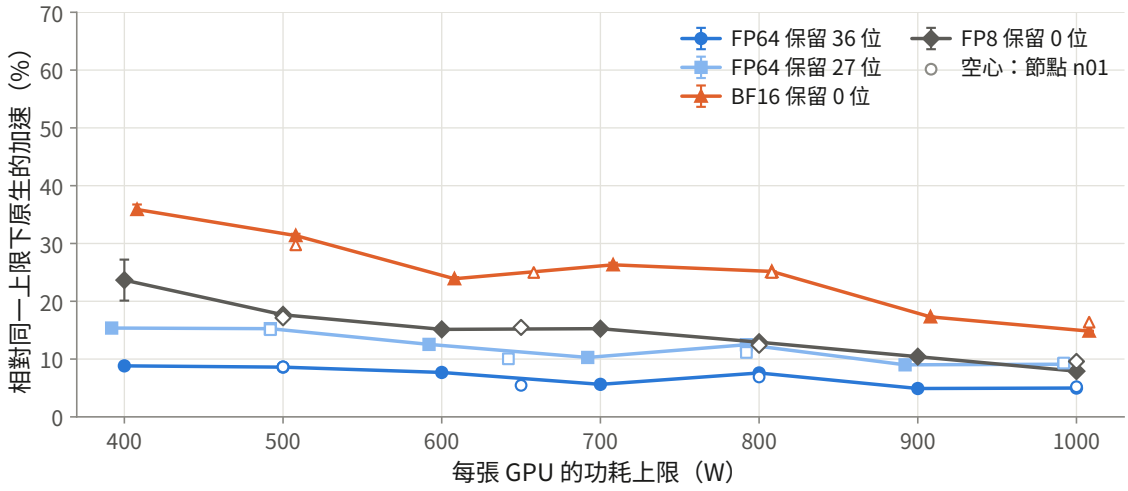


圖 6.1　MI350X 矩陣乘法在各封裝功耗上限下保留不同位數的加速。實心：節點 n02 兩個量測視窗的平均（每個視窗先平均同時執行的 8 張 GPU），誤差棒為兩個視窗的範圍；空心：節點 n01。部分系列左右錯開約 8 W 以免重疊。

資料來源：`mi350/results/e1n02_1029_0502_raw.csv`（n02）、`e1_1028_1709_raw.csv`（n01）

的第 5 到 95 百分位），保留 0 位則為 1235–1240 MHz；BF16 原生 1088–1100 MHz，保留 0 位 1132–1142 MHz，溫度只有 59–63 °C。功耗與溫度限制的時間都是 0，每個 XCD 的「時脈低於主機上限」計數器即使在功耗限制生效時也一直是 0，所以沒有任何現有的計數器回報這種降速。電源電壓瞬間下陷時的電路層級時脈延展（clock stretching）可以解釋這個現象，但我們沒有直接量測；截斷同樣減輕了它。第 4.6 節的模型以 σ 描述這個效應。

6.3　功耗上限下的矩陣乘法

測試使用 rocBLAS ／ hipBLASLt 矩陣乘法（FP64 為 $32768 \times 32768 \times 2048$，BF16 與 FP8 為 $k = 8192$），輸入為均勻隨機值，每個上限以 ABBA 順序量測兩個視窗。歸零在每個上限下都加速（圖 6.1）。在 1000 W，FP64 保留 36、27、0 位分別快 5.1、9.2、16.9%，BF16 保留 3、0 位快 7.7、15.6%，FP8 保留 0 位快 8.8%（兩節點共 4 個視窗的平均，95% 區間 $\pm$0.6–1.6 個百分點）；在 500 W，同樣的設定快 8.6、15.2、60.0、13.5、30.6 與 17.4%（$\pm$0.1–1.5）。兩個節點相差最多 1.8 個百分點，溫度校正改變任何數值都不超過 1.1 個百分點。FP64 保留 36 位在 600–900 W 相當於每張 GPU 多給 42–72 W。作為對照，GH200 在預設的 1000 W 下 FP64 保留 35 位只快 3.9%（第 5.2 節）。

這些短視窗不是暖機造成的假象：在穩態下（每個設定先執行 4 分鐘再量 2 分鐘），FP64 保留 36 位與 BF16 保留 0 位在 800 W 快 6.1 與 26.4%、1000 W 快 5.4 與

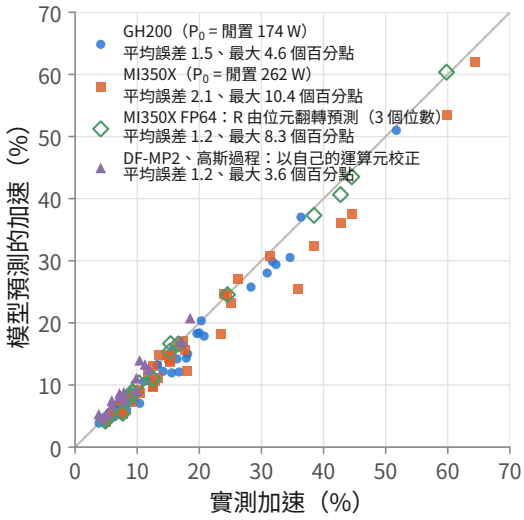


圖 6.2　功耗模型預測與實測的矩陣乘法加速，P_0 為實測的閒置功耗。每個點是一個功耗上限與格式的組合。空心菱形：R 由合成運算元的位元翻轉預測（3 個位數 × 7 個上限）；三角形：DF-MP2 與高斯過程在 500–1000 W，以各自運算元的鎖頻矩陣乘法校正。

資料來源：`mi350/results/model_eval_points.csv`（`mi350/analysis/model_eval.py`）

表 6.3　功耗模型預測加速的誤差（百分點），每個平台 42 個點。閒置：P_0 為實測閒置功耗；留一：以其他格式擬合 P_0 後預測被留下的格式；零：把所有功耗都當成動態功耗。

平台	P_0	平均誤差	最大誤差	FP64 27–36 位的最大誤差
GH200	閒置 174 W	1.5	4.6	2.9
GH200	留一（190–200 W）	1.0	3.6	—
GH200	0	6.1	12.9	—
MI350X	閒置 262 W	2.1	10.4	2.9
MI350X	留一（190–250 W）	2.3	9.4	—
MI350X	0	4.0	17.2	—

15.6%，與同一節點的短視窗（7.6、25.2、5.0、14.8%）相差不超過 1.5 個百分點（第 6.9 節）。

6.4　功耗模型的驗證

第 4.6 節的模型把 P_0 設為實測的閒置功耗（GH200 模組 174 W，MI350X 每張 GPU 262 W），不以驗證資料擬合任何參數。MI350X 以 1300 MHz 鎖頻的量測得到 R 與 σ（不使用上限掃描的任何一點）；GH200 無法鎖頻，以最低上限的速度推得 R（$\sigma = 1$，該點不列入評估）。每個平台各評估 42 個「上限 × 格式」的點（FP64 保留 0、12/27、35/36 位，BF16 保留 0、3 位，FP8 保留 0 位）。

平均誤差在 GH200 為 1.5 個百分點（最大 4.6），在 MI350X 為 2.1（最大 10.4；圖 6.2、表 6.3）。若改以其他格式擬合 P_0 再預測被留下的格式，誤差為 1.0 與 2.3；

把全部功耗都當成動態功耗（$P_0 = 0$）則增為 6.1 與 4.0；MI350X 若省略 σ 為 2.5。實測的比值為：FP64 保留 36、27、0 位的 $R = 0.94$、0.89、0.67，BF16 保留 3、0 位為 0.92、0.85，FP8 保留 0 位為 0.90，$\sigma = 1.01$–1.05。應用實際使用的 FP64 寬度（27–36 位）在兩個平台上誤差都不超過 2.9 個百分點；誤差最大的是 MI350X 上 BF16 保留 0 位在 400 W（原生幾乎跑不動），以及 FP64 保留 0 位在 500–800 W（時脈超出原生曲線的範圍）。

由位元翻轉預測。R 與 σ 也可以不必執行截斷版本就預測：在 1300 MHz 鎖頻下，用表 6.2 中 12 種「不是單純截斷」的排列擬合 R、σ 對翻轉變化的關係，再代入截斷後運算元的翻轉數，得到 36、27、0 位的 R 為 0.935、0.877、0.619（實測 0.934、0.887、0.654）；由此算出的 FP64 矩陣乘法在各上限下的加速，21 個點平均誤差 1.2 個百分點（最大 8.3），比用鎖頻實測校正的 2.3 還小（`mi350/analysis/switching_model.py`）。不過這 21 個點是 3 個位數 × 7 個上限，實際上只檢驗了三組預測的 R 與 σ，而且都來自合成運算元。但套用到整個應用還不夠：用 DF-MP2 張量的翻轉數預測 32、20 位的 R 為 0.90、0.81（實測 0.92、0.86），加速平均高估 4.4 個百分點（最大 9.0）。原因是 GEMM 形狀而不是資料：在 DF-MP2 的形狀（$2002 \times 302302 \times 5477$，$A$ 轉置）上改用合成均勻運算元，$R = 0.915$、0.848，與 DF-MP2 自己的運算元只差 0.005（job 299）；翻轉模型每次翻轉的能量屬於擬合時用的 $32768^2 \times 2048$ kernel，必須在目標形狀上重新擬合。

與簡單預測法比較。把某一個上限量到的加速直接套用到其他上限，平均誤差在 GH200 為 19 個百分點、MI350X 為 10（最大 73 與 48）；只用鎖頻下的能量比預測（MI350X）誤差 6.4，只用鎖頻下的時脈比 15.9。用最低與最高兩個上限量到的加速做線性內插，GH200 誤差 6.0，MI350X 只有 1.8，模型在同一組點上為 1.7 與 2.0；但內插需要對每個截斷版本做兩次受限量測，模型在 MI350X 一次都不需要（GH200 需要一次）：在 MI350X 上模型並沒有比較準，它的價值在於量測成本。留一擬合出的 P_0（190–250 W）低於閒置功耗，而滿載時的漏電只會更高，表示它吸收了共用的 $g(f)$ 形狀誤差，所以我們使用獨立量到的閒置值。因此這個一階模型適合估計收益大小、省去大部分的上限掃描；在兩種方法只差一兩個百分點的交叉點附近，它的誤差並不小（`mi350/results/model_baselines.csv`）。

外部驗證。上述矩陣乘法的點參與了模型的發展。為了在沒有參與的程式上檢驗，我們在節點 n02 上以 1300 MHz 鎖頻執行 DF-MP2 與高斯過程，從整個程式的功耗與時脈得到 R 與 σ（DF-MP2 保留 32、20 位：$R = 0.92$、0.86，$\sigma = 1.01$、1.02；高斯過程保留 36、28 位：$R = 0.93$、0.89，$\sigma = 1.01$、1.02，由程式內讀取的能量計

數器得到，見下段），再由同一節點上原生程式的效能曲線預測 500–1000 W 的加速。16 個點的平均誤差為 1.0 個百分點，最大 2.6。（先前以取樣器量的高斯過程整支程式得到 $R = 0.96$、0.93，各 GPU 之間從 0.91 散布到 0.95，而程式內量測只在 0.875–0.891 之間；那次的預測在每個上限都低估，原因不明。）

由應用自己的運算元校正（kernel 層級）。截斷版本的程式其實不必跑：把應用自己的運算元交給矩陣乘法微基準，在同樣的 1300 MHz 鎖頻、1000 W 下量原生與截斷的功耗與時脈（腳本 `kcal.sh`；bench 新增 `--load-a`/`--load-b` 讀入運算元，`gp_chol --dump-panel` 寫出 Cholesky 的 trailing-update 面板）。DF-MP2 取第 $i = 150$ 次呼叫（$2002 \times 302302 \times 5477$，約為以 flop 加權的平均呼叫），得到 32、20 位的 $R = 0.910$、0.844；高斯過程取兩個步驟 × 兩個 range 共四個面板，四者的 R 相差不到 0.003：36、28 位為 0.927、0.886（job 222）。只有矩陣乘法被截斷，所以整個程式的 $R_{\text{app}} = 1 - f(1-R)$、$\sigma_{\text{app}} = 1 + f(\sigma - 1)$，$f$ 是矩陣乘法佔原生**動態能量**的比例。第一版用的是時間比例（DF-MP2 0.96、高斯過程 0.90，job 227），預測誤差 1.0，看起來比整支程式校正的 1.6 還準，但高斯過程的整支程式 R（0.956 ／ 0.933）與時間比例推得的值對不上（第二位審稿人指出舊的解釋在算術上不成立）；$\beta = 1$ 的面板只讓 R 變 0.004–0.006（job 315），也不是原因。第九輪審查建議改用各階段的能量比例：在 `gp_chol` 加上 `GP_ENERGY=1`，在每次 trailing update 前後同步並以 amdsmi 讀硬體能量計數器（job 426，`gpenergy.sh`、`analysis/gp_energy.py`）。結果：非 GEMM 階段（POTRF、TRSM、共變異數、求解）只用 363 W，GEMM 階段 651 W，所以 GEMM 雖只佔 90% 的時間，卻佔 97% 的動態能量；程式內 GEMM 的 R 為 0.885 ／ 0.930，與面板的 0.886 ／ 0.927 相同；整支程式的 R 為 0.885 ／ 0.931，8 張 GPU 之間只差 0.016。也就是說，有問題的是先前那次取樣器量測，而不是運算元校正。改用能量比例（高斯過程 0.97、DF-MP2 仍用時間比例 0.96），16 點的平均誤差為 1.2（最大 3.6；不加權 1.5），與整支程式校正（改用程式內量測後 1.0）一樣準。DF-MP2 換成第 50、225 次呼叫，R 相差不到 0.004（job 296、297），所以第 150 次呼叫有代表性（圖 6.2 的三角形；`mi350/analysis/kernel_cal.py`，`results/kernel_cal.csv`，完整數字見英文補充材料表 S9）。

因此只要知道平台的閒置功耗與應用的原生曲線，一次以應用運算元進行的鎖頻矩陣乘法量測，就能把截斷版本的效能曲線預測到幾個百分點以內；同時脈的部分 $\Delta W_f \propto P - P_0$ 也說明了為什麼實測的等效功耗隨上限增加，而相對加速並不會：節點 n02 上 FP64 保留 36 位的等效功耗從 400 W 的每張 GPU 12 W 增加到 900 W 的 72 W，加速則從 8.9% 降到 4.9%。平均 1.5–2.1 個百分點的誤差相對 5–20 個百分點

表 6.4　MI350X（一個節點 8 張 GPU，1000 W）上 HPL 的殘差（門檻 16，重複執行完全相同）。位數為含隱藏位的有效位數，最近捨入（平手取偶）；(F) 表示未通過檢查。

N	原生	36 位	32 位	30 位	28 位
262144	5.4×10^{-5}	0.46	11.1	84 (F)	514 (F)
393216	4.6×10^{-5}	0.15	12.0	20.2 (F)	104 (F)
540672	7.8×10^{-6}	0.27	3.5	17.3 (F)	102 (F)

資料來源：`mi350/logs/hplbits262k_*`、`hplbits393k_*`、`hplbits540k_*`

的收益很小，所以模型可以估計收益、省去大部分的上限掃描；但相對交叉點附近一兩個百分點的差距並不小，我們不宣稱它能在那裡選出較快的方法。

6.5　HPL 與 HPL-MxP

安全的位數因平台而異。表 6.4 列出 MI350X 單節點 8 張 GPU 在三種問題大小下的殘差。28 位在每個大小都失敗（超過門檻 6–32 倍），30 位也是；32 位勉強通過；36 位有 35–100 倍的餘裕。GH200 上 28 位在四個大小中有三個通過（第 5.3 節）。兩個平台的 HPL 版本、DGEMM 的分塊與累加順序都不同，所以在一個平台驗證過的位數不能直接搬到另一個平台。

FP64 HPL。原生 HPL（$N = 540672$，1×8）在 1000 W 兩個節點分別為 424.9 與 429.8 TFLOP/s。保留 36 位在預設上限快 4.6–5.1%，800 W 快 4.5–4.9%，650 W 快 6.2–8.0%，500 W 快 6.3–7.3%（兩節點合併：1000 W 4.8 ± 0.3%、500 W 7.0 ± 0.6%，各 6 次執行；表 6.5），每次求解少耗能 4.6–6.8%，在 650–800 W 的等效功耗為每張 GPU 39–50 W（每個節點約 0.35–0.4 kW）。

HPL-MxP。原生 HPL-MxP（$N = 1034752$，4×2，對角優勢的測試矩陣）在 1000 W 為 7.61 與 7.65 PFLOP/s，保留 3 位時為 7.98 與 8.04 PFLOP/s（+4.9 與 +5.1%），在 500–800 W 快 9.2–19.5%，每次求解少耗能 5.3–14.6%。迭代修正從 2 次增為 4 次，約佔 LU 時間的 1.5%。保留 0 位在 MI350X 上不收斂，所以兩個平台的操作點都是 3 位。GH200 上的 HPL-MxP 在 1000 W 會觸發機箱的功耗煞車而不報；MI350X 沒有這個問題。這些結果只適用於對角優勢的測試矩陣：程式的產生器一律把對角元設為該列絕對值的和，我們沒有正式規則允許的產生器，所以 3 位的操作點與其收益都不是 HPL-MxP 的一般性質。不過它們不依賴對角 tile 的近似分解：改用精確分解（`--exact-diag`，節點 n02，一組 ABBA），保留 3 位仍通過殘差檢查（0.43），在 1000 W 快 4.4%、650 W 快 9.4%（溫度校正後 4.4 與 9.3%），近似分解則為 5.1 與 9.7%。在兩個節點（16 張 GPU，4×4，$N = 1453056$，每個變體 4 次，job 178）上，跨節點

通訊沒有稀釋收益：保留 3 位在 1000 W 快 6.0%（5.9–6.1；14.61 到 15.48 PFLOP/s，每次求解少耗能 5.6%），500 W 快 16.5%（12.7–20.3）；但 3 位的殘差升到 11.8（門檻 16）。單節點上 3 位的殘差反而隨 N 下降（$N = 264192$ 為 3.4–4.2、528384 為 1.4–1.9、792576 為 0.6–0.7、1034752 為 0.43–0.46；4 位約為一半，job 200–202），而在 16 張 GPU、與單節點相同的 $N = 1034752$ 下，3 位的殘差是 0.20，4 位反而是 5.7（job 214；這次 3 位與 4 位在 1000 W 分別快 3.5 與 3.9%）：原因在求解器的停止條件：只要 scaled residual 低於 16 就停，所以最後的殘差只反映停在哪一步，而不是 LU 的精度。單節點各 N 都是 1 步 Richardson 加 2 步 GMRES，步數相同時殘差隨位數與 N 單調下降；16 張 GPU、$N = 1034752$ 時 4 位做完 Richardson 就以 5.7 通過，3 位第一步只到約 24，要再做兩步 GMRES 才停在 0.20；兩節點 $N = 1453056$ 的 11.8 也是一步就通過。每個設定都跑了兩次並重現。所以該看的是迭代步數而不是殘差，安全的保留位數仍須逐一驗證。

非對角優勢的矩陣（第八輪審查）。為了回應「對角優勢矩陣太簡單」，在產生器加上 `HPLMXP_DIAG=sqrtn`：對角元改為 $\sqrt{n} \approx 1017$，每列非對角元絕對值和約 $n/2 \approx 2.6 \times 10^5$，完全不是對角優勢；隨機部分的特徵值落在以 $\sqrt{n}$ 為中心、半徑約 $\sqrt{n/3}$ 的圓盤內，所以仍可解（Dongarra 與 Luszczek 建議以 $[-1, 1]$ 均勻元素加平方根縮放）。使用精確的對角 tile 分解（近似分解依賴對角優勢），單節點 $N = 1034752$（job 317）：原生 6.97 PFLOP/s，1 步 Richardson 加 3 步 GMRES 通過（殘差 1.2–1.7；500 W 為 15.1–15.4）；保留 6、5、4 位步數相同，3 位多 1 步，全部通過（殘差 1.0–15.8）。1000 W 時 6 ／ 5 ／ 4 ／ 3 位快 1.2 ／ 2.9 ／ 4.2 ／ 5.0%，500 W 快 3.7 ／ 8.1 ／ 11.6 ／ 16.3%（各兩次）。對角優勢矩陣上 3 位為 +5.0 ／ +19.3%。這個產生器是我們自己的，不是官方的，所以操作點仍不是 HPL-MxP 的一般性質；這個修改只在本機子模組的 commit `08ab728`，沒有推到 NTHU-SC 的 fork。

6.6　DF-MP2 與高斯過程

DF-MP2。GPU4PySCF 不能在 MI350X 上執行，所以我們以 PySCF 在 CPU 上建立相同的三指標張量（taxol，def2-TZVP，RI 輔助基底），再以自寫的 HIP 程式在 GPU 上收縮：每一對佔據軌域 $(i, j \le i)$ 以一次 FP64 DGEMM 計算 $B_i B_j^{\mathsf{T}}$，內積長度為輔助基底的大小（$k = 5477$）。相關能與 PySCF 相差 $2 \times 10^{-13}\,E_\mathrm{h}$ 以內，與 GPU4PySCF 相差 $7.5 \times 10^{-8}\,E_\mathrm{h}$。張量建好後原地歸零一次（最近捨入）：保留 32 位時 MP2 相關能最多改變 $5 \times 10^{-12}\,E_\mathrm{h}$，20 位最多 $2.2 \times 10^{-8}\,E_\mathrm{h}$。在 1000 W 原生有 95% 的

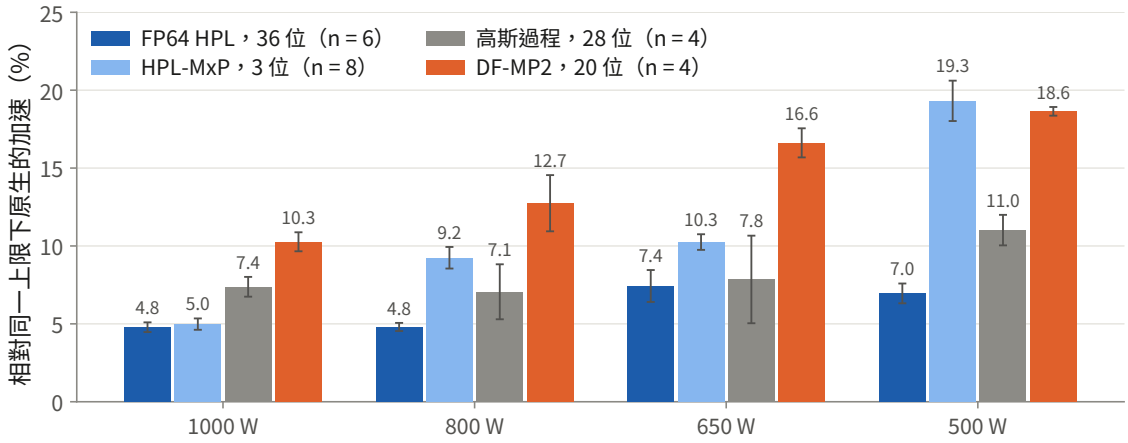


圖 6.3　MI350X 上程式內截斷相對未修改程式的加速，兩節點合併。誤差棒為分析單位的 95% *t* 區間：HPL 與 HPL-MxP 為整節點的一次執行，高斯過程與 DF-MP2 為 8 張 GPU 的一輪；*n* 列於圖例。各節點與溫度校正後的數值見表 6.5。

資料來源：`mi350/results/apps_units.csv`（`mi350/analysis/apps_units.py`）

表 6.5　MI350X 在預設上限與 500 W 的加速（%）。合併：兩節點所有分析單位的平均 ± 95% *t* 區間（視窗：8 張 GPU 平均的一個量測視窗；執行：整節點的一次執行；輪：8 張 GPU 的一輪）。n01、n02：各節點的原始值（相同散熱環境）／校正到相同「開始時」junction 溫度的值（第八輪改為開始溫度；1000 W 的受控實驗：DF-MP2 20 位 12.6 ± 0.1、高斯過程 28 位 7.3 ± 0.1）。

		1000 W			500 W		
結果	單位，*n*	合併	n01	n02	合併	n01	n02
矩陣乘法 FP64，36 位	視窗，4	5.1 ± 0.6	5.2/5.4	5.0/5.5	8.6 ± 0.1	8.6/8.6	8.6/8.8
矩陣乘法 BF16，0 位	視窗，4	15.6 ± 1.4	16.4/16.6	14.8/15.1	30.6 ± 1.5	29.8/29.8	31.4/31.7
矩陣乘法 FP8，0 位	視窗，4	8.7 ± 1.6	9.6/9.5	7.9/8.1	17.4 ± 0.6	17.1/15.9	17.7/17.7
HPL，36 位	執行，6	4.8 ± 0.3	4.6/4.5	5.1/4.8	7.0 ± 0.6	7.3/7.9	6.3/6.5
HPL-MxP，3 位	執行，8	5.0 ± 0.4	4.9/4.4	5.1/5.2	19.3 ± 1.3	19.5/19.0	19.1/20.5
高斯過程，28 位	輪，4	7.4 ± 0.6	7.6/8.2	7.1/8.0	11.0 ± 1.0	10.8/12.5	11.2/12.9
DF-MP2，20 位	輪，4	10.3 ± 0.6	10.2/11.5	10.4/11.8	18.6 ± 0.3	18.8/19.9	18.5/20.1
DF-MP2，32 位	輪，4	5.7 ± 0.4	5.5/6.6	5.9/6.7	9.8 ± 0.1	9.7/10.5	9.8/11.1

資料來源：`mi350/results/apps_units.csv`、`*_tempadj.csv`

時間撞上限，原生時間在 1000、800、650、500 W 分別為 21.7、26.4、34.1、55.8 s。兩節點共 4 輪，保留 32 位快 5.7 ± 0.4、6.7 ± 1.4、9.5 ± 0.7、9.8 ± 0.1%，保留 20 位快 10.3 ± 0.6、12.7 ± 1.8、16.6 ± 0.9、18.6 ± 0.3%。GH200 在 1000 W 只快 0.5–0.8%（主要表現為省電，第 5.5 節）；MI350X 在預設上限就快 5.7–10.3%。

高斯過程。我們把第 5.6 節的似然函數移植為 HIP 程式（rocBLAS ／ rocSOLVER 的分塊 Cholesky，$n = 80000$，Matérn-3/2，每個面板一次 FP64 DGEMM 做尾端更新，$k = 1024$），在每個面板做尾端更新前原地截斷（最近捨入）。對數似然約為 1.2–1.7 × 10^5，保留 36 位最多偏離 1.9 × 10^{-5}，28 位最多 5.3 × 10^{-3}，相對誤差約 3 × 10^{-8}，遠小於估計所需。兩節點共 4 輪，保留 36 位在 1000、800、650、500 W 快 3.9 ± 0.2、4.1 ± 0.6、5.3 ± 1.7、6.9 ± 0.7%，保留 28 位快 7.4 ± 0.6、7.1 ± 1.8、7.8 ± 2.8、

表 6.6　MI350X 上截斷與 GEMMul8 相對原生的加速：預設上限／500 W，括號內為誤差。矩陣乘法（$m = n = 32768$）、DF-MP2、高斯過程：截斷 35、20、28 位，GEMMul8 12 個模數（每一項都比截斷精確）。HPL：各方法在已測設定中，殘差低於 HPL 本身門檻（16）或作為敏感度分析低於 1 時最快的設定，相對同一節點最快的原生設定（表 6.7）。

工作負載	截斷	GEMMul8
FP64 矩陣乘法，$k = 1024$	+5.1/+9.2%（8×10^{-12}）	−13/−45%（4×10^{-13}）
FP64 矩陣乘法，$k = 4096$	+5.5/+9.0%（8×10^{-12}）	+84/+18%（8×10^{-13}）
HPL，殘差 < 16	+5.7/+7.3%（3.5/0.27）	+34/+12%（1.79）
HPL，殘差 < 1	+5.1/+7.3%（0.27）	+17/−4.0%（0.033）
DF-MP2（$\lvert\Delta E\rvert$，E_h）	+10/+19%（2×10^{-8}）	+67/+34%（7×10^{-12}）
高斯過程，$n = 64000$（$\lvert\Delta\ell\rvert$）	+5.9/+11.8%（2×10^{-3}）	−11/−43%（3×10^{-7}）
HPL-MxP（BF16）	+4.9/+19%（殘差 0.5）	沒有對應的模擬

$11.0 \pm 1.0\%$。完整的三參數最大概似估計（變異數以解析式 profile 掉，range 與 nugget 以 Nelder–Mead，$n = 80000$，兩組資料，job 111）中，保留 28 位讓估計值最多偏移 0.25%（變異數）、0.24%（nugget）、0.09%（range），36 位在 0.004% 以內；兩組資料本身估出的 range 就相差約 2%。

6.7　與 INT8 模擬（GEMMul8）的正面比較

MI350X 的 INT8 與 FP64 吞吐量比約為 GH200 的兩倍，對 INT8 模擬較有利。表 6.6 讓兩種方法各用自己允許的最佳設定並列比較：在內積長、上限寬鬆、模擬的固定成本能攤銷時（HPL、DF-MP2、$k \geq 2048$ 的矩陣乘法），模擬大幅領先；上限收緊時領先縮小，甚至反轉；內積短時（高斯過程、$k = 1024$ 的矩陣乘法）模擬比原生還慢。截斷的收益較小，但處處為正、隨上限收緊而增加、不需要額外記憶體，而且適用於沒有模擬可用的 BF16 與 FP8。

分界：形狀與上限。圖 6.4 是 $m = n = 32768$ 時兩種方法相對原生 FP64 的加速隨內積長度 k 的變化。截斷到 35 位（誤差 7.8×10^{-12}）在每個 k 都比原生快，1000 W 快 5–6%、500 W 快 8–9%。GEMMul8 用 12 個模數（較精確，約 5×10^{-13}）在 1000 W 時，$k = 1024$ 比原生慢 13%，$k = 2048$、4096、8192 則快 36、84、121%；在 500 W 變為 −45、−18、+18、+48%。$k = 2048$ 時，模擬在 1000 與 700 W 勝出，截斷在 500 W 勝出。模擬一直用滿功耗上限，所以受限時額外的整數運算要以時脈支付；截斷則是從功耗預算中拿掉工作。GH200 上的 GEMMul8 也是如此，而只用約 706 W 的 cuBLAS 模擬在低上限反而相對有利（第 5.7 節）：方向取決於模擬本身的功耗。

相同精度下的比較。圖 6.4 只比較了固定的設定。圖 6.5 改為掃描截斷保留的位

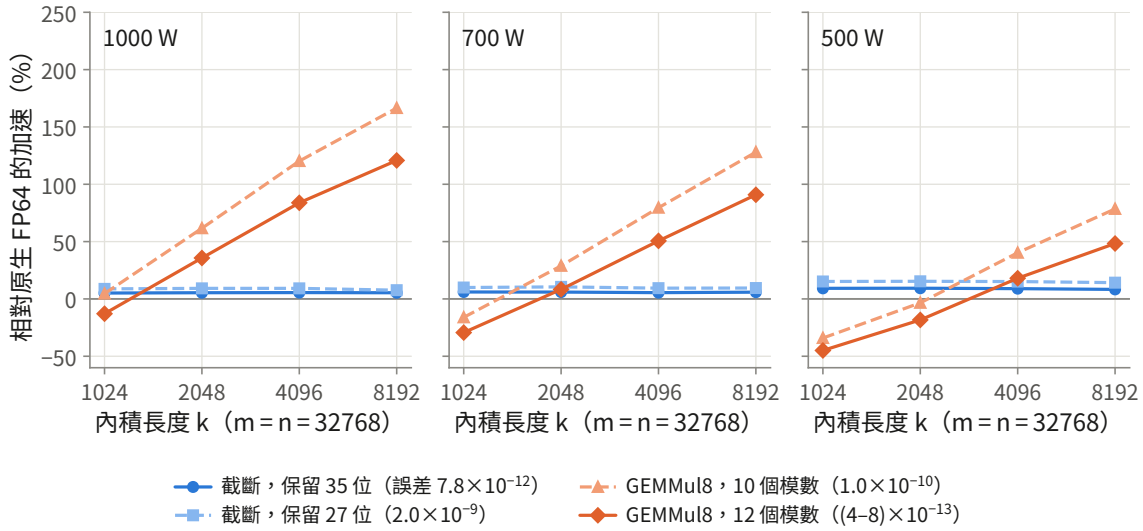


圖 6.4　MI350X 上截斷（最近捨入）與 GEMMul8 相對原生 FP64 的加速隨內積長度 k 的變化（$m = n = 32768$，節點 n02，8 張 GPU × ABBA），三個封裝功耗上限。上限收緊時，模擬的曲線全部下降，截斷的曲線全部上升。

資料來源：`mi350/results/xover_1029_1508_summary.csv`

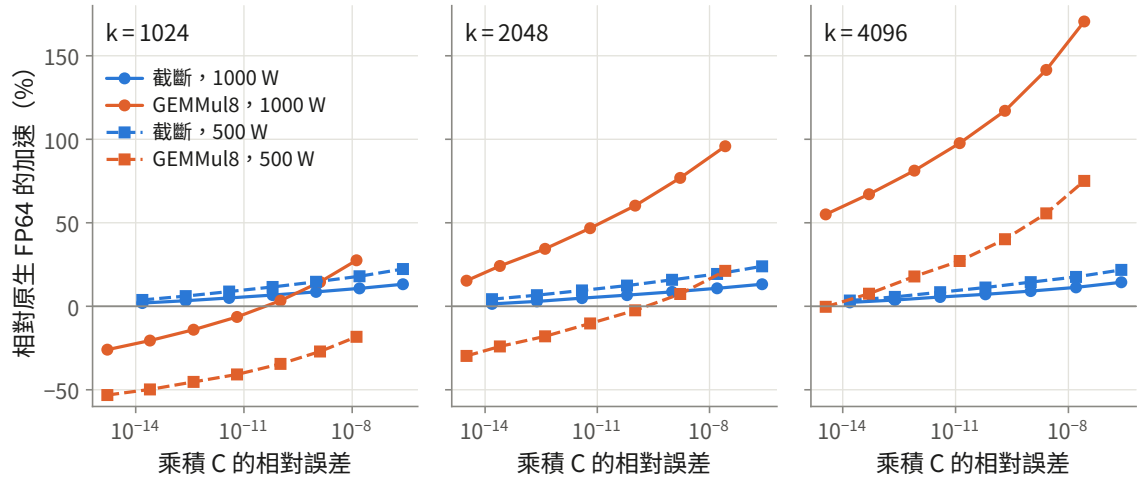


圖 6.5　MI350X 上截斷（最近捨入，保留 44 到 20 位）與 GEMMul8（8 到 14 個模數）相對原生 FP64 的加速對乘積相對誤差（$m = n = 32768$，節點 n02，8 張 GPU × ABBA）。

資料來源：`mi350/results/pareto_1101_0953_summary.csv`（job 104）

數（44 到 20 位）與 GEMMul8 的模數（8 到 14 個），在相同的乘積誤差下比較速度。$k = 4096$ 時，除了 500 W 下最嚴格的精度，模擬在每個精度都較快；$k = 2048$ 時，1000 W 下模擬在每個精度都較快，500 W 下只有誤差容許達 10^{-8} 以上時才較快（job 204）。$k = 1024$ 時勝負取決於精度與上限：1000 W 下，誤差容許大於約 10^{-10} 時模擬勝出（8 個模數在 1.3×10^{-8} 快 27%，截斷 24 位快 10%），更嚴格時截斷勝出（5×10^{-12} 附近 +4.8 對 −6.2%）；500 W 下截斷在每個精度都勝出。

應用。DF-MP2（$k = 5477$）中，GEMMul8 用 12 個模數比原生快 34–67%，每個上限都勝過截斷；若只要求與截斷 20 位相同的精度（$2.2 \times 10^{-8}\, E_\mathrm{h}$），GEMMul8

表 6.7　MI350X 上 HPL 對共同殘差門檻的敏感度：各方法殘差低於門檻、在**已測**設定中最快者，相對同一節點、同一上限下最快原生設定的加速（%）。

	殘差 < 16		< 1		< 0.1	
上限	截斷	模擬 10	截斷	模擬 12	截斷	模擬 12
1000 W	+6.6	**+34**	+5.1	**+17**	+4.5	**+17**
800 W	+6.7	**+33**	+5.0	**+15**	+4.4	**+15**
650 W	+8.7	**+25**	+8.0[a]	+7.8[a]	+6.4[b]	+7.8[b]
500 W	+9.4	**+11**	**+7.3**	−4.0	**+5.9**	−4.0

16 是 HPL 本身的門檻，1 與 0.1 為敏感度分析。候選（皆含原生）：1 × 8、N = 540672 的 31、36、37、39 位，四個上限（32 位只有 1000 W）；1 × 8 的 N = 262144、393216、458752；4 × 2、NB 4096 的 N = 491520（36 位、10 個模數）與 458752（36 位、12 個模數），四個上限。GEMMul8 在 1 × 8 或更大的 N 放不下。截斷取符合門檻的最快者，皆為 1 × 8、N = 540672：< 16 為 31 位（殘差 6.4，job 318），< 1 為 36 位（0.27），< 0.1 為 37 位（0.087，job 318）；模擬 10、12 為 GEMMul8 的模數（N = 491520，1.79；N = 458752，0.033）。粗體為排序確定時測得效能較高者。[a] 同一工作內配對（n02，job 319，4 個 ABBA 區塊）時 36 位與 12 個模數分別快 6.7 與 8.0%：模擬減截斷 1.2 ± 2.4 個百分點，排序未定。[b] 不同工作，沒有配對區間。

資料來源：`mi350/results/hpl_threshold.csv`（`mi350/analysis/hpl_threshold.py`）

只需 9 個模數（$2.4 \times 10^{-8}\, E_{\mathrm{h}}$），在 1000、650、500 W 快 112、86、65%，截斷只快 12–20%（3.5.4 直接呼叫，job 203）。但對保留 20 位的領先從 1000 W 的 57 個百分點縮小到 500 W 的 16 個百分點（GH200 上的 cuBLAS 模擬反而比原生慢 18–35%）。FP64 HPL 中，我們指定共同的殘差門檻，讓每種方法在**已測的**設定中各自選出符合門檻、最快的一個（表 6.7；分母是同一節點、同一上限下最快的原生設定）。HPL 本身的門檻是 16；更嚴格的門檻只是敏感度分析，不是 HPL 的要求，也不代表一般應用需要的精度（應用的需求由 DF-MP2 與高斯過程的案例說明）。以 HPL 的門檻來看，GEMMul8 用 10 個模數（殘差 1.79；N = 491520，NB 4096，4 × 2）在每個上限都領先：1000 W 達 576.8 TFLOP/s，比同一節點最快的原生設定快 34%，截斷（31 位，殘差 6.4）只快 6.6%；500 W 為 11.3 對 9.4%（截斷值來自第八輪在 n02 的補測，job 318）。門檻收緊到 1，10 個模數就不合格；12 個模數（0.033）最大只放得下 N = 458752，在 1000 W 比最快的原生快 16.9%，在 500 W 卻慢 4.0%，而 36 位截斷快 7.3%，排序明確反轉。650 W 原本只有不同工作之間的比較（截斷 8.0 對模擬 6.2%）；第八輪在同一個工作內配對（job 319，n02，4 個 ABBA 區塊），12 個模數與 36 位分別快 8.0 與 6.7%，差距 1.2 ± 2.4 個百分點，仍分不出先後。門檻收緊到 0.1 時，37 位在最佳原生設定上就通過（殘差 0.087），快 4.4–6.4%，在 500 W 領先、650 W 略低於模擬的 7.8%（不同工作，沒有配對區間；原本只量到 40 位在較小的 N，數值是下界）。因此，收緊共同殘差門檻會改變最緊的幾個上限下測得的排序。

高斯過程的尾端更新 k = 1024，GEMMul8 在每個上限都比原生慢（n = 64000 時結果正確：12 個模數在 1000、800、650、500 W 為 −11、−22、−37、−43%，對

數似然差 3×10^{-7} 以內），保留 28 位的截斷則快 5.9–11.8%。

只在應用中出現的成本。模擬需要工作空間：最佳原生 HPL 的 $N = 540672$ 時，矩陣本身就佔每張 GPU 288 GiB 中的 272 GiB，GEMMul8 的工作空間放不下；1×8 的格網即使在 $N = 458752$ 也要求 100–114 GB；12 個模數連 $N = 491520$ 都失敗，HPL 必須先縮小問題才能使用模擬。透過 BLAS 攔截使用時，每遇到一種新的矩陣形狀都有一次很大的初始化成本：DF-MP2 第一次走過 226 種形狀要 138–141 s，而不是 13 s，GEMMul8 3.2.0 與最新的 3.5.4 都一樣（上述時間已扣除這次暖機）；改為直接呼叫函式庫、只配置一次工作空間，這個成本就消失（兩次都是 12.8–13.0 s，job 123），代價是要修改應用程式。

靜默的錯誤。高斯過程在 $n = 80000$ 時，第一次尾端更新（$m = n = 78976$，$k = 1024$）被攔截的 DGEMM 回報成功，結果卻是錯的，256 次執行無一例外，也沒有任何訊息。我們以均勻隨機輸入寫了一個最小重現程式（`mi350/apps/g8_repro.hip`），精確定出界線：不論直接呼叫函式庫還是透過攔截，$k = 1024$ 的 $C = AB^{\mathsf{T}}$ 在 $m = n = 65535$ 時正確（相對誤差 1.6×10^{-15}），從 $m = n = 65536$ 起就錯（相對誤差 31，所有元素都是有限值），也就是 mn 達到 2^{32} 時；最新的 3.5.4 版仍然如此（job 122）。原因在更下一層：GEMMul8 把多個模數的 INT8 乘積合成一次 strided batched INT8 GEMM，輸出間隔為 mn；ROCm 7.2.3 的這個呼叫（rocBLAS 與 hipBLASLt 兩種後端都一樣）在間隔達到 2^{32} 個元素時，回傳成功，卻完全沒有寫入第一個 batch 之後的結果（`mi350/tools/int8_gemm_check.hip`）。這是實作上的缺陷，與資料無關，也不是模擬方法本身的性質；本報告其他的模擬結果都在這個大小以下。我們只把它當成整個替換 BLAS 呼叫時的部署風險：在一個大小驗證過的設定，換一個大小就可能安靜地出錯。

指數跨度。與 GH200 相同（第 5.8 節），均勻合成的指數散佈幾乎不影響 GEMMul8：每個元素或每列的指數散佈到 64 個 binade，12 個模數的逐元素誤差中位數仍為 5.7×10^{-13}–8.0×10^{-13}，第 99 百分位最多 4.4×10^{-11}。GH200 上的失效證據來自真實資料（纈胺黴素），列內跨度寬只是必要條件。

在模擬之內再截斷。先截斷輸入再交給 GEMMul8，希望降低 INT8 引擎的翻轉：在受限時只多 1–4 個百分點（12 個模數、19 位輸入，500 W 從快 18.4% 變為 22.6%），在 1000 W 沒有差別，誤差卻變成截斷的誤差（5.1×10^{-7}）；不截斷、改用 8 個模數更快也更準。在模擬之內，該調的是模數的個數，不是輸入的位數。

表 6.8　逐次呼叫選擇與固定選擇的比較：選出實測最快、且符合誤差要求之方法的比例（括號內為相對該選擇平均損失的時間，%）。矩陣乘法：逐形狀留一，MI350X 22 種形狀 × 3 個上限 × 2 種容許誤差，GH200 20 種形狀 × 3 個上限。DF-MP2：每次呼叫 × 3 個上限（12 個模數對 20 位）。高斯過程：整個執行，區塊大小即 k（12 個模數對 35 位）。跨度檢查：補充資料圖 S1 的六個真實資料案例。擬合都沒有用到應用的資料。

驗證	案例數	逐次選擇	永遠模擬	永遠截斷
MI350X 矩陣乘法	132	97.7（0.1）	51.5（39）	48.5（24）
GH200 矩陣乘法，10^{-9}	60	98.3（0.1）	40.0（72）	60.0（24）
GH200 矩陣乘法，10^{-11}	60	83.3（1.3）	20.0（94）	80.0（3.0）
DF-MP2 的每次呼叫	678	99.3（0.0）	99.3（0.0）	0.7（29）
高斯過程	24	22（0.3）	12（16）	12（11）
跨度檢查	6	5/6	—	—

資料來源：`mi350/results/selector_cv.csv`、`selector_mp2.csv`、`selector_gh200.csv`、`selector_span_guard.csv`、`gpsel_summary.csv`

6.8　逐次呼叫的選擇

上述的規律可以寫成一條逐次呼叫的規則。對一次 GEMM 呼叫 (m, n, k)、功耗上限 P 與可容許的乘積相對誤差 ε，每種方法先選出實測誤差不超過 ε 的最便宜設定（保留位數或模數），再選預測時間最短的方法。截斷的加速幾乎與形狀無關（35 位在 22 種形狀上：1000、650、500 W 分別快 4.6–6.8、5.0–7.4、8.0–9.8%），所以每個上限一個數字即可（也可用第 4.6 節的模型）。模擬相對原生的時間則以 $r_e = a + b/k + c\,(1/m + 1/n)$ 描述，每個上限與模數各擬合一次（22 種形狀：$m = n$ 從 4096 到 32768、k 從 512 到 8192，另加兩種長方形；GEMMul8 3.5.4；$R^2 = 0.97$–0.99，job 159、160）；透過攔截使用時再加上每種形狀一次的初始化成本。這條規則只用矩陣乘法的量測。

另外加上跨度檢查：若元素比所在列最大值小 2^{-32} 以上的比例（攔截工具可以便宜地量測，第 5.8 節）超過 1%，就不使用固定位數的模擬。

MI350X 上，這條規則在保留下來的矩陣乘法案例中有 97.7% 選到最快的方法（表 6.8）；最好的單一 k 門檻（$k \geq 2048$，用全部資料挑選，屬於樣本內，而規則是留一形狀評分，因此門檻的數字偏樂觀）是 92.4%，但最多損失 33%。在 GH200 上用同樣的規則重新擬合（cuBLAS 模擬、GEMMul8、三個上限），容許誤差 10^{-9} 時選對 98.3%，10^{-11} 時只有 83.3%，因為那時幾乎總是截斷最好（永遠截斷也有 80%）。DF-MP2（$k = 5477$）沒有鑑別力：99.3% 的呼叫都是模擬最快，規則與「永遠模擬」相同；它每次呼叫的預測時間比與實測相差 0.05–0.13。有鑑別力的是高斯過程，因為區塊大小決定 k（1024 到 4096），而交叉點隨上限移動：1000 W 時 $k \geq 1536$ 就是模擬較快，800 W 要 $k \geq 2048$，650 W 要 $k \geq 3072$，500 W 只有 $k = 4096$（例

如 $k = 2048$：1000 W 模擬 +21.7 對截斷 +4.1%，500 W 則是 −19.8 對 +8.3%；job 174–176、194–199）。規則在 24 次執行中選對 22 次，兩次失誤都在交叉點附近、差距不到 4%；永遠模擬或永遠截斷都只對 12 次，最好的單一 k 門檻對 18 次。跨度旗標標出會讓模擬出錯的高斯基底 DFT 資料，也標出模擬其實正確的高斯過程；只憑旗標拒絕模擬，在高斯過程上最多損失 32%（$k = 4096$、1000 W）。第九輪審查指出，「交給應用自己的檢查」在纈胺黴素上就不成立：cuBLAS 模擬讓 SCF 照常收斂到差 52.5 mEh 的能量，SCF 的收斂判準正是應用自己的檢查。規則因此改為：模擬的呼叫在每種形狀的前幾次做**逐元素**抽查（門檻 10^{-6}），不通過就以原生重算；若錯誤是寬跨度的損失而不是整體性錯誤，這個應用與輸入要先做一次「原生對模擬」的參考比較，達到應用的容許誤差才允許模擬。GH200 上這個比較會擋下纈胺黴素（52.5 mEh），MI350X 上會放行高斯過程（$|\Delta\ell| \le 4.2 \times 10^{-7}$）；跨度寬只用來預測哪些輸入需要這一步。表中高斯過程的 22 ／ 24 是在參考比較已放行模擬的前提下計算的。

事後 FP64 抽查。另一種做法是檢查結果而不是資料：模擬算完之後，用原生 DGEMM 重算 C 的 64 個隨機列（$n > m$ 時沿較大的維度抽欄），以 normwise 誤差與模擬的結果比較（`mi350/apps/g8_spot.hip`、`spot.sh`、`spot2.sh`，job 223、237、295）。

- **抓得到整體性的錯誤**。GEMMul8 在 $m = n = 65536$（ROCm 的 2^{32} 錯誤）時誤差 32，抽查估到 31.6；65535 時正常。合成「隱藏」資料（每項乘積都比列最大值小 2^{-40}）上 10、12、14 個模數錯 100%、25%、0.1%，抽查估計相差不到 0.3%。
- **抓不到寬跨度的損失**。第一位審稿人要求在真的會失敗的資料上驗證。我們在 n02 的 CPU 上以 PySCF 重做 GH200 上的纈胺黴素 DFT（DF-B3LYP ／ def2-TZVP，`mi350/apps/valino_prep.py`，SCF 1383 s），總能量 −3795.5392524229 Eh，與 GH200 的 GPU4PySCF 相差 10^{-10} Eh；取出三種交換相關（XC）GEMM 的真實運算元在 MI350X 上跑 GEMMul8。normwise 誤差（10 ／ 12 模數約 10^{-10} ／ 10^{-12}）與截斷 28–36 位同級，抽查估計也在 3 倍以內，所以 normwise 抽查會放行；但逐元素誤差 $\max|\Delta C|/(|A||B|)$ 達到 1，有些小元素被整個丟掉，截斷 36 位只有 10^{-11}。高斯過程的面板一樣逐元素誤差為 1，但似然值完全不受影響。在同樣 64 列上改算逐元素誤差（多一次對 $|A|$、$|B|$ 的 DGEMM，忽略 $|A||B| < 10^{-290}$ 的元素）就抓得到：XC 運算元上為 1.0，截斷為 10^{-11}；但它同樣會標出高斯過程（1.0），而高斯過程的結果無害。所以矩陣乘積本身分不出損失有沒有害，收斂的 SCF 也不是判準；要靠一次原生對模擬

的參考比較來判定（見下一節的規則）。逐元素抽查的成本在 DF-MP2 與高斯過程的形狀上是模擬呼叫的 0.6–0.7%，在小 XC 形狀上會超過呼叫本身，所以只在每種形狀的前幾次呼叫做。

- **成本取決於形狀**。DF-MP2 與高斯過程的形狀上是模擬呼叫時間的 0.2–0.6%，瘦長的 MO 形狀 1–1.5%，但 $m = n \le 2958$ 的小 XC 形狀要 44–75%，因為模擬本身很快。

原本摘要說「抽查能抓到寬跨度資料上的靜默失敗」，這週的實驗顯示不成立，論文已修正。逐元素誤差的分布（job 324）：12 個模數時，逐元素誤差超過 10^{-6} 的元素在 XC $k = 4096$ 佔 19%、XC $k = 76800$ 佔 62%、MO 形狀為 0、高斯過程面板 95%；截斷 36 與 32 位沒有任何元素超過 10^{-9}（XC $k = 76800$ 上截斷的最大值 0.064 來自 31189 個 $|A||B| < 10^{-290}$、接近下溢的元素）。

從應用的容許誤差到設定。規則的容許誤差是矩陣乘積的誤差。對高斯過程，可以用一個係數 κ 把對數概似的容許誤差 τ 換算過去：$\varepsilon = \tau/\kappa$，其中 $\kappa = |\Delta\ell|/\varepsilon = 4.1 \times 10^6$，取先前兩次截斷執行（$n = 64000$、區塊 1024、28 與 36 位）中較大者。$\tau = 10^{-2}$、$10^{-4}$、$10^{-6}$ 時規則分別選 28、36、40 位或 10、11、13 個模數；另一次獨立執行（$n = 64000$、區塊 2048、1000 與 500 W，job 224）中，規則六個案例全部選對（1000 W 模擬、500 W 截斷），六個設定中有五個在 τ 以內。第六個是 40 位：$|\Delta\ell| = 1.2 \times 10^{-6}$，超過 10^{-6}，因為 κ 在不同執行之間相差四倍（$1.3\text{–}5.0 \times 10^6$），所以換算時要留一兩個位元的餘裕。模擬的實際誤差比這個以截斷校正的預測小約十倍，會讓規則多選模數、對模擬不利；改用模擬自己的 $\kappa_e = 5.2 \times 10^5$（由先前 12 模數的 gpsel 執行估計，排除區塊 2048 以保持獨立），$\tau = 10^{-2}$、10^{-4}、10^{-6} 分別選 9、11、12 個模數，實測 $|\Delta\ell|$ 為 7.4×10^{-4}、3.6×10^{-6}、1.7×10^{-7}，全部在 τ 內，勝負也不變（1000 W 模擬 +51 ／ +30 ／ +22%，500 W 截斷較快；job 298）。κ 也不必靠完整的執行來估：它大致與 n 成正比，$n = 16000$、32000 的幾秒鐘執行（job 320、321）得到 $\kappa/n = 36\text{–}77$，$n = 64000\text{–}80000$ 為 25–78；用小 n 的最大值 $77n$，$\tau = 10^{-6}$ 時會改選 44 位而不是 40 位，正好補上那個超出的案例（`mi350/analysis/gptol.py`，`results/gptol.csv`）。截斷這一側的 R 可以由運算元預測；模擬這一側則沒有翻轉項。

6.9　量測陷阱：風扇與溫度

在 MI350X 的 800 W 下，一次三個變體（原生、4 位、3 位）的 ABBA 序列讓 HPL-MxP 保留 4 位看起來比 3 位快（+11.3 對 +9.3%）。原因是機箱風扇：一次高溫的執行之後，風扇加速，junction 溫度在 20 s 內下降 25 °C，降溫條件立刻滿足，下一次執行便低溫約 12 °C；排在中間的變體永遠跟在高溫執行之後。改成每次只比兩個變體的成對 ABBA，並規定最短休息時間，只減輕而沒有消除這個效應：高溫執行之後的那次仍可能低溫 15 °C、快 4.5%，因為較冷的晶片漏電較少，留給切換的功耗預算較多。

溫度因此以兩種方式出現，不能混為一談。前一次執行與風扇造成的差異是干擾因素；截斷的程式本身耗電較少而較冷，則是它的收益之一（只在不受功耗限制時；受限時兩者都吃滿上限）。我們報告兩個量：**相同散熱環境下的收益**是原始的成對比值，由機箱自己控制風扇，是使用者實際得到的；**相同溫度下的資料效應**則在每個上限內，以變體與該次執行的平均 junction 溫度對效能的對數做迴歸（係數為每 °C −0.1 到 −0.9%），移除了干擾，但在截斷本身會降溫時也移除了部分收益。以溫度校正後，被偏誤的那一格中 3 位為 +9.0%（成對重跑 +9.1%，n02 為 +9.1%），4 位為 +4.4–6.2%；其他 HPL 與 HPL-MxP 的格子校正後最多移動 0.7 個百分點。高斯過程與 DF-MP2 的變體在同一個程序中依序執行（原生、32/36 位、20/28 位，再反向），截斷的變體位在每一輪中間最熱的部分；以平均溫度校正，它們的收益會增加 0–4 個百分點（DF-MP2 在 1000 W：10.2 增為 13.3%、10.4 增為 14.2%）。第二位審稿人指出，執行中的平均溫度含有截斷本身造成的降溫（中介變數），拿它當校正變數會把截斷的部分收益也扣掉。改用執行開始時的溫度（窗口開始前 1 s 到開始後 0.5 s），DF-MP2 為 11.5 ／ 11.8%，高斯過程 8.2 ／ 8.0%（表 6.5 已改為這個版本）；但 HPL 每次都先冷卻到同一溫度，開始溫度幾乎相同，校正等於沒做，也修不好上面那格 800 W 的 HPL-MxP（開始溫度校正後 4 位仍為 12.0%）：真正的干擾是風扇狀態，兩種溫度都看不到。所以第八輪補做了這兩個程式的受控實驗（job 316，`mi350/scripts/thermal_apps.sh`）：每輪先以 8 張 GPU 跑 180 s 原生 BF16 GEMM 預熱，不等冷卻，再以 ABBA 與 BAAB 交替各 6 輪（1000 W）。DF-MP2 20 位快 12.6 ± 0.1%（ABBA 12.7、BAAB 12.6），高斯過程 28 位快 7.3 ± 0.1%。DF-MP2 的原始值低估了約 2 個百分點，真值落在兩種迴歸校正之間；高斯過程的原始值本來就對。不過四種方法的差距（DF-MP2 10.3–14.2%、高斯過程 7.1–8.2%）比所有統計區間都大；第九輪審查後，主要結果一律報原始值（使用者實際拿到的，對 DF-MP2 也最保守），四種方法的範圍另外標示。

相同的平均溫度不代表相同的風扇轉速或熱歷史，所以我們在兩個關鍵設定上直接控制熱歷史，兩者都重現了校正後的數值：

- **穩態**（節點 n02）：每個設定先執行 4 分鐘矩陣乘法、再量 2 分鐘，各自達到穩態。FP64 保留 36 位與 BF16 保留 0 位在 800 W 快 6.1 與 26.4%，1000 W 快 5.4 與 15.6%，與短視窗（7.6、25.2、5.0、14.8%）相差 1.5 個百分點以內。
- **相同的熱歷史**（節點 n01，800 W，成對 ABBA，每個變體 4 次）：每次 HPL-MxP 執行前，8 張 GPU 都先執行同樣 3 分鐘的原生 BF16 矩陣乘法。3 位快 10.4 ± 2.8%、4 位快 6.0 ± 3.5%（原始），校正後為 9.0 與 6.2%，與先前各工作的校正值（9.0–9.1 與 4.4–6.2%）相同；原始數據的排序也恢復正常。

我們沒有權限固定風扇轉速，這仍是限制之一（第 7.3 節）。

6.10　小結：兩個平台的異同

表 6.9 並列兩個平台的主要結果。機制（翻轉）、應用的排序與「需要以多組大小驗證位數」在兩個平台上一致；不同的是預設上限是否受限、安全的位數，以及與 INT8 模擬的分界。MI350X 的預設上限就壓住時脈，所以截斷在出廠設定下就快 4.6–10%；FP64 相對 INT8 較弱，使模擬在內積長、上限寬鬆時大幅領先，但在嚴格上限、內積短、記憶體吃緊時仍是截斷較好。

表 6.9　GH200 與 MI350X 的主要結果比較。GH200 的 HPL、HPL-MxP 為 8 個節點；MI350X 為一個節點的 8 張 GPU。

項目	GH200	MI350X
預設上限是否受限	部分時間（FP64 矩陣乘法 95%）	是（矩陣乘法 82–100%）
填值保留的效益	62–97%	41–83%
FP64 矩陣乘法 35/36 位，1000 W	+3.9%	+5.1 ± 0.6%
HPL 36 位	500–800 W：+3.3–4.7%	1000 W：+4.8 ± 0.3%；500 W：+7.0 ± 0.6%
HPL 安全位數	28 位在 4 個大小中通過 3 個	28、30 位全部失敗；36 位安全
HPL-MxP 3 位	500 W：+5.8%；600–800 W：+1.7–2.3%	1000 W：+5.0 ± 0.4%；500 W：+19.3 ± 1.3%
DF-MP2 20 位	500–700 W：+7.4–7.7%；1000 W：+0.8%	1000 W：+10.3 ± 0.6%；500 W：+18.6 ± 0.3%
高斯過程 28 位	500–700 W：+7.5–7.9%；1000 W：+3.8%	1000 W：+7.4 ± 0.6%；500 W：+11.0 ± 1.0%
功耗模型平均誤差	1.5 個百分點	2.1（兩個應用：1.6）
INT8 模擬	cuBLAS 模擬多半較慢；GEMMul8 只在長內積勝出	GEMMul8 在長內積、寬鬆上限大幅領先；嚴格上限、短內積時截斷較好

第七章　討論

7.1　適用條件

尾數歸零的效益取決於以下三個條件，缺少其中任何一個，收益就會縮小或消失：

1. **系統受功耗限制**。省下的功耗只有在功耗上限壓住時脈時才會轉成速度。HPL 在 500–800 W 快 3.3–4.7%（低於規格表下限的 400 W 為 9.9%）；在預設的 1000 W，原生仍有 66–75% 的時間撞上限，每次求解少耗電 7.5–8.6%，但速度只在原生時脈被壓得較低的那組節點快 3.9%（那組的原生同時有溫度降頻），另一組只快 0.6%。幾乎不受限的程式（例如 GH200 在 1000 W 的 DF-MP2，只有部分時間撞上限）則主要表現為省電，速度只多 0.5–0.8%。MI350X 則不同：出廠的 1000 W 上限就壓住矩陣乘法的時脈，HPL、HPL-MxP、高斯過程與 DF-MP2 在預設設定下就快 4.8–10.3%（第 6.5、6.6 節）。第 4.6 節的模型把這個條件量化：收益取決於 R 與上限高出閒置功耗多少。

2. **被截斷的資料直接且大量地進入乘法器或資料通道**。矩陣乘法的每個輸入都要參與多次乘法，所以效益最大；N 體計算第一步就做減法，差值的尾數又是滿的；CG 與 SW4 被截斷的資料只佔記憶體流量的一部分；LAMMPS 的位能表常駐快取，幾乎不經過記憶體。

3. **有機制把誤差攔下來**。HPL 的殘差檢查、HPL-MxP 的迭代修正與 SCF 的自洽迭代都會修正或偵測截斷誤差；AI 的權重直接決定輸出品質，截斷便直接轉成精度損失。

表 7.1 彙整各類工作負載在功耗受限時的收益與主要原因。

7.2　與改變格式做法的關係

尾數歸零與 INT8 模擬 FP64、FP32/3xTF32、FP8 並非互相取代，而是互補。MI350X 上的正面比較（第 6.7 節）最清楚地說明了這一點：INT8 對 FP64 的吞吐

表 7.1　各類工作負載在功耗受限時的收益（Quantum ESPRESSO、PySCF、MFEM、SW4、LAMMPS 與 AI 只在 GH200 上量測）

工作負載	收益	主要原因
具殘差檢查的稠密求解（HPL、HPL-MxP）	GH200：HPL 在 500–800 W 快 3.3–4.7%；HPL-MxP 在 500 W 快 5.8%、600–800 W 快 1.7–2.3%。MI350X：HPL 在 1000 ／ 500 W 快 4.8 ／ 7.0%；HPL-MxP 快 5.0 ／ 19.3%（皆為對角優勢測試矩陣）	矩陣乘法主導，殘差檢查把關
高功耗的量子化學（DF-MP2）	GH200：快 4.7–7.7%，1000 W 時省電 8.4–14.3%；MI350X：20 位在 1000 ／ 500 W 快 10.3 ／ 18.6%	MP2 階段矩陣乘法佔 78%，張量建好後截斷一次即可
跨度寬且功耗高（高斯過程地統計）	GH200：快 7.5–7.9%，1000 W 時快 0–4%（依設定）、省電 8–15%；MI350X：28 位在 1000 ／ 500 W 快 7.4 ／ 11.0%	Cholesky 以矩陣乘法為主；FP32 不可用、模擬較慢
自洽迭代（Quantum ESPRESSO）	矽快 2.1%；金表面快 3.6%（約一半來自迭代減少）；總能量印出的每一位都不變	截斷誤差低於當圈的容許誤差
高斯基底量子化學（PySCF）	預設格點估計約 2%（400 W 約 4%）；細格點的纈胺黴素在 450–550 W 實測 3.3–3.5%	矩陣乘法只佔總時間 32%
記憶體受限、係數佔流量多（MFEM、SW4）	快 0.3–2.0%	其餘滿尾數的向量稀釋了效益
係數常駐快取（LAMMPS）	落在雜訊內	被截斷的資料不經過記憶體
已使用 FP8 的 AI 訓練與推論	訓練變慢；推論只是以精度換速度	FP8 只剩 3 個尾數位元

量比約為 GH200 的兩倍，GEMMul8 在內積長、上限寬鬆時大幅領先（DF-MP2 快 67%；HPL 在它本身的殘差門檻下每個上限都領先）；把共同的殘差門檻收緊到 1 的敏感度分析中，500 W 下已測設定的排序反轉（模擬 –4.0%，截斷 +7.3%）；內積只有 1024 時模擬比原生還慢；HPL 必須先縮小問題，工作空間才放得下；每種新形狀都有一次很大的初始化成本；而且整個替換 BLAS 呼叫，在某個大小驗證過的設定，換一個大小就可能安靜地出錯。GH200 上，INT8 模擬在所需精度較低（約 28 位以下），或 n 與 k 都達數千以上時較快；截斷則在瘦長矩陣、需要 32 位以上的精度，以及 GEMMul8 這類高功耗模擬遇到嚴格上限時較快。第 5.7 節的三參數公式（式 (5.1)）在測過的範圍內能描述勝負，但只以單一 m 擬合、形狀與節點差異未分開、樣本外案例都離分界很遠，目前只是經驗參考，還不足以直接作為函式庫選擇

方法的依據。FP32 與 3xTF32 的誤差隨內積長度成長，把所有 GEMM 換成它們，在長內積的平面波 DFT 上無法收斂；截斷保留 FP64 累加，誤差與內積長度無關。FP8 已經把 AI 的尾數減到 3 位，在其上再截斷只是以精度換取速度。

整個矩陣改用同一種格式的做法還有一個前提：資料的指數跨度必須夠窄。逐張量縮放的格式（FP32、FP8）取決於整體跨度，逐列切片的 INT8 模擬取決於同一列內的跨度（第 5.8 節）。DF-MP2 的張量跨度窄，所以內建的 FP32 反而比截斷快；高斯基底 DFT 依運算量加權有 24–44% 的元素（最寬的格點原子軌域矩陣達 87%）比所在列的最大值小 2^{-32} 以上，固定位數的模擬便在沒有警告的情況下算錯。不過列內跨度寬只是必要條件（高斯過程的跨度更寬卻沒有算錯），以 tile 為單位選擇格式的混合精度也能處理這類資料。這個跨度可以用攔截工具事先量測，作為風險指標；cuBLAS 的自動模式也會偵測它而改用原生 FP64，但如此便不再比截斷快（第 5.8 節）。

因此，尾數歸零的定位是：在 FP64 科學計算中，一個每個元素只多一次位元運算、精度可以 1 位為單位調整、誤差不受矩陣形狀與指數跨度影響的預設選項。在跨度窄、矩陣大而方、精度需求較低時，改用模擬或 FP32 可能更快；對於能依空間排序把小數值集中到整個 tile 的應用（例如地統計），tile 層級的混合精度能快得更多，截斷則可以疊加在它剩下的 FP64 tile 上。

7.3　限制與效度威脅

1. **兩個平台，各一個世代**。實驗在 GH200 與 MI350X 上進行，MI350X 最多用到兩個節點（16 張 GPU）。GH200 的 FP64 tensor core 相對強，是 INT8 模擬優勢最小的平台之一（Uchino 等人的 DGEMM 模擬在 RTX 5080 上快 18.5 倍，在 A100 與 GH200 上都只快 1.4 倍 [26]）；MI350X 上模擬已在長內積大幅領先，在 FP64 更弱的平台上分界會再往模擬移動。其他世代的功耗管理與散熱也不同，效益大小與模型的參數都可能改變。GEMMul8 在 GH200 使用 3.5.1 版、在 MI350X 使用 3.2.0 HIP 版，之後的版本可能改變結果。

2. **保留位數需以多組問題大小、在每個平台上驗證**。HPL 的殘差隨矩陣大小跳動：GH200 上 28 位在三組 N 通過（殘差都接近門檻）、在一組失敗；MI350X 上 28 與 30 位在每個大小都失敗。本報告以兩個平台都有數十倍以上餘裕的 36 位（含隱藏位，即 35 個尾數位元）作為操作點。HPL-MxP 保留 0 位在 GH200 可行、在 MI350X 不收斂。

3. **應用覆蓋有限**。每個科學軟體只測試一到兩個輸入系統，沒有涵蓋幾何最佳化、長時間分子動力學等情境。

4. **攔截工具的複製成本（依應用約 3–9%）**。因此攔截版本一律與「複製但不截斷」的基準比較，只有程式內版本與未修改的原生程式比較。

5. **節點差異**。同一功耗上限下，節點間的 SM 時脈可相差 13–17%，因此所有比較都在同一組節點上成對進行。

6. **散熱條件**。風扇轉速低時 GPU 會先撞到溫度上限；風扇全開時，原生的 HPL-MxP 在 1000 W 會觸發機箱的硬體功耗煞車；甲組 HPL 的原生在 1000 W 也有 15% 的取樣出現溫度降頻。MI350X 的機箱風扇會依前一次執行調整，使下一次執行低溫多達 15 °C、快多達 4.5%，交錯順序無法消除（第 6.9 節）；我們以溫度迴歸校正，並在兩個關鍵設定上以穩態與相同的熱歷史確認，但沒有權限固定風扇轉速。這些條件都會改變結論，需要一併記錄。

7. **共用叢集**。其他使用者的工具至少兩次重設叢集的功耗上限；我們在每次執行前檢查實際生效的上限，受影響的量測一律重做。

8. **功耗感測器**。功耗讀自驅動程式，沒有外接電表；在 A100/H100 上，驅動程式只取樣部分時間 [44]。模組功耗也包含 Grace CPU 與記憶體（例如 ExaGeoStat 在 CPU 產生共變異數時就與 GPU 共用同一個上限）。MI350X 以硬體的能量累加器計算能量，避免了取樣偏差，但同樣不是外接電表。本報告的耗電數字都是同一量測路徑下的成對相對值，絕對能量仍需以外接電表驗證。

9. **訓練的收斂點有雜訊**。LoRA 的達標點即使設定完全相同也會相差一個評估區間，而我們只用了兩個亂數種子，所以只把它當作「FP8 沒有再截斷的空間」的佐證。

10. **HPL-MxP 的測試矩陣**。主要結果使用對角優勢的矩陣（對角元等於該列非對角元絕對值的和）與近似的對角分解，規則只允許這種矩陣用於測試 [42]。在 MI350X 上另以我們自己的非對角優勢產生器（對角元 $\sqrt{n}$）與精確分解驗證，3 位仍通過、仍快 5.0 ／ 16.3%（第 6.5 節），但兩者都不是官方產生器。

11. **樣本數與節點**。MI350X 的區間以分析單位（視窗、整節點的一次執行、8 張 GPU 的一輪）計算，每個實驗都在第二個節點上重複，但單位數只有 4–8 個。GH200 的 HPL 與 HPL-MxP 每個上限有兩組節點共 6 對，DF-MP2 與高斯過程每格 6 輪，Quantum ESPRESSO 先暖機後 6 對，都附 95% 區間；但區間把同

一工作或同一組節點內的各對當成獨立樣本，可能偏窄。HPL 換一組節點後，600–800 W 的加速少了 0.5–0.9 個百分點，大於同一工作內各對的差異（最多 0.6 個百分點），而第二組量測當天的網路較慢，節點與網路狀態的影響無法分開；節點組之間的差異是比單次量測雜訊更大的不確定性來源（HPL-MxP 的兩組在 600–800 W 則只差 0.2–0.3 個百分點）。分界點掃描的四個 n 值分別在不同節點上量測，形狀效應與節點差異無法分開，是實驗設計上的缺陷。

12. **功耗上限的意義**。1000 W 是預設也是最高上限，並不代表不受限：HPL 與 FP64 矩陣乘法在 1000 W 仍有部分時間撞上限；限電器依據的時間尺度不明；低於 450 W 的上限超出規格表的可設範圍 [41]。

13. **模型與推論**。功耗模型的外部驗證只有兩個應用（16 個點），高斯過程在每個上限都被低估 0.6–3.6 個百分點；MI350X 上歸因於時脈延展的降速是推論，沒有直接量測。

14. **機制實驗的範圍**。填值對照只做了保留 0 位與 20 位，500 W 只在一個節點上量測兩次；保留 0 位時「數值為 0」的額外省電原因尚未驗證。

15. **收益不全來自功耗**。在 Quantum ESPRESSO 金表面上，固定 24 位時 Davidson 迭代數與原生相同（138 次），SCF 快 3.2%，這是等工作量下的功耗效果；自動調整只需 126 次（少 9%），SCF 快 4.5%，多出的收益來自迭代數，原因尚未查明（第 5.4.3 節）。

7.4　規則合規性

TOP500 的 HPL 要求以完整的 64 位元浮點運算求解，明文排除以低精度求解再迭代修正，以及以整數運算模擬浮點的方法 [45]；截斷運算元後的 HPL 也不符合這個要求。HPL-MxP 自 2025 年 6 月起規定最低精度為 16 位元浮點，且對角優勢的測試矩陣只能用於測試與調校 [42]；保留 3 位的 BF16 實質上只有 12 位元，我們的測試矩陣也不符合正式規定。因此本報告所有 HPL/HPL-MxP 數字都是研究結果，並非正式成績。MLPerf Inference 的封閉組允許數學上的量化、禁止剪枝，尾數歸零屬於前者 [46]；MLPerf Training 的權重能否截斷需依各項規則判斷。科學軟體沒有這類限制，判準是物理結果是否在所需的精度之內。

第八章　結論與未來工作

8.1　結論

本專題在兩家廠商的現役 GPU（NVIDIA GH200 叢集與兩個 8 張 AMD MI350X 的節點）上，以純軟體將用不到的尾數位元歸零，回答了第 1.4 節的四個問題：

1. **機制：**在兩個平台的矩陣引擎、FP64 向量單元與記憶體搬移上，以全 1 或固定樣式填入被捨去的位元，都保留了大部分的效益（41–97%），所以省電主要來自低位元不再翻轉，數值為 0 再多省一些。在數值與 1 的個數都固定時，只改變運算元的排列順序就讓每次運算的能量相差 8–9 個百分點，相鄰運算元之間的翻轉數解釋 83% 的變異，所以主因是相鄰運算元之間變化的位元。MI350X 上，截斷在鎖定時脈下還提高了有效時脈，可能來自電路層級的時脈延展。

2. **效益：**在功耗受限且截斷實作於程式內時，以矩陣乘法為主、具正確性檢查的完整程式在兩個平台上都獲得正面收益。GH200 上，HPL（調參後）保留 36 位有效位數在 500–800 W 快 3.3–4.7%，HPL-MxP 保留 3 位在 500 W 快 5.8%，Quantum ESPRESSO 依收斂門檻調整位元數，在總能量印出的每一位都不變的情況下快 2.1–3.6%，DF-MP2 在 500–700 W 快 4.7–7.7%，高斯過程快 7.5–7.9%；在預設的 1000 W 主要表現為省電（HPL 每次求解少耗電 7.5–8.6%）。MI350X 的預設上限就壓住時脈，所以在出廠設定下 HPL 快 4.8 ± 0.3%、HPL-MxP 5.0 ± 0.4%、高斯過程 7.4 ± 0.6%、DF-MP2 10.3 ± 0.6%，在 500 W 快 7.0–19.3%（兩節點合併的分析單位）。HPL-MxP 的結果只適用於對角優勢的測試矩陣。代價是必須在每個平台上以多組問題大小驗證保留位數（HPL 28 位在 GH200 大多通過、在 MI350X 全部失敗），並使用最近捨入。

3. **預測：**若截斷只改變切換電容，截斷版本在上限 P 下就等於原生版本在 $P_0 + (P - P_0)/R$ 下。以實測的閒置功耗作為 P_0，這個模型對兩個平台各 42 個矩陣乘法的點平均誤差為 1.5 與 2.1 個百分點，對兩個沒有參與建模的應用為 1.6 個百分點（最大 3.6）。因此只要知道平台的閒置功耗與應用的原生曲線，一次鎖頻量測就能估計截斷版本在每個上限下的效能，省去大部分的上限掃描；但

在兩種方法只差一兩個百分點的交叉點附近，這樣的誤差還不足以可靠地選出較快的方法。

4. **適用範圍：**尾數歸零與 INT8 模擬 FP64 互補。MI350X 上，模擬在內積長、上限寬鬆時大幅領先（DF-MP2 快 67%；HPL 在它本身的殘差門檻下，1000 W 快 34%、500 W 快 12%，都勝過截斷）；把共同的殘差門檻收緊到 1（這是敏感度分析，不是 HPL 的要求），500 W 下已測設定的排序反轉（截斷 +7.3%，模擬 −4.0%）。模擬在內積短時比原生還慢，需要的工作空間迫使 HPL 縮小問題，在列內指數跨度寬時會安靜地損失精度（GH200 上的高斯基底 DFT 錯 52.5 mE_h）；我們所用的 GEMMul8 HIP 版本另有一個尺寸相關的實作缺陷。把所有 GEMM 換成 FP32/3xTF32 在長內積上無法取代截斷；在已使用 FP8 的 AI 上，FP8 已取得幾乎全部的好處。本報告比較的兩種方法中，只有截斷適用於 BF16 與 FP8。只用矩陣乘法量測擬合的逐次呼叫選擇規則，在完全沒有參與擬合的 DF-MP2 上 99.3% 的呼叫選對，只比逐次最佳慢 0.02%。

另外，在功耗受限的量測中，溫度是必須控制的干擾：MI350X 的機箱風扇依前一次執行調整，使下一次執行快多達 4.5%，交錯順序無法消除；我們以溫度迴歸校正，並以穩態與相同的熱歷史兩種受控實驗確認了校正後的數值。

總結而言，在功耗受限的 FP64 科學計算中，尾數歸零是一個每個元素只多一次位元運算、不需要新硬體、精度可以 1 位為單位調整、而且能事先預測收益的調整手段，最適合具殘差修正或自洽迭代的求解器。隨著吞吐量成長得比功耗預算快，在上限下運作會成為常態，翻轉用不到的位元所消耗的能量也會成為效能的一階參數。

8.2　未來工作

更多硬體：在 FP64 相對弱的平台（例如 Blackwell 的消費級或資料中心 GPU）與其他世代上重做分界掃描與 HPL，檢驗模型與分界能否移植；在多節點 MI350X 上重做 HPL 與 HPL-MxP。

正式規則下的 HPL-MxP：改用規則允許的矩陣產生器，重做保留位數的掃描。

函式庫整合：把「依形狀、上限與列內跨度選擇截斷或模擬」的規則實作於矩陣乘法函式庫，使截斷不需要額外複製；功耗模型可以提供上限這一側的判斷。

更多自適應求解器：把「依容許誤差決定位元數」推廣到其他迭代法，例如 Krylov 求

解器的內層與混合精度迭代修正；也可以利用資料的位元資訊量事先估計需要保留的位數 [22]，以減少試驗。

AI 的第二項研究：在功耗上限下，降低精度能否縮短達到目標品質的時間與能耗，需要另外設計實驗（附錄 C 只是初步的負面結果）。

量測：以外接電表交叉驗證絕對能量；在可以固定風扇轉速的環境下重做溫度控制實驗。

參考文獻

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, Apr. 2024.

[4] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[5] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[6] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[7] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[8] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[9] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[10] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[11] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[12] ——, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[13] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[14] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2, Sep. 27, 2022.

[15] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[16] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[17] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[18] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013, pp. 1–12.

[19] A. Dawson and P. D. Düben, "rpe v5: An emulator for reduced floating-point precision in large numerical simulations," *Geoscientific Model Development*, vol. 10, pp. 2221–2230, 2017.

[20] N. J. Higham and S. Pranesh, "Simulating low precision floating-point arithmetic," *SIAM Journal on Scientific Computing*, vol. 41, no. 5, pp. C585–C602, 2019.

[21] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[22] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[23] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, “Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications,” *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[24] H. Ootomo, K. Ozaki, and R. Yokota, “DGEMM on integer matrix multiplication unit,” *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[25] K. Ozaki, Y. Uchino, and T. Imamura, “Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique,” arXiv preprint arXiv:2504.08009, 2025.

[26] Y. Uchino *et al.*, “High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1824–1831.

[27] NVIDIA, “cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1),” https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[28] A. Schwarz *et al.*, “Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme,” in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[29] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, “Acceleration of PageRank with customized precision based on mantissa segmentation,” *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[30] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, “ExaGeoStat: A high performance unified software for geostatistics on manycore systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2771–2784, 2018.

[31] ——, “Geostatistical modeling and prediction using mixed precision tile Cholesky factorization,” in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[32] S. Abdulah *et al.*, “Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[33] Q. Cao *et al.*, “Reshaping geostatistical modeling and prediction for extreme-scale environmental applications,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[34] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, “Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling,” in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[35] S. Abdulah *et al.*, “Boosting earth system model outputs and saving PetaBytes in their storage using exascale climate emulators,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–12.

[36] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, “Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion,” in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[37] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, “Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library,” *Journal of Chemical Theory and Computation*, vol. 6, no. 1, pp. 135–144, 2010.

[38] J. Woo, S. Kim, and W. Y. Kim, “Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units,” *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[39] H. Huang, W. Shao, and J. Hammond, “Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators,” arXiv preprint arXiv:2601.08077, 2026.

[40] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, “Reducing numerical precision requirements in quantum chemistry calculations,” *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[41] NVIDIA, “NVIDIA GH200 Grace Hopper superchip datasheet,” Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[42] HPL-MxP, “HPL-MxP rules,” https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.

[43] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, “Numerical behavior of NVIDIA tensor cores,” *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[44] Z. Yang, K. Adamek, and W. Armour, “Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU’s built-in power sensor,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24)*, 2024.

[45] TOP500.org, “The Linpack benchmark,” https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.

[46] MLCommons, “MLPerf inference rules, section 8.1: Weight definition and quantization,” https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc, accessed Oct. 5, 2026.

[47] J. Dongarra and P. Luszczek, “HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation,” *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[48] P. Giannozzi *et al.*, “Quantum ESPRESSO toward the exascale,” *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[49] X. Wu *et al.*, “Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks,” *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[50] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, “Introducing GPU acceleration into the Python-based simulations of chemistry framework,” *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.

附錄 A　實驗環境、程式位置與重現方式

A.1　實驗環境

表 A.1　實驗環境與軟體版本

項目	內容
硬體	14 個節點，各一顆 NVIDIA GH200 480GB（aarch64），200 Gb/s InfiniBand（2026 年 10 月 6 日起的多節點工作把 NCCL 與 MPI 固定在 `mlx5_0`，見第 5.3.1 節），Slurm 分區 `compute`
編譯器與函式庫	NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）；ExaGeoStatCPP 連結 cuBLAS 13.6
基準	HPL（rocHPL 的 CUDA 移植，分支 `feat/cuda-port`）、HPL-MxP（分支 `perf/h200-mantissa`）[47]
科學軟體	Quantum ESPRESSO 7.6 [48]、GPU4PySCF 1.8.1 [49, 50]、MFEM（提交 `c362dea`）、SW4（提交 `52b51c9`）、LAMMPS `stable_22Jul2025_update6`、GEMMul8 3.5.1
AI	MLPerf Training Llama 2 70B LoRA，NVIDIA v5.0 實作（8 節點）
MI350X 硬體	2 個節點（`p01-r01-n01`、`n02`），各 8 張 AMD Instinct MI350X OAM（gfx950，288 GB HBM3E），Slurm 分區 `gpu`；兩節點的家目錄不共用
MI350X 軟體	ROCm 7.2.3（rocBLAS、hipBLASLt、rocSOLVER、RCCL）、amdgpu 6.16.13、Open MPI 5.0.7 + UCX；rocHPL 分支 `feat/adaptive-mixed-storage`（提交 `37d0475`）、HPL-MxP 與 GH200 同一份原始碼（`src_mi350/`）、GEMMul8 3.2.0 HIP 版（提交 `833e576`）與 3.5.4（`f8fdf0b`，Pareto、形狀掃描、逐次呼叫選擇、溢位與初始化成本的檢查）、PySCF 2.14；兩節點以 `srun --mpi=pmix` 與 `mi350/env_multinode.sh` 執行
分析	Python（pandas、numpy、scipy、matplotlib），虛擬環境 `gh200/.venv` 與 `mi350/.venv`

A.2　程式位置

表 A.2 列出本專題的程式與資料位置（路徑相對於專案根目錄）。

表 A.2　程式與資料位置

路徑	用途
`gh200/bench/`	cuBLAS 尾數矩陣乘法微基準（第 5.2、5.7 節）

表 A.2（續）

路徑	用途
gh200/shim/mantshim.cu	cuBLAS 攔截函式庫：截斷（MANT_BITS、MANT_ROUND）、cuBLAS 模擬、單精度替換（MANT_F32）、cuBLAS 自動模式與位數紀錄（MANT_EMU_DYN、MANT_EMU_LOG）、運算元統計與傾印
apps/q-e/KS_Solvers/Davidson/	Quantum ESPRESSO 的程式內截斷與自動調整規則（QE_MANT_ADAPT）
rocHPL/src/blas/HPL_quantize_device.cpp	HPL 的運算元捨入（HPL_QUANT_A_BITS 等）
rocHPL-MxP/src_mi350/hplmxp.h	HPL-MxP 寫回 BF16 時的歸零（編譯時指定 MANT_BITS）
apps/（見 apps/**建置日誌**.md）	MFEM、SW4、LAMMPS 的係數截斷插入點與建置方式
apps/pyscf-inputs/mp2_variants.py	DF-MP2 各變體：程式內截斷 DF 張量、內建 FP32、cuBLAS 模擬（第 5.5 節）
apps/geostat/gp_mle.py	高斯過程對數概似與最大概似估計：截斷、cuBLAS 模擬、FP32、Morton 排序與 tile 混合精度（第 5.6 節）
apps/exageostatcpp/、apps/patches/	ExaGeoStatCPP（submodule）與在 GH200、CUDA 13 上建置所需的修正（建置日誌第 9 節）
mi350/bench/	MI350X 的尾數矩陣乘法微基準（rocBLAS ／ hipBLASLt，可經 GEMMul8 攔截）與非矩陣乘法 FP64 核心（第 6.2、6.3 節）
mi350/apps/	MI350X 上的應用：高斯過程 gp_chol.hip、DF-MP2 張量準備 mp2_prep.py 與收縮 dfmp2.hip、GEMMul8 溢位的最小重現程式 g8_repro.hip
mi350/scripts/、mi350/slurm/run.sbatch	MI350X 的實驗腳本（e1_gemm_tdp.sh、e1b_fixedclk.sh、hpl_tdp.sh、gp_tdp.sh、mp2_tdp.sh、lockclk_apps.sh、g8_threshold.sh 等，參數在各檔開頭的註解）
mi350/tools/	功耗上限工具 pcap.sh 與 amdsmi 取樣器 smi_sampler.py（含能量累加器）

表 A.2（續）

路徑	用途
mi350/analysis/	e1_summary.py、hpl_summary.py、gp_summary.py、mp2_summary.py、功耗模型 power_model.py、model_eval.py、model_apps.py、分析單位的區間 apps_units.py、HPL 共同殘差門檻的比較 hpl_threshold.py、機制對照 mech_toggle.py、逐次呼叫選擇 selector.py、mixed-effects mixed_effects.py、GP 最大概似 gp_mle_summary.py、溫度校正 temp_adjust.py、本報告第六章的圖 report_figs_zh_mi350.py
mi350/results/、mi350/logs/、mi350/實驗日誌.md	MI350X 的彙整 CSV、原始紀錄（節點 n02 的在 logs/n02/）與實驗過程紀錄
paper/	英文論文（main_sc.tex 與補充資料 supplement.tex、refs.bib、figs/）
paper/zh_report/	本報告的 LaTeX 原始檔（report.tex、chapters/、figs/）
gh200/refs/	各輪文獻查證紀錄（round4_refs.md、round5_refs.md）
gh200/slurm/	工作腳本：e1_gemm_tdp.sbatch（微基準與形狀掃描）、hpl_tdp.sbatch（HPL/HPL-MxP）、sci_sweep.sbatch（科學軟體）、app_tdp.sbatch（第三方軟體）、mp2_tdp.sbatch（DF-MP2）、gp_tdp.sbatch（高斯過程）、exageostat_tdp.sbatch（ExaGeoStat）、hpl_tune.sbatch（HPL 調參）、hpl_shim.sbatch（單 GPU HPL 經攔截工具：逐列跨度與模擬）
gh200/analysis/	summarize.py、sci_summary.py、app_summary.py、crossover.py、qe_adapt_ci.py、range_sweep.py、span_report.py、mp2_summary.py、gp_summary.py、exa_summary.py、paper_figures.py（英文論文的向量圖）、report_figs_zh.py（本報告的圖）；圖 5.11 的座標來自 gh200/results/range_map_points.csv
gh200/results/	所有彙整後的 CSV；圖下方、表註與內文列出的結果檔都在此目錄
gh200/實驗日誌.md、gh200/應用實驗日誌.md	完整的實驗過程紀錄（含失敗與重做）

A.3　重現方式

GH200 的工作都以 Slurm 送出；MI350X 的工作除最早幾個直接在節點上執行外，也以 Slurm 送出。腳本只在自己獨佔的節點上設定功耗上限，結束時恢復為 1000 W，並在每次執行前檢查實際生效的上限。以下為 GH200 的三個例子：

```
module load nvhpc/26.5
cd gh200
# 分界點掃描（第 5.7 節）
sbatch -w work1 --export=ALL,TDPS="1000 575 450",SHAPES="8192x4096x16384
 ↪  8192x256x81920",CONFIGS="fp64:52 fp64:36 fp64emu:32
 ↪  gemmul8:10",WARMUP=2,SECONDS_PER=6 slurm/e1_gemm_tdp.sbatch
.venv/bin/python analysis/crossover.py < 工作編號 ...>
# Quantum ESPRESSO 自動調整位元數（第 5.4 節）
sbatch -w work2
 ↪  --export=ALL,APP=qe,CASE=si1000,INP=si1000.in,TDPS=550,VARIANTS="native
 ↪  adapt0:nearest src28:nearest src28:nearest adapt0:nearest native"
 ↪  slurm/sci_sweep.sbatch
.venv/bin/python analysis/sci_summary.py logs/sci/< 工作名>_< 編號>
.venv/bin/python analysis/qe_adapt_ci.py < 彙整 CSV> src28:nearest
# 第三方軟體（第 5.1.2 節）
sbatch -w work10 --export=ALL,APP=mfem slurm/app_tdp.sbatch
.venv/bin/python analysis/app_summary.py logs/apps/< 工作名>_< 編號>
```

MI350X 的例子（先 `source mi350/env.sh`，它會取消系統預設的 `HIP_VISIBLE_DEVICES`）：

```
# HPL-MxP 保留 3 位，成對 ABBA、每次執行前 3 分鐘的熱預處理（第 6.8 節）
sbatch -w p01-r01-n01 --export=ALL,SCRIPT=hpl_tdp.sh,BENCH=mxp,TAG=mxpprec
 ↪  ond,CAPS=800,VARIANTS="native 4 3",REPEAT=2,PAIRWISE=1,PRECOND=180
 ↪  mi350/slurm/run.sbatch
mi350/.venv/bin/python mi350/analysis/hpl_summary.py mxpprecond_<日期_時間>
mi350/.venv/bin/python mi350/analysis/temp_adjust.py mxpprecond_<日期_時間>
# 功耗模型的外部驗證（第 6.4 節）
mi350/.venv/bin/python mi350/analysis/model_apps.py
```

圖下方的資料來源、表註與引用各表的段落列出對應的 Slurm 工作編號與結果檔，原始紀錄可以在 `gh200/logs/` 與 `gh200/results/` 找到。本報告的圖與 PDF 以下列指令重新產生（在專案根目錄執行；字型約 90 MB，不納入版本控制）：

```
paper/zh_report/fetch_fonts.sh                          # 下載並檢查字型
gh200/.venv/bin/python gh200/analysis/report_figs_zh.py
 ↪  paper/zh_report/fonts   # 第五章的圖
mi350/.venv/bin/python mi350/analysis/report_figs_zh_mi350.py
 ↪  paper/zh_report/fonts   # 第六章的圖
paper/zh_report/build.sh                                # tectonic 或 xelatex +
 ↪  bibtex → report.pdf
```

附錄 B　結果與 Slurm 工作對照

正文中各項結果對應的 Slurm 工作列於下表；原始輸出在 `gh200/logs/`，彙整後的結果檔在 `gh200/results/`，完整的執行過程（含失敗與重做）記錄在 `gh200/實驗日誌.md` 與 `gh200/應用實驗日誌.md`。

表 B.1　各結果對應的 Slurm 工作

結果	Slurm 工作
省電機制的填值對照（圖 5.1）	1752–1753、1762、1815、1858
矩陣乘法微基準（圖 5.3）	1667–1669
矩陣乘法的誤差模型（第 4.7 節）	1664
原生 HPL 的調參（第 5.3.1 節）	3303、3306
調參後 HPL 的功耗上限掃描（表 5.2、圖 5.4）	甲組 3315；乙組 3582（3490、3579 因網路問題取消，見第 5.3.1 節）
HPL 多組 N 的殘差（表 5.3）	1820（N = 286720 取自 3315、3582）
未調參 HPL 的功耗上限掃描	1670
HPL-MxP 的功耗上限掃描（圖 5.5、表 5.4）	甲組 1671；乙組 3583
HPL-MxP 的功耗煞車診斷、多組 N	1674、1675；1821、1859
Quantum ESPRESSO 金表面攔截掃描（圖 5.6）	1729、1738
科學軟體所需的位數（表 5.5）	矽 1728；纈胺黴素 1730、1745（門檻 10^{-7}）；紫杉醇 1731
Quantum ESPRESSO 攔截版的效能（表 5.6 最後一列）	1737
GPU4PySCF 的剖析（矩陣乘法佔時間的比例）	1713、1717
以單精度取代 FP64 矩陣乘法（第 5.4.4 節）	金表面 1928、矽 1930
Quantum ESPRESSO 自動調整（表 5.6）	先暖機：金表面 3479、矽 3488；先前未暖機：金表面 1758、2009，矽 1759、2010
Quantum ESPRESSO 不受功耗限制的對照執行	2003
方陣上的截斷、模擬與單精度（圖 5.7）	1894
纈胺黴素真實形狀（圖 5.8）	1894、1896、1897

表 B.1（續）

結果	Slurm 工作
纈胺黴素矩陣乘法的形狀統計	1895
單精度做法的誤差與內積長度（表 5.11）	1934
分界點掃描（圖 5.9）	2005–2008
非矩陣乘法核心（圖 5.2）	1815、1858、1860
MFEM、SW4、LAMMPS（表 5.1）	1992、1995、1996
LoRA 微調（圖 C.1）	1732、1736、1760 起、1901、1904、1990、1993；捨入方式 1922、1959；分層截斷 2011、2020、2027
語言模型推論的速度	1849
語言模型推論的精度（圖 C.2、表 C.1）	困惑度 1817、1823、1829–1831、1837；注意力 1827、1828；鑰匙檢索 1841；影片 1852、1853；BERT 1857；解碼的省電 1903
DF-MP2（表 5.7）	精度掃描 2037；功耗上限 2038、3489
合成資料的指數跨度（圖 5.10）	2034–2036
真實應用的跨度與模擬（表 5.12、圖 5.11）	纈胺黴素 2046、2195；QE 2136、2137；cuBLAS 自動模式 3292；HPL 3304、3305；高斯過程 3248、3291；各應用的逐列跨度統計 2128–2131
寬跨度 DFT 在功耗上限下（第 5.6 節）	2821
高斯過程對數概似評估（表 5.8）	3248、3501（3482 範圍參數設錯，已取消）
高斯過程的舊版截斷實作；小規模 FP32 估計	2827；3213
高斯過程最大概似估計、tile 混合精度（表 5.9）	3249；3244
ExaGeoStatCPP（表 5.10）	3229

MI350X 的結果對應的紀錄列於表 B.2。紀錄名稱為「標籤 _ 日期 _ 時間」，原始輸出在 `mi350/logs/`（節點 n02 的在 `mi350/logs/n02/`），彙整後的結果檔在 `mi350/results/`，執行過程記錄在 `mi350/實驗日誌.md`。括號內為 MI350X 叢集的 Slurm 工作編號；沒有編號的是在設定 Slurm 之前直接在節點上執行的。

表 B.2　MI350X 各結果對應的紀錄

結果	紀錄（Slurm 工作）
矩陣乘法各上限（圖 6.1、表 6.5）	n01 `e1_1028_1709`；n02 `e1n02_1029_0502`（13）

表 B.2（續）

結果	紀錄（Slurm 工作）
填值對照與鎖頻下的時脈（表 6.1）	`e1b_clk900`（20）、`e1b_clk1300`、`e1b_clk1300b`（23）、`e1b_clk1700`（21）、`e1c_lim1300`（24）、`e1c_free`（25）；非矩陣乘法核心 `nongemm`（34）
功耗模型（圖 6.2、表 6.3）	上列矩陣乘法與鎖頻紀錄；外部驗證 `mp2lock1300`、`gplock1300`（67）
HPL 的安全位數（表 6.4）	`hplbits262k`、`hplbits393k`、`hplbits540k`（17–19）
HPL 各上限	n01 `hpl_1029_0336`；n02 `hpln02_1029_1045`（15）
HPL-MxP 各上限	n01 `mxp_1028_2359`；n02 `mxpn02_1029_0810`（14）；800 W 成對重跑 `mxp800pair`（50）；精確對角分解 `mxpexact`（65）
DF-MP2	張量準備 `mp2prep`（31、58）與驗證 `mp2val`（29、30）；n01 `mp2_1029_1607`（32）；n02 `mp2n02`（59）；GEMMul8 `mp2emu_1029_1741`（40）
高斯過程	n01 `gp_1029_0731`；n02 `gpn02_1030_0521`（57）；GEMMul8 $n = 80000$ `gpemu`（52）、$n = 64000$ `gpemu64k`（63）
截斷與 GEMMul8 的分界（圖 6.4）	`xover_1029_1508`（28）
HPL 與 GEMMul8（表 6.6）	最佳設定 `hpl3B`（49）；相近精度 `hpl3B2`（64）；1×8 工作空間不足 `hpl3A`（43、48）、`hpl3A2`（54）
GEMMul8 在 $mn = 2^{32}$ 的錯誤	`g8thr`（66）
指數範圍、模擬內再截斷	`range_*`（53）；`synergy`（51）
穩態與熱預處理（第 6.9 節）	`steady_1030_0901`（68）；`mxpprecond_1030_0944`（69）
位元變化對照（表 6.2）	`mech1300`（102）、`mechb1300`（109）、`mechb1700`（110）
高斯過程的三參數最大概似估計	`gpmle`（111）
DF-MP2 每次呼叫的時間	`mp2call`（112）
GEMMul8 3.5.4 的溢位與初始化成本	`g8thr354`（122）、`g8cold`（123）
相同精度的比較（圖 6.5）	`pareto`（104）
逐次呼叫選擇的形狀掃描（表 6.8）	`shpA`（159）、`shpB`（160）

附錄 C　AI 工作負載的細節

C.1　訓練：Llama 2 70B LoRA 微調

MLPerf Training 比較的是「達到目標品質所需的時間」（驗證損失 ≤ 0.925）。基底權重已經是 FP8（只有 3 個尾數位元），我們在載入時再截斷一次，使用 8 個節點（圖 C.1）。

兩個亂數種子、四組配對（575 W 一組、800 W 三組）的結果一致：截斷到 2 位的驗證損失從第一次評估起就高 0.008–0.010，需要多花 2–3 個評估區間才達標（例如 seed 5678：原生 3072 個樣本、22.74 分鐘；截斷 4224 個樣本、31.56 分鐘），而每一步只快約 1%。截斷在載入時就削弱了基底模型，LoRA 無法補回。最近捨入與隨機捨入也沒有改善（分別需要 5376 與 4224 個樣本）。只截斷部分層的結果相同：只截斷 MLP 需要 4224 個樣本，與全部截斷相同；只截斷注意力需要 3840 個，退化落在雜訊內，但注意力只佔線性層權重約 18%，可以節省的部分本來就很少。只截斷注意力的設定以完全相同的參數再跑一次，第一次評估幾乎相同（0.9428 對 0.9425），達標點卻差一個區間（3840 對 4224），所以我們以第一次評估的差距作為主要證據。結論是：對已經使用 FP8 的訓練，尾數已經沒有可以再節省的空間。

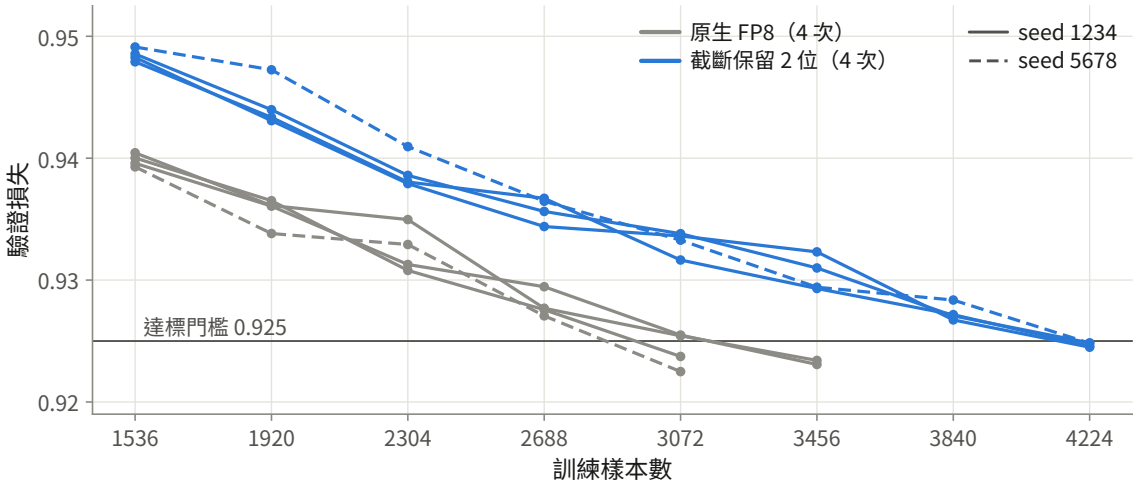


圖 C.1　Llama 2 70B LoRA 原生 FP8 與截斷保留 2 位的驗證損失隨訓練樣本數的變化（實線為 seed 1234、虛線為 seed 5678；水平線為達標門檻 0.925）。

資料來源：Slurm 工作 1732、1736、1760 起、1901、1904、1990、1993（分層截斷 2011、2020、2027 只列於內文）；`gh200/logs/lora`

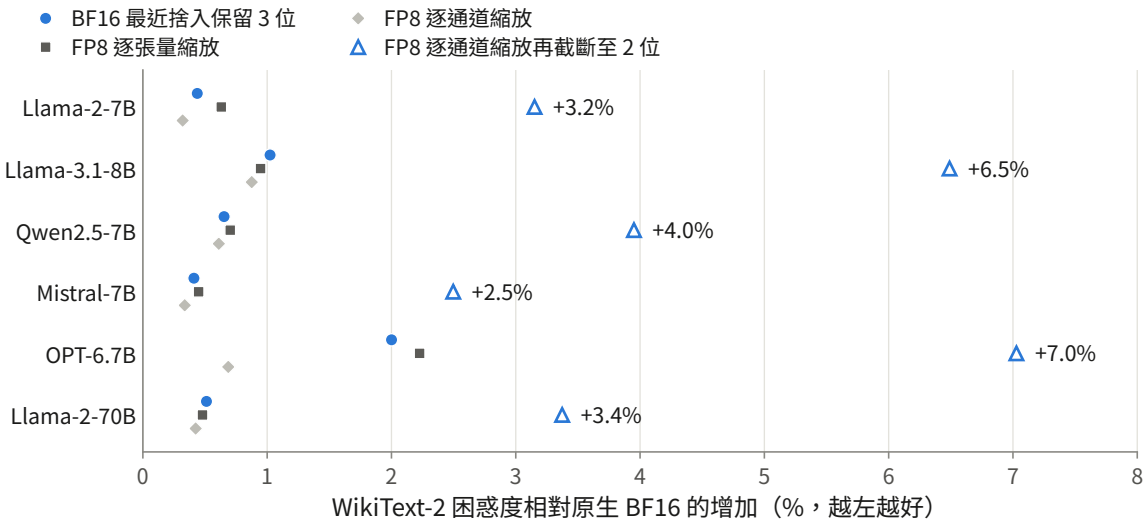


圖 C.2　六個語言模型在各格式下的 WikiText-2 困惑度相對原生 BF16 的增加（越左越好）。資料來源：`gh200/results/infer_ppl.csv`（70B 為逐層評估）

表 C.1　BF16 最近捨入保留 3 位與 FP8 在 FP8 容易出問題的場景下的比較

場景	BF16 最近捨入保留 3 位	FP8
注意力（2k/8k 上下文的困惑度增加）	+0.49%/+1.06%	+0.52%/+1.08%
長上下文鑰匙檢索（8k–32k）	100%	100%
BERT-large SQuAD（MLPerf 99.9% 門檻）	通過	通過
CogVideoX-2B 影片（相對原生的 PSNR）	20–29 dB	18–38 dB

C.2　推論：BF16 截斷與 FP8 的比較

我們尋找兩種可能的機會：比 FP8 更快、精度又不會差太多的做法；或 BF16 加上尾數截斷比 FP8 更好的場景（圖 C.2）。

以最近捨入保留 3 位的 BF16 與 FP8 的尾數位數相同，矩陣乘法誤差也相當（均勻資料上 3.3% 對 3.6%；截斷需保留 4 位才接近，約 4.4%）。我們接著在文獻中 FP8 最容易出問題的場景比較兩者，結果都打平（表 C.1）。

另一方面，FP8 的矩陣乘法比 BF16 快約 1.8 倍（575 W 實測）。在不改用結構化稀疏或 INT4 權重的前提下，更快的路只剩在 FP8 之上再截斷：以 Llama 2 70B 的真實權重，兩邊都截斷到 2 位快 3.2–8.2%，但困惑度增加 3.4%，速度與精度大致以 1 比 1 交換；只截斷權重時，解碼（decode）階段的省電也只有 1.5–5.5%。

小結　在 Hopper 上，FP8 已經取得「減少尾數位元」幾乎全部的好處。尾數歸零的價值在 FP64 科學計算，而不在已經使用 FP8 的 AI。